

Package ‘DescToolsX’

August 11, 2026

Title Tools for Descriptive Statistics - New Generation

Version 0.0.0.938

Description Provides a collection of functions for descriptive statistics, data analysis, and statistical testing. The package is a redesigned and streamlined successor to DescTools, with a focus on improved consistency, performance, and maintainability.

Encoding UTF-8

Depends R (>= 4.6.0),
pharos, lumen, bedrock

License GPL (>= 2)

LinkingTo Rcpp, RcppParallel, RcppArmadillo

Roxygen list(markdown = TRUE)

Config/testthat/edition 3

Imports stats, boot, cli, withr, stringi, mvtnorm, Rcpp, RcppParallel

Suggests goftest, testthat (>= 3.0.0), R.rsp, readr, vegan, VGAM, nnet,
MASS, rstudioapi, gld, lme4, hexbin, ppcor, forecast, MuMIn,
survival, pons, dbscan, prettyunits

VignetteBuilder R.rsp

URL <https://andrisignorell.github.io/DescToolsX/>,
<https://github.com/AndriSignorell/DescToolsX/>

BugReports <https://github.com/AndriSignorell/DescToolsX/issues>

Config/roxygen2/version 8.1.0

Contents

abstract	4
addMonths	6
Agreement	7
Association	9
as_ym	10
atkinson	11
attach-detach-aliases	13
auc	14
blandAltmanData	16

boxCox	17
boxCoxLambda	19
brierScore	20
ccc	22
coefVar	24
cohenD	27
cohenH	29
cohenKappa	30
concepts	33
conDisPairs	34
conf	36
ConfidenceIntervals	39
constants	40
contCoef	40
corPart	42
corPolychor	44
countWorkDays	46
cramerV	47
cronbachAlpha	49
cStat	52
cut.integer	53
cutAge	54
cutQ	55
date-time-predicates	57
date_functions	58
desc	63
desc.Date	66
desc.factor	68
desc.nn	70
desc.nq	72
desc.numeric	73
desc.qn	75
desc.qq	76
desc.ts	77
DescToolsX	79
divCoef	82
entropy	83
etaSq	84
expFreq	87
extremes	88
findCorrX	89
Formulas	91
freq	91
freq2D	93
generation	95
gini	96
gkTau	97
glassDelta	100
gmean	102
herfindahl	104
hmean	105
hodesLehmann	107

hoeffdingD	109
huberM	112
icc	114
impute	116
imputeKnn	117
iqrX	119
isConfusionTable	120
kappaM	121
keepSig	123
kendallW	124
krippAlpha	126
kurt	129
lambda	131
lc	132
lof	135
logSt	136
madX	138
mae	140
mape	141
meanAD	142
meanSE	143
meanX	144
medianX	145
modeX	147
mse	149
mutInf	150
nmae	152
nmse	153
normalizeToConfusion	154
oddsRatio	156
ordAssocs	159
outlier	162
pabak	163
pearsonCor	165
percAgreement	167
percTable	168
phi	171
print.BlandAltman	172
print.Desc.Date	173
print.Desc.qq	174
quantileX	177
randolphKappa	179
rangeX	180
raterFrame	181
relRisk	183
rmse	185
rosenbluth	186
scaleX	187
setDescToolsXOption	189
simpson	190
skew	191
smape	193

spearmanCor	194
theil	196
time-conversions	197
tOne	198
tschuprowT	202
tukeyBiweight	204
uncertCoef	206
varX	207
yeoJohnson	209
yule	211
zodiac	213

Index	215
--------------	------------

abstract	<i>Display Compact Abstract of a Data Frame</i>
----------	---

Description

Compactly display the content and structure of a `data.frame`, including variable labels. `str()` is optimized for lists and its output is relatively technical, when it comes to e.g. attributes. `summary()` on the other hand already calculates some basic statistics.

Usage

```
abstract(
  x,
  sep = ", ",
  zeroForm = ". ",
  maxLevels = 5,
  maxVars = Inf,
  truncate = TRUE
)

## S3 method for class 'Abstract'
print(x, width = NULL, truncate = NULL, print.gap = 2, ...)
```

Arguments

<code>x</code>	a <code>data.frame</code> to be described
<code>sep</code>	the separator for concatenating the levels of a factor
<code>zeroForm</code>	a symbol to be used when a variable has zero NAs
<code>maxLevels</code>	integer; maximum number of factor levels to display. Default is 5. Set this to <code>Inf</code> if all levels are needed.
<code>maxVars</code>	integer; maximum number of variables (rows) to display. Default is <code>Inf</code> , meaning all variables.
<code>truncate</code>	logical; whether level names exceeding the column width should be truncated. Default is <code>TRUE</code> .
<code>width</code>	console width. If <code>NULL</code> , defaults to <code>options("width")</code> .
<code>print.gap</code>	integer; number of spaces between columns
<code>...</code>	further arguments passed to the <code>print</code> method

Details

The levels of a factor and describing variable labels (as created by `label()`) will be wrapped within the columns.

The first 4 columns are printed with the needed fix width, the last 2 (Levels and Labels) are wrapped within the column. The width is calculated depending on the width of the screen as given by `getOption("width")`.

`toWord` has an interface for the class `Abstract`.

Value

a data frame of class `Abstract` with columns:

`Nr` column number

`Class` column class

`ColName` column name

`NAs` number of missing values

`Levels` factor levels, if applicable

`Label` descriptive column label

When printing, the `Label` column is hidden if no labels are set.

See Also

`utils::str()`, `base::summary()`, `columnWrap()`, `desc()`

Other data.inspection: `outlier()`

Examples

```
d.mydata <- C02

# let's use describing labels
label(d.mydata) <- "C02 contains data from an experiment on the cold
tolerance of the grass species Echinochloa crus-galli."

label(d.mydata$Plant) <- "an ordered factor with levels Qn1 < Qn2 < Qn3 < ... < Mc1
giving a unique identifier for each plant."

label(d.mydata$Type) <- "a factor with levels Quebec Mississippi giving the
origin of the plant"

abstract(d.mydata)
```

addMonths

Add Months to a Date

Description

Adds or subtracts whole calendar months while ensuring that the result is always a valid date. Negative values of `n` subtract months.

Usage

```
addMonths(x, n, ...)
```

Default S3 method:
 addMonths(x, n, ...)

S3 method for class 'ym'
 addMonths(x, n, ...)

Arguments

<code>x</code>	a Date object or an object coercible to one with <code>as.Date</code>
<code>n</code>	numeric vector containing finite whole numbers of months or missing values
<code>...</code>	further arguments passed to <code>as.Date</code> , such as <code>origin</code> , <code>format</code> , or <code>tz</code>

Details

Naively adding months to a date can produce invalid results. For example, adding one month to `as.Date("2013-01-31")` returns `"2013-02-28"` rather than a date in March.

The arguments `x` and `n` are recycled to their common maximum length using `bedrock::recycle()`. Partial recycling does not produce a warning.

If either argument has length zero, an empty Date vector is returned. Missing values in either argument produce missing values in the corresponding result.

Inputs supplied as `POSIXct` are converted to `Date`, and their time-of-day information is discarded. By default, `as.Date.POSIXct` performs this conversion in UTC. Supply `tz` through `...` if another time zone should determine the calendar date.

Value

a vector of class `Date` having the length of the longer of `x` and `n`

Note

Based on code by Roland Rapold and Antonio, adapted to conform to package standards.

References

<https://stackoverflow.com/questions/14169620/add-a-month-to-a-date>

See Also

[as.ym\(\)](#), [year\(\)](#), [month\(\)](#)

Other date.time: [countWorkDays\(\)](#), [date-time-predicates](#), [date_functions](#), [generation\(\)](#), [time-conversions](#), [zodiac\(\)](#)

Examples

```
# character input is converted to Date
addMonths("2013-01-31", 1)

# negative values subtract months
addMonths(as.Date("2013-03-31"), -1)

# x and n are recycled
addMonths(
  c("2013-01-31", "2013-03-31", "2013-10-31", "2013-12-31"),
  c(1, -1)
)

# missing values propagate
addMonths(c("2013-01-31", NA), 1)

# POSIXct input is converted to Date
x <- as.POSIXct(c("2015-01-31", "2015-08-31"), tz = "UTC")
addMonths(x, c(1, 3), tz = "UTC")

# numeric dates require an origin
x <- as.numeric(as.Date(c("2015-01-31", "2015-08-31")))
addMonths(x, c(1, 3), origin = as.Date("1970-01-01"))
```

Agreement

Agreement Measures - Common Interface

Description

All agreement measures in this package share a common interface.

Arguments

x	matrix or data frame of ratings (subjects x raters), or a single vector if y is provided
y	optional second vector for two raters
conf.level	confidence level for confidence intervals. If NA, no confidence interval is computed.
sides	character string specifying the side of the confidence interval, one of "two.sided" (default), "left" or "right". It names the side on which the <i>finite</i> bound lies, not the direction of an alternative hypothesis - see ConfidenceIntervals for the full definition and for the difference to DescTools .
method	method used for inference; available methods depend on the measure
weights	optional weighting scheme for ordinal data (e.g. "linear", "quadratic")

<code>model</code>	model for the ICC, "oneway" or "twoway"
<code>type</code>	type of ICC, "consistency" or "agreement"
<code>unit</code>	unit of the ICC, "single" or "average"
<code>...</code>	additional arguments passed to helper functions (e.g. bootstrap routines)

Details

Agreement is defined between two or more raters evaluating the same subjects. The data can be provided in different formats.

If only `x` is passed, it must be a matrix or `data.frame` where:

- rows represent subjects
- columns represent raters

Alternatively, for two raters only, two vectors `x` and `y` of equal length can be supplied.

Missing values (NA) are by default omitted pairwise, i.e. only subjects rated by all involved raters are used. If complete-case analysis across all raters is required, use `complete.cases` beforehand.

Agreement measures depend on:

- Number of raters (2 vs. >2)
- Scale type (nominal, ordinal, metric)

For ordinal data, weighted approaches are available.

Most functions support confidence intervals via `conf.level`. If `conf.level = NA`, no interval is computed. One-sided intervals can be requested via `sides`, which names the side carrying the finite bound; see [ConfidenceIntervals](#).

Inference methods may include classical large-sample approaches or bootstrap methods ("boot"). Specific measures may provide additional estimation procedures.

Some agreement measures define additional parameters such as:

- `weights` (for ordinal agreement)
- `model` (for ICC variants)
- `type` (consistency vs. absolute agreement)
- `unit` (single vs. average rating)

Function List

Following agreement measures are implemented in **DescToolsX**:

function	description
cohenKappa	Cohen's Kappa
scottsPi	Scott's Pi
gwetAC1	Gwet's AC1 / AC2
cronbachAlpha	Cronbach's Alpha
kappaM	Fleiss' and Light's Kappa
krippAlpha	Krippendorff's Alpha
kendallW	Kendall's W
icc	Intraclass Correlation (ICC)
ccc	Lin's Concordance Correlation

References

- Cohen, J. (1960) A coefficient of agreement for nominal scales. Educational and Psychological Measurement.
- Fleiss, J. L. (1971) Measuring nominal scale agreement among many raters. Psychological Bulletin.
- Krippendorff, K. (2004) Content Analysis.
- Shrout, P. E. and Fleiss, J. L. (1979) Intraclass correlations.
- Lin, L. I.-K. (1989) A concordance correlation coefficient.

Association

Association Measures - Common Interface

Description

All association measures in this package share a common interface.

Arguments

- | | |
|---|---|
| x | either a contingency table, a two-column object (matrix, data.frame or list), or a vector of observations (together with y) |
| y | optional second vector. If x is not a vector, y must be NULL. |

Details

The association is defined between two variables that can be provided to the calculation functions in different ways. If only x is passed, this can either be a matrix, which is then interpreted as a contingency table (this seems in the case of frequency data the natural interpretation and is by the way also what `chisq.test` expects). However, it can also be a data.frame or a list, which must then contain exactly 2 elements containing the data. Alternatively, two single data vectors x and y can be passed. The two element arguments are typically processed by forming a contingency table with `table(x, y, ...)`. NAs are by default handled the same way as the function does, so NAs omitted.

If the measure should be calculated pairwise for a set of variables `pairApply` can be used. This easily allows to create matrices of association measures (the same way as the `cor` does). NAs again are by default omitted pairwise, which corresponds to the `pairwise.complete` option of `cor`. Use `complete.cases`, if only the complete cases of a data.frame are to be used. (see examples)

Most functions support calculation of confidence intervals. These can be requested by setting `conf.level` to the desired value (usually 0.95). If it is set to NA, no confidence interval is computed. One-sided confidence intervals can be controlled using the `sides` argument. It names the side on which the *finite* bound lies, which is NOT the convention used for the alternative hypothesis of a test: "left" yields an interval bounded below and corresponds to an alternative of "greater". See [ConfidenceIntervals](#). Frequently there is a classic and a bootstrap approach ("classic", "boot"). However most measures have their own specific confidence intervals methods.

Some association measures define additional parameters such as `direction`, `base`, or `correct`. Those are documented with the respective functions.

Function List

Following association measures are implemented in **DescToolsX**:

[cramerV](#) Cramer's V

contCoef	Pearson's Contingency Coefficient
lambda	Goodman's Lambda
gkTau	Goodman Kruskal's Tau
gkGamma	Goodman Kruskal's Gamma
kendallTauB	Kendall's Tau-b
stuartTauC	Stuart's Tau-c
somersDelta	Somers' Delta
uncertCoef	Theil's Uncertainty Coefficient
mutInf	Mutual Information
hoeffdingD	Hoeffding's D
corPolychor	Polychoric Correlation

References

- Cramer, H. (1946) *Mathematical Methods of Statistics*. Princeton University Press
- Agresti, Alan (1996) *Introduction to categorical data analysis*. NY: John Wiley and Sons

as_ym

A Class for Dealing with the Yearmonth Format

Description

Representing year and month information as an integer in YYYYMM format is compact and efficient. Calendar arithmetic must nevertheless preserve the date structure: for example, subtracting two months from 201201 should return 201111 rather than 201199. [addMonths\(\)](#) provides this arithmetic for objects of class "ym".

Usage

```
as.ym(x)

## S3 method for class 'ym'
as.Date(x, d = 1, ...)

## S3 method for class 'ym'
print(x, ...)
```

Arguments

- | | |
|-----|---|
| x | a vector of integers, representing the dates in the format YYYYMM, to which a number of months has to be added. YYYY must lie in the range of 1000-3000, MM in 1-12. Values outside that range become NA. |
| d | the day to be used for converting a yearmonth to a date. Default is 1. Combinations that do not exist (e.g. 30 February) yield NA. |
| ... | further arguments; currently unused |

Details

All parameters are recycled if necessary, following the usual arithmetic recycling rules; a warning is issued when the longer argument is not a multiple of the length of the shorter one.

Value

`as.ym()` an integer vector of class "ym"
`as.Date.ym()` a vector of class "Date"
`addMonths.ym()` an integer vector of class "ym"
`print.ym()` invisibly, x

Note

Based on code by Roland Rapold, adapted to conform to package standards.

Examples

```
month(as.ym(202408))
year(as.ym(202408))

year(as.Date("2024-12-05"))
year(as.ym(202412))

month(as.Date("2024-12-05"), fmt = "mm")
month(as.ym(202412), fmt = "mm")

# arithmetic stays in the ym domain, so it can be chained
addMonths(as.ym(201511), 5)
as.ym(201511) + 5 - 2

addMonths(as.ym(c(201511, 201302)), c(5, 15))
addMonths(as.ym(c(201511, 201302)), c(5, -4))

# out-of-range input is flagged, not silently mangled
as.ym(c(201513, 999901))
```

atkinson

Atkinson Index

Description

Computes the Atkinson inequality index.

Usage

```
atkinson(
  x,
  n = rep(1, length(x)),
  epsilon = 0.5,
  na.rm = FALSE,
  tol = 0.00000001
)
```

Arguments

x	numeric vector of non-negative values, such as incomes
n	optional frequency weights; either a single non-negative whole number or a vector having the same length as x
epsilon	single non-negative numeric value specifying the inequality aversion parameter
na.rm	logical; whether missing values in x are removed
tol	single non-negative numeric value specifying the tolerance for treating epsilon as equal to one

Details

With frequency weights n_i , the weighted arithmetic mean is

$$\bar{x}_n = \frac{\sum_i n_i x_i}{\sum_i n_i}.$$

For $\varepsilon \neq 1$, the Atkinson index is

$$A(\varepsilon) = 1 - \frac{\left(\frac{\sum_i n_i x_i^{1-\varepsilon}}{\sum_i n_i} \right)^{1/(1-\varepsilon)}}{\bar{x}_n}.$$

For $\varepsilon = 1$,

$$A(1) = 1 - \frac{\exp\left(\frac{\sum_i n_i \log(x_i)}{\sum_i n_i}\right)}{\bar{x}_n}.$$

The calculation uses normalized frequency weights and logarithmic power means. It therefore does not construct the potentially very large vector that would result from `rep(x, n)`.

Observations with zero frequency are ignored. If all frequencies are zero or no observations remain after removing missing values, `NA_real_` is returned.

If all values are zero, the index is defined as zero. If at least one value is zero and `epsilon >= 1`, the equally distributed equivalent value is zero and the index is one.

Negative values, non-finite values, and missing values when `na.rm = FALSE` produce `NA_real_`. A negative `epsilon` also produces `NA_real_`.

Value

a numeric value in the interval $[0, 1]$, or `NA_real_` if the index is undefined

References

Atkinson, A. B. (1970). On the measurement of inequality. *Journal of Economic Theory*, 2(3), 244–263.

See Also

Other inequality: [divCoef\(\)](#), [gini\(\)](#), [lc\(\)](#), [rosenbluth\(\)](#), [theil\(\)](#)

Examples

```
x <- c(541, 1463, 2445, 3438, 4437,
      5401, 6392, 8304, 11904, 22261)

atkinson(x)
atkinson(x, epsilon = 1)
atkinson(x, epsilon = 2)

# frequency weights
atkinson(c(10, 20, 30), n = c(3, 1, 1))

# zero incomes
atkinson(c(0, 10, 20), epsilon = 1)
```

attach-detach-aliases *Attach and Remove Short Aliases for Selected DescToolsX Functions*

Description

Assigns a set of convenient short-name aliases into an environment (by default the global environment). This is an explicit opt-in: nothing is exported under these names, so no namespace conflicts arise when the package is merely attached with `library()`.

Usage

```
attachAliases(envir = .GlobalEnv, overwrite = FALSE)

detachAliases(envir = .GlobalEnv)
```

Arguments

<code>envir</code>	the environment into which the aliases are assigned. Defaults to <code>.GlobalEnv</code> . Supply a different environment (e.g. the calling frame via <code>environment()</code>) if you want script-local aliases.
<code>overwrite</code>	logical; if <code>TRUE</code> , objects already present under an alias name are replaced. Default is <code>FALSE</code> , in which case such names are skipped with a warning.

Details

The short-name aliases that were previously created by `attachAliases()` from the given environment can be removed by `detachAliases()`. Names that do not exist, or that hold something other than the aliased function, are left untouched.

The following aliases are created:

Alias	Function
or	<code>oddsRatio()</code>
rr	<code>relRisk()</code>

Aliases are plain function objects assigned with `assign()`; they are indistinguishable from calling the original function directly. To remove them, call `detachAliases()` or simply `rm(or, rr, envir = .GlobalEnv)`.

An existing object of the same name is not overwritten unless `overwrite = TRUE` is given, so a user's own `or` or `rr` cannot be clobbered by accident.

Value

invisibly, a character vector of the alias names that were created, resp. removed

See Also

`detachAliases`, `oddsRatio`, `relRisk`

Examples

```
# aliases are written into an environment; use a local one in scripts
e <- new.env()
attachAliases(envir = e)

m <- matrix(c(10, 20, 30, 40), nrow = 2)
e$or(m)  # same as oddsRatio(m)
e$rr(m)  # same as relRisk(m)

detachAliases(envir = e)
```

auc

Compute Area Under the Curve

Description

Calculates the area under a curve defined by points (x, y) using different numerical integration methods.

Usage

```
auc(
  x,
  y,
  from = min(x, na.rm = TRUE),
  to = max(x, na.rm = TRUE),
  method = c("trapezoid", "step", "spline"),
  absoluteArea = FALSE,
  subdivisions = 100,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x, y</code>	numeric vectors of equal length defining the curve coordinates
<code>from, to</code>	single numeric values specifying the integration interval; by default, the range of <code>x</code>
<code>method</code>	character string specifying the integration method: "trapezoid", "step", or "spline"
<code>absoluteArea</code>	logical; whether areas below zero are counted as positive
<code>subdivisions</code>	positive whole number specifying the maximum number of subdivisions used for spline integration
<code>na.rm</code>	logical; whether incomplete (<code>x, y</code>) pairs are removed
<code>...</code>	additional arguments passed to approx for trapezoidal interpolation

Details

The available methods are:

- "trapezoid": linear interpolation between successive points
- "step": a right-continuous step function using the value at the left endpoint of each interval
- "spline": a natural cubic spline integrated numerically

For `method = "step"`, an integration boundary lying between two observed `x` values retains the value of the preceding point. No linear interpolation is performed at the boundary.

If `absoluteArea = TRUE`, the absolute value of the interpolated curve is integrated. Sign changes in linear segments are split at their exact zero-crossing. For the step method, the absolute values of the constant step heights are used.

Both integration limits must lie inside the range of `x`. Extrapolation is not performed.

Value

a numeric value representing the computed area

See Also

[approx](#), [splinefun](#), [integrate](#)

Other `model.metrics`: [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(1, 2, 3, 5)
y <- c(0, 1, 1, 2)

auc(x, y)
auc(x, y, method = "step")
auc(x, y, method = "spline")
auc(x, y, absoluteArea = TRUE)

# interval boundaries between observed x values
auc(
  x = c(0, 1, 2),
  y = c(-2, 10, 4),
  from = 0.5,
```

```

    to = 1.5,
    method = "step"
  )

  auc(
    x = c(0, 1, 2),
    y = c(-2, 10, 4),
    from = 0.5,
    to = 1.5,
    method = "step",
    absoluteArea = TRUE
  )

```

blandAltmanData

Bland-Altman Agreement Data

Description

Computes the quantities required for a Bland-Altman agreement analysis.

Usage

```

blandAltmanData(x, ...)

## Default S3 method:
blandAltmanData(x, y, conf.level = 0.95, na.rm = FALSE, ...)

## S3 method for class 'formula'
blandAltmanData(x, data = NULL, conf.level = 0.95, na.rm = FALSE, ...)

```

Arguments

<code>x</code>	numeric vector or formula
<code>...</code>	further arguments passed to or from other methods
<code>y</code>	numeric vector
<code>conf.level</code>	confidence level for the intervals reported for the bias and the limits of agreement
<code>na.rm</code>	logical; if TRUE, incomplete observation pairs are removed before computation
<code>data</code>	optional data frame used with the formula interface

Details

For each observation pair, the arithmetic mean and the difference ($y - x$) are calculated. The function further computes the mean difference (bias), limits of agreement (LoA), and approximate confidence intervals for the bias and LoA according to Bland and Altman.

The returned object is of class "BlandAltman" and can be plotted using by a `plot()` routine residing in the **pharos** package.

The limits of agreement are the conventional bias $\pm 1.96 * sd$, independent of `conf.level`: the multiplier fixes the nominal coverage of the interval of *differences*, whereas `conf.level` governs the confidence intervals reported for the bias and for the two limits. The standard error of a limit uses the approximation $\sqrt{3s_d^2/n}$ given by Bland and Altman.

Value

an object of class "BlandAltman" with components:

mean pairwise means
 diff pairwise differences ($y - x$)
 bias mean difference
 loaLower lower limit of agreement
 loaUpper upper limit of agreement
 biasCI confidence interval for the bias
 loaLowerCI confidence interval for the lower LoA
 loaUpperCI confidence interval for the upper LoA
 nObs number of observations used
 conf.level the confidence level used

References

Bland JM, Altman DG (1986). Statistical methods for assessing agreement between two methods of clinical measurement. *Lancet*, 327, 307-310.

See Also

Other agreement: [normalizeToConfusion\(\)](#), [raterFrame\(\)](#)

Examples

```
set.seed(1)
x <- rnorm(100)
y <- x + rnorm(100, sd = 0.5)

ba <- blandAltmanData(x, y)
ba
```

boxCox

Box-Cox Transformation

Description

boxCox() applies the Box-Cox transformation to a numeric vector.
 boxCoxInv() reverses the transformation.

Usage

```
boxCox(x, lambda, tol = 0.000001)

boxCoxInv(x, lambda, tol = 0.000001)
```

Arguments

<code>x</code>	a numeric vector. Must contain strictly positive values (except NAs).
<code>lambda</code>	a single numeric transformation parameter
<code>tol</code>	numeric tolerance for detecting the special case $\lambda \approx 0$

Details

The Box-Cox transformation is defined for strictly positive values of `x` and is given by

$$f_{\lambda}(x) = \begin{cases} (x^{\lambda} - 1)/\lambda & \text{if } \lambda \neq 0 \\ \log(x) & \text{if } \lambda = 0 \end{cases}$$

The transformation requires strictly positive input values. If $|\text{lambda}| < \text{tol}$, the logarithmic transformation is used instead for numerical stability.

The inverse transformation recovers the original data (up to numerical precision) when the same `lambda` and `tol` are used.

Value

a numeric vector of the same length as `x`. An input consisting only of NA is an error.

References

Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society, Series B*, **26**(2), 211–252.

See Also

[boxCoxLambda](#)

Other transform: [boxCoxLambda\(\)](#), [logSt\(\)](#), [scaleX\(\)](#), [yeoJohnson\(\)](#)

Examples

```
set.seed(1)
x <- rlnorm(500, 1, 0.5)

y <- boxCox(x, lambda = 0.5)
xBack <- boxCoxInv(y, lambda = 0.5)

# check inversion
max(abs(x - xBack))

# log-transform (lambda ~ 0)
y0 <- boxCox(x, lambda = 0)
```

boxCoxLambda

*Automatic Selection of Box-Cox Transformation Parameter***Description**

Selects the Box-Cox transformation parameter automatically, using either Guerrero's method or the profile log likelihood.

Guerrero's (1993) method yields a lambda which minimizes the coefficient of variation for subseries of x . For method "loglik", the value of lambda is chosen to maximize the profile log likelihood of a linear model fitted to x . For non-seasonal data, a linear time trend is fitted while for seasonal data, a linear time trend with seasonal dummy variables is used.

Usage

```
boxCoxLambda(
  x,
  method = c("guerrero", "loglik"),
  lower = -1,
  upper = 2,
  nonseasonalLength = 2
)
```

Arguments

<code>x</code>	a numeric vector or univariate time series. All values must be strictly positive and finite, as the Box-Cox transformation is undefined otherwise; missing values are not removed but rejected, since subsetting would strip a ts of its frequency and cycle positions.
<code>method</code>	method to be used in calculating lambda. Can be either "guerrero" (default) or "loglik".
<code>lower</code>	lower limit for possible lambda values; defaults to -1
<code>upper</code>	upper limit for possible lambda values; defaults to 2
<code>nonseasonalLength</code>	number of observations per subseries used by the "guerrero" method for non-seasonal data, default is 2. Must be a whole number ≥ 2 . For seasonal time series the series' own frequency is used instead, whenever it is larger.

Details

Seasonality is taken from x itself: a `ts` object with `frequency(x) > 1` is treated as seasonal, anything else (including a plain numeric vector) as non-seasonal. For method "loglik" the profile log likelihood is therefore computed from `lm(x ~ trend)` for non-seasonal data and from `lm(x ~ trend + factor(cycle(x)))` for seasonal data. Both methods optimise lambda continuously over `[lower, upper]` via `optimize`.

Both methods need enough data to identify their criterion, and signal an error rather than falling back silently when they do not have it: "loglik" requires at least three observations, and more than `frequency(x) + 1` for a seasonal series, that being the number of parameters in the seasonal model (intercept, trend and `frequency(x) - 1` dummies); "guerrero" requires at least two complete subseries. Constant series are rejected by both, since the coefficient of variation degenerates to 0/0 and the profile log likelihood is singular.

Value

a numeric scalar containing the estimated Box-Cox transformation parameter

Note

Based on code by Leanne Chhay and Rob J Hyndman previously published as `BoxCox.lambda()` in the **forecast** package, adapted to conform to package standards.

References

Box, G. E. P. and Cox, D. R. (1964) An analysis of transformations. *JRSS B* **26** 211–246.

Guerrero, V.M. (1993) Time-series analysis supported by power transformations. *Journal of Forecasting*, **12**, 37–48.

See Also

Other transform: [boxCox\(\)](#), [logSt\(\)](#), [scaleX\(\)](#), [yeoJohnson\(\)](#)

Examples

```
lambda <- boxCoxLambda(AirPassengers)

# profile log likelihood, seasonal trend model
boxCoxLambda(AirPassengers, method = "loglik")

# plain numeric vector, treated as non-seasonal
set.seed(1)
boxCoxLambda(rlnorm(100), method = "loglik")
```

brierScore

Brier Score

Description

Computes the Brier score for binary probabilistic predictions, optionally with a confidence interval via a normal approximation or bootstrap.

Usage

```
brierScore(
  x,
  pred = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("normal", "boot"),
  scaled = FALSE,
  ...
)
```

Arguments

<code>x</code>	either a numeric vector of observed binary outcomes (0/1) when <code>pred</code> is supplied, or a fitted model object (glm or similar) from which both response and predictions are extracted
<code>pred</code>	a numeric vector of predicted probabilities in $[0, 1]$. Required when <code>x</code> is a numeric vector; ignored when <code>x</code> is a model object.
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>method</code>	confidence interval method: "normal" (delta-method approximation, default) or "boot" (bootstrap via <code>brier_boot_cpp()</code>)
<code>scaled</code>	logical. Should the scaled Brier score be returned? Default FALSE.
<code>...</code>	further arguments passed to the bootstrap engine when <code>method = "boot"</code> : <code>R</code> , <code>type</code> , <code>parallel</code> , <code>ncpus</code> . See Details.

Details

The Brier score is defined as

$$BS = \frac{1}{n} \sum_{i=1}^n [y_i(1 - \hat{p}_i)^2 + (1 - y_i)\hat{p}_i^2]$$

where $y_i \in \{0, 1\}$ and $\hat{p}_i$ is the predicted probability. Lower is better; a perfect model scores 0.

The scaled Brier score (`scaled = TRUE`) expresses skill relative to the climatological baseline $BS_{\max}$, yielding 1 for a perfect model and 0 for the no-skill reference.

`sides` names the side on which the finite bound lies: "left" yields $[lci, \infty)$, "right" yields $(-\infty, uci]$.

Normal interval (`method = "normal"`)

A delta-method normal approximation based on the variance of the per-observation Brier losses. Fast and deterministic; reliable for moderate to large samples. With `scaled = TRUE` the standard error is carried onto the skill scale by dividing through $BS_{\max}$, which is treated as fixed; the interval therefore ignores the sampling variability of the baseline and is mildly anti-conservative. Prefer `method = "boot"` for scaled scores.

Bootstrap interval (`method = "boot"`)

Case-resampling bootstrap via `brier_boot_cpp()`. The bootstrap type is controlled by the `type` argument (passed through `...`):

"bca" bias-corrected and accelerated (default). Most accurate; requires $R \geq 200$.

"perc" percentile interval

"norm" normal approximation using the bootstrap standard error

Further bootstrap arguments passed through `...` via `.extractBootArgs()`:

`R` number of bootstrap replicates (default 999)

`parallel` parallelisation: "no", "multicore", or "snow" (default "no")

`ncpus` number of CPUs (default `getOption("boot.ncpus", 1L)`)

Value

if `conf.level = NA`, a numeric scalar containing the Brier score; otherwise a named numeric vector with elements:

`est` point estimate of the Brier score.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

See Also

[predict](#)

Other `model.metrics`: [auc\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
set.seed(1)
resp <- rbinom(200, 1, 0.4)
pred <- plogis(rnorm(200, ifelse(resp == 1, 0.5, -0.5)))

brierScore(resp, pred)
brierScore(resp, pred, conf.level = 0.95)
brierScore(resp, pred, conf.level = 0.95, method = "boot", type = "bca")
brierScore(resp, pred, conf.level = 0.95, scaled = TRUE)
```

ccc

Lin's Concordance Correlation Coefficient

Description

Computes Lin's concordance correlation coefficient (CCC) for assessing agreement between two continuous measurements.

Usage

```
ccc(
  x,
  y,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("z-transform", "boot", "asymptotic"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a numeric vector
<code>y</code>	a numeric vector of equal length to <code>x</code>
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.

<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>method</code>	a character string specifying the confidence interval method. One of "z-transform", "boot", or "asymptotic".
<code>na.rm</code>	logical; if TRUE, incomplete observation pairs are removed before computation
<code>...</code>	additional arguments controlling the bootstrap procedure. Currently R, <code>parallel</code> and <code>ncpus</code> are supported.

Details

The CCC combines measures of precision and accuracy and quantifies the deviation of the observed data from the line of perfect concordance. Values range from -1 to 1, where 1 indicates perfect agreement.

Confidence intervals can be computed using a Fisher z-transformation, a nonparametric bootstrap, or the asymptotic approximation of Lin (2000). The asymptotic variance implemented here is the corrected form given by Lin (2000), superseding the expression in Lin (1989). Internally it is held on the scale of ρ_c itself; the "z-transform" method rescales it to the z scale via the delta method, where $d \operatorname{atanh}(\rho)/d\rho = 1/(1 - \rho^2)$.

`sides` names the side on which the finite bound lies: "left" yields an interval bounded below, with the upper limit fixed at 1, and "right" one bounded above, with the lower limit fixed at -1.

Missing values are handled according to package conventions: if `na.rm = FALSE` and either `x` or `y` contains missing values, NA is returned. If `na.rm = TRUE`, complete cases are used. Infinite values carry no comparable convention - they leave the moments undefined and are rejected with an error.

Value

a named numeric vector containing only `est` when `conf.level = NA`; otherwise a named numeric vector with elements:

`est` point estimate.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

Additional diagnostics are stored as attributes:

`nObs` number of observations used

`scaleShift` scale shift parameter

`locationShift` location shift parameter

`biasCorrection` bias correction factor

`method` confidence interval method, if applicable

`confLevel` confidence level, if applicable

`sides` confidence interval type, if applicable

References

Lin, L. I.-K. (1989). A concordance correlation coefficient to evaluate reproducibility. *Biometrics*, 45(1), 255-268.

Lin, L. I.-K. (2000). A note on the concordance correlation coefficient. *Biometrics*, 56(1), 324-325.

See Also

Other assoc.agreement: [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
set.seed(123)

x <- rnorm(100)
y <- x + rnorm(100, sd = 0.2)

ccc(x, y)

ccc(x, y, conf.level = 0.95)

ccc(
  x, y,
  conf.level = 0.95,
  method = "boot",
  R = 999
)
```

coefVar

*Coefficient of Variation***Description**

Calculates the coefficient of variation and its confidence limits using various methods.

Usage

```
coefVar(x, ...)

## Default S3 method:
coefVar(x, weights = NULL, unbiased = FALSE, na.rm = FALSE, ...)

## S3 method for class 'lm'
coefVar(x, unbiased = FALSE, na.rm = FALSE, ...)

## S3 method for class 'aov'
coefVar(x, unbiased = FALSE, na.rm = FALSE, ...)

coefVarCI(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("nct", "vangel", "mckay", "naive"),
  weights = NULL,
  unbiased = FALSE,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a non-empty numeric vector of data values, or a fitted model for the <code>lm/aov</code> methods
<code>...</code>	further arguments
<code>weights</code>	a numeric vector of weights the same length as <code>x</code> giving the weights to use for elements of <code>x</code>
<code>unbiased</code>	logical; whether to apply a bias correction. See Details. Defaults to <code>FALSE</code> .
<code>na.rm</code>	logical. Should missing values be removed? Defaults to <code>FALSE</code> , in which case missing values are an error.
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of <code>"two.sided"</code> (default), <code>"left"</code> or <code>"right"</code>). See ConfidenceIntervals .
<code>method</code>	character string specifying the confidence interval method: <code>"nct"</code> (default), <code>"vangel"</code> , <code>"mckay"</code> , or <code>"naive"</code> . Partial matching is supported. See Details.

Details

In order for the coefficient of variation to be an unbiased estimate of the true population value, the coefficient of variation is corrected as:

$$CV_{corr} = CV \cdot \left(1 - \frac{1}{4 \cdot (n - 1)} + \frac{1}{n} \cdot CV^2 + \frac{1}{2 \cdot (n - 1)^2} \right)$$

For determining **the confidence intervals** for the coefficient of variation a number of methods have been proposed. `coefVarCI()` currently supports four different methods. The details for the methods are given in the specific references.

The **"naive" method** is based on dividing the standard confidence limit for the standard deviation by the sample mean.

McKay's approximation is asymptotically exact as n goes to infinity. McKay recommends this approximation only if the coefficient of variation is less than 0.33. Note that if the coefficient of variation is greater than 0.33, either the normality of the data is suspect or the probability of negative values in the data is non-negligible. In this case, McKay's approximation may not be valid. Also, it is generally recommended that the sample size should be at least 10 before using McKay's approximation.

Vangel's modified McKay method is more accurate than the McKay method in most cases, particularly for small samples. According to Vangel, the unmodified McKay is only more accurate when both the coefficient of variation and α are large. However, if the coefficient of variation is large, then this implies either that the data contains negative values or the data does not follow a normal distribution. In this case, neither the McKay or the modified McKay should be used. In general, the Vangel's modified McKay method is recommended over the McKay method. It generally provides good approximations as long as the data is approximately normal and the coefficient of variation is less than 0.33.

See also: <https://www.itl.nist.gov/div898/software/dataplot/refman1/auxillar/coefvacl.htm>

nct uses the noncentral t-distribution to calculate the confidence intervals. See Smithson (2003).

sides names the side on which the finite bound lies: `"left"` yields $[lci, \infty)$, `"right"` yields $(-\infty, uci]$.

Note: Analytic (precision) weights are not supported. For likelihood-based weighted variance estimation, see [cov.wt](#).

Value

an unnamed numeric scalar containing the coefficient of variation for `coefVar()`. If recycling in `coefVarCI()` yields a single case, it returns a named numeric vector with elements:

`est` point estimate of the coefficient of variation.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

Otherwise, `coefVarCI()` returns a numeric matrix with one row per case and the columns `est`, `lci`, and `uci`.

Note

Parts of the code contributed by Michael Smithson.

References

- McKay, A. T. (1932). Distribution of the coefficient of variation and the extended t distribution, *Journal of the Royal Statistical Society*, 95, 695–698.
- Johnson, B. L., Welch, B. L. (1940). Applications of the non-central t -distribution. *Biometrika*, 31, 362–389.
- Mark Vangel (1996) Confidence Intervals for a Normal Coefficient of Variation, *American Statistician*, Vol. 15, No. 1, pp. 21-26.
- Kelley, K. (2007). Sample size planning for the coefficient of variation from the accuracy in parameter estimation approach. *Behavior Research Methods*, 39 (4), 755-766
- Kelley, K. (2007). Constructing confidence intervals for standardized effect sizes: Theory, application, and implementation. *Journal of Statistical Software*, 20 (8), 1-24
- Smithson, M.J. (2003) *Confidence Intervals, Quantitative Applications in the Social Sciences Series*, No. 140. Thousand Oaks, CA: Sage. pp. 39-41
- Steve Verrill (2003) Confidence Bounds for Normal and Lognormal Distribution Coefficients of Variation, *Research Paper 609*, USDA Forest Products Laboratory, Madison, Wisconsin.
- Verrill, S. and Johnson, R.A. (2007) Confidence Bounds and Hypothesis Tests for Normal Distribution Coefficients of Variation, *Communications in Statistics Theory and Methods*, Volume 36, No. 12, pp 2187-2206.

See Also

[meanX](#), [sdX](#), (both supporting weights)

Other dispersion: [iqrX\(\)](#), [madX\(\)](#), [meanAD\(\)](#), [meanSE\(\)](#), [rangeX\(\)](#), [varX\(\)](#)

Examples

```
set.seed(15)
x <- runif(100)
coefVar(x)
coefVarCI(x)

# Coefficient of variation for a linear model
r.lm <- lm(Fertility ~ ., swiss)
coefVar(r.lm)
```

```
# conf.level is recycled, so several levels can be requested at once.
# Reproduces the NIST reference table for the ZARR13 dataset (see the
# URL above) from a sample rebuilt to its published mean and sd, so the
# example needs no network access.
set.seed(1)
zarr <- 9.261460 + 0.022789 * as.vector(scale(rnorm(195)))

round(coefVarCI(zarr, method = "vangel", sides = "two.sided",
               conf.level = c(.5, .8, .9, .95, .99, .999)), 5)

# -----
# Confidence      Coefficient      Lower      Upper
# Value (%)      of Variation      Limit      Limit
# -----
#      50.0      0.00246      0.00238      0.00255
#      80.0      0.00246      0.00231      0.00263
#      90.0      0.00246      0.00227      0.00269
#      95.0      0.00246      0.00224      0.00273
#      99.0      0.00246      0.00217      0.00283
#      99.9      0.00246      0.00210      0.00294
```

cohenD

Cohen's and Hedges' Effect Size

Description

Computes the Cohen's d and Hedges' g effect size statistics.

Usage

```
cohenD(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  correct = FALSE,
  na.rm = FALSE
)
```

Arguments

x	a non-empty numeric vector of data values
y	an optional non-empty numeric vector of data values
conf.level	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
correct	logical; whether to apply the Hedges correction. Defaults to FALSE.
na.rm	logical. Should missing values be removed? Defaults to FALSE.

Details

For a single sample, $d = \bar{x}/s$; for two samples, $d = (\bar{x} - \bar{y})/s_{pooled}$. With `correct = TRUE` Hedges' bias correction $J = 1 - 3/(4\nu - 1)$, with ν the residual degrees of freedom, is applied to the estimate and, where computed, to the interval.

Confidence intervals invert the noncentral t distribution (Steiger & Fouladi): the noncentrality parameter is $d\sqrt{n}$ with $n - 1$ degrees of freedom in the one-sample case, and $d/\sqrt{1/n_x + 1/n_y}$ with $n_x + n_y - 2$ degrees of freedom in the two-sample case.

`sides` names the side on which the finite bound lies: "left" yields $[lci, \infty)$, "right" yields $(-\infty, uci]$.

Value

if `conf.level = NA`, a numeric scalar containing the effect size; otherwise a named numeric vector with elements:

`est` point estimate of Cohen's d or Hedges' g .

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

The magnitude category and pooled standard deviation are stored in the attributes `magnitude` and `sdPooled`, respectively.

Note

Based on code by William Revelle.

References

Cohen, J. (1988) *Statistical power analysis for the behavioral sciences (2nd ed.)* Academic Press, New York.

Hedges, L. V. & Olkin, I. (1985) *Statistical methods for meta-analysis* Academic Press, Orlando, FL

Smithson, M.J. (2003) *Confidence Intervals, Quantitative Applications in the Social Sciences Series*, No. 140. Thousand Oaks, CA: Sage. pp. 39-41

See Also

[glassDelta](#), [meanX](#), [varX](#)

Other effect.size: [cohenH\(\)](#), [etaSq\(\)](#), [glassDelta\(\)](#), [oddsRatio\(\)](#), [relRisk\(\)](#)

Examples

```
x <- Pizza$price[Pizza$driver == "Carter"]
y <- Pizza$price[Pizza$driver == "Miller"]

cohenD(x, y, conf.level = 0.95, na.rm = TRUE)

# Hedges' g
cohenD(x, y, conf.level = 0.95, correct = TRUE, na.rm = TRUE)

# one-sided: the finite bound lies on the left
cohenD(x, y, conf.level = 0.95, sides = "left", na.rm = TRUE)
```

cohenH

*Cohen's h for a 2x2 Table***Description**

Computes Cohen's h , a standardized effect size for the difference between two proportions in a 2x2 contingency table.

Usage

```
cohenH(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  ...
)
```

Arguments

<code>x</code>	a 2x2 contingency table or matrix, or a categorical vector when <code>y</code> is supplied
<code>y</code>	an optional second variable used together with <code>x</code> to create a contingency table via <code>table(x, y, ...)</code>
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>...</code>	additional arguments passed to <code>table()</code>

Details

Cohen's h is defined as:

$$h = 2 \arcsin(\sqrt{p_1}) - 2 \arcsin(\sqrt{p_2})$$

where p_1 and p_2 are the event probabilities in the first and second row, respectively.

Optionally, an approximate asymptotic confidence interval is computed.

Cohen's h is a variance-stabilized standardized effect size for comparing two proportions.

Approximate interpretation thresholds suggested by Cohen are:

$ h < 0.2$	negligible effect
$ h \geq 0.2$	small effect
$ h \geq 0.5$	medium effect
$ h \geq 0.8$	large effect

The confidence interval is based on the asymptotic standard error:

$$SE(h) = \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}$$

sides names the side on which the finite bound lies: "left" yields $[lci, \infty)$, "right" yields $(-\infty, uci]$.

Value

if `conf.level = NA`, a numeric scalar containing Cohen's h ; otherwise a named numeric vector with elements:

`est` point estimate of Cohen's h .

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

References

Cohen J (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum Associates.

See Also

Other effect.size: [cohenD\(\)](#), [etaSq\(\)](#), [glassDelta\(\)](#), [oddsRatio\(\)](#), [relRisk\(\)](#)

Examples

```
tab <- matrix(
  c(26, 26,
    6, 7),
  nrow = 2,
  byrow = TRUE
)

cohenH(tab)
cohenH(tab, conf.level = 0.95)

x <- c(rep("A", 52), rep("B", 13))
y <- c(rep(c("yes", "no"), c(26, 26)),
      rep(c("yes", "no"), c(6, 7)))

cohenH(x, y, conf.level = 0.95)
```

cohenKappa

Cohen's Kappa and Weighted Kappa

Description

Computes Cohen's kappa and weighted kappa as measures of inter-rater agreement, together with asymptotic confidence intervals.

Usage

```
cohenKappa(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  weights = c("unweighted", "equal-spacing", "fleiss-cohen"),
  ...
)
```

Arguments

<code>x</code>	a square confusion matrix (or data frame), or a categorical vector when <code>y</code> is provided
<code>y</code>	NULL (default) or a categorical vector with compatible dimensions to <code>x</code> . When supplied, <code>table(x, y, ...)</code> is computed. The vector interface is available for unweighted kappa only (see Details).
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>weights</code>	either a character string selecting a built-in weight scheme – "unweighted" (default), "equal-spacing", or "fleiss-cohen" – or a numeric matrix with the same dimensions as <code>x</code> supplying user-defined weights for each cell
<code>...</code>	further arguments passed to table for the vector interface, for example <code>useNA</code>

Details

Cohen's kappa is the diagonal sum of the (possibly weighted) relative frequencies, corrected for chance agreement and standardised by its maximum value.

The equal-spacing weights (Cicchetti & Allison, 1971) are defined as

$$1 - \frac{|i - j|}{r - 1}$$

and the Fleiss-Cohen weights as

$$1 - \frac{(i - j)^2}{(r - 1)^2}$$

where r is the number of rows/columns. The Fleiss-Cohen weights attach greater importance to closer disagreements.

Data can be passed either as a square confusion matrix (or data frame) in `x`, or as two vectors `x` and `y`, in which case `table(x, y, ...)` is computed internally. Note that the vector interface supports **unweighted kappa only**: the function raises an error if `weights` is not "unweighted" and `y` is supplied, because the level ordering of two independent factors cannot be guaranteed to be consistent when constructing the confusion table.

Missing values are handled as [table](#) does – excluded by default. Pass `useNA = "ifany"` via `...` to include them.

`sides` names the side on which the finite bound lies: "left" yields an interval bounded below, with the upper limit fixed at 1, and "right" one bounded above, with the lower limit fixed at -1. Kappa is a bounded parameter, so the open side is reported at the range boundary rather than as $\pm\infty$ (design_rules.md 4.1), matching [ccc](#).

Value

if `conf.level = NA`, a numeric scalar containing kappa; otherwise a named numeric vector with elements:

`est` point estimate of kappa.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

Note

Based on code by David Meyer, adapted to conform to package standards.

References

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.

Everitt, B. S. (1968). Moments of statistics kappa and weighted kappa. *The British Journal of Mathematical and Statistical Psychology*, 21(1), 97–103.

Fleiss, J. L., Cohen, J., & Everitt, B. S. (1969). Large sample standard errors of kappa and weighted kappa. *Psychological Bulletin*, 72(5), 323–327.

Cicchetti, D. V., & Allison, T. (1971). A new procedure for assessing reliability of scoring EEG sleep recordings. *American Journal of EEG Technology*, 11(3), 101–109.

See Also

[pairApply](#)

Other assoc.agreement: [ccc\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
# from Bortz et al. (1990), p. 459
m <- matrix(c(53, 5, 2,
              11, 14, 5,
              1, 6, 3), nrow = 3, byrow = TRUE,
            dimnames = list(rater1 = c("V", "N", "P"),
                           rater2 = c("V", "N", "P")))

cohenKappa(m)
cohenKappa(m, conf.level = 0.95)

# vector interface (unweighted only)
x <- bedrock::untable(m)
cohenKappa(x$rater1, x$rater2)

# equal-spacing weights
cats <- c("<10%", "11-20%", "21-30%", "31-40%", "41-50%", ">50%")
mw <- matrix(
  c(5,8,1,2,4,2, 3,5,3,5,5,0, 1,2,6,11,2,1,
    0,1,5,4,3,3, 0,0,1,2,5,2, 0,0,1,2,1,4),
  nrow = 6, byrow = TRUE,
  dimnames = list(rater1 = cats, rater2 = cats))
```

```

cohenKappa(mw, weights = "equal-spacing", conf.level = 0.95)

# user-supplied weight matrix
wm <- outer(1:6, 1:6, function(i, j) 1 - abs(i - j) / (6 - 1))
cohenKappa(mw, weights = wm, conf.level = 0.95)

# pairwise kappa across raters
rating <- data.frame(
  rtr1 = c(4,2,2,5,2,1,3,1,1,5,1,1,2,1,2,3,1,1,2,1,5,2,2,1,1,2,1,2,1,5),
  rtr2 = c(4,2,3,5,2,1,3,1,1,5,4,2,2,4,2,3,1,1,2,3,5,4,2,1,4,2,1,2,3,5))

bedrock::pairApply(rating, FUN = cohenKappa, symmetric = TRUE)

```

concepts

*Concept Utilities for Package Documentation***Description**

Helper functions to inspect and analyse the use of `\concept` tags within a package.

Usage

```

getConcepts(pkg)

conceptMap(pkg)

conceptAudit(pkg)

```

Arguments

`pkg` character string. Name of the installed package.

Details

These utilities extract concept metadata from Rd files and allow structured auditing of conceptual organisation inside a package.

Functions

- `getConcepts()` - Returns all unique concepts used in a package.
- `conceptMap()` - Returns a mapping of concepts to functions.
- `conceptAudit()` - Returns a summary table of concept usage.

The functions use [Rd_db](#) to parse Rd files and extract `\concept` tags programmatically.

These tools are intended for package development, documentation consistency checks, and conceptual audits.

Value

`getConcepts` character vector of unique concept names, sorted

`conceptMap` named list mapping concepts to topics

`conceptAudit` data frame with the columns `concept` and `nTopics`, ordered by decreasing frequency

Examples

```
getConcepts("DescToolsX")
head(conceptMap("DescToolsX"))
head(conceptAudit("DescToolsX"))
```

conDisPairs

Concordant and Discordant Pairs

Description

Counts, for all $n(n - 1)/2$ pairs of observations, how many are concordant, how many discordant, and how the remaining ties are distributed. This is the quantity every rank-based association measure for ordinal data is built on.

Usage

```
conDisPairs(x, y = NULL)
```

Arguments

x a numeric vector, an ordered factor, or a contingency table

y optional second numeric vector or ordered factor. If provided, vector mode is used.

An [ordered](#) factor is converted to its level codes, which is exactly the ordinal information the counts rest on. An *unordered* factor is refused: its levels have no order, and converting it anyway would silently impose the alphabetical one and report concordances that are an artefact of the level names.

Details

Two observations (x_i, y_i) and (x_j, y_j) are **concordant** if they are ordered the same way in both variables - one is larger in x and larger in y - and **discordant** if the orderings disagree. If either variable ties the pair, it is neither, and is counted among the ties instead.

The five counts partition the pairs exhaustively and without overlap:

$$C + D + T_X + T_Y + T_{XY} = \frac{n(n - 1)}{2}$$

Ties_X counts pairs tied in x but *not* in y , Ties_Y the reverse, and Ties_XY the pairs tied in both. The counts are therefore *exclusive*; the inclusive marginal counts that the classic tau-b formula uses are Ties_X + Ties_XY and Ties_Y + Ties_XY. That separation is what the ordinal measures need: Goodman-Kruskal's gamma ignores all ties, Kendall's tau-b corrects for Ties_X and Ties_Y separately, Somers' D for one of them only, and the c statistic scores half a point for a tie in y . All of them are therefore a short formula on top of this one function - see [ordAssocs](#).

Two input modes: **Vector mode** (x and y given) works on the raw observations and keeps the full resolution of the data. Missing values are removed pairwise.

Table mode (x a matrix or table) works on a cross tabulation. Use it when the data are already tabulated, or when the variables have few distinct values: the count then depends on the table's dimensions rather than on the number of observations, so a million observations in a 4x5 table cost the same as a hundred.

Both modes return the same five numbers for the same data.

Why this is fast: Counted naively, the definition is a double loop over all pairs and costs $O(n^2)$ - a hundred thousand observations are five billion comparisons, which is where a straightforward implementation stops being usable.

Vector mode instead sweeps the observations in the order of x and keeps a two-dimensional Fenwick tree (binary indexed tree) over the ranks seen so far. Each new observation is answered with a range query rather than a scan, which brings the total to $O(n \log^2 n)$. The difference is not a constant factor: it is what makes six-figure sample sizes a matter of a moment instead of a coffee break.

Table mode uses the cumulative-sum identity over the table and is $O(rc)$ in the table's dimensions, independent of n .

Both are implemented in C++ and return exact integer counts - no approximation, no sampling.

Value

a named numeric vector with elements:

C number of concordant pairs

D number of discordant pairs

Ties_X pairs tied in x only

Ties_Y pairs tied in y only

Ties_XY pairs tied in both

If fewer than two observations remain, all five elements are NA.

References

Goodman, L. A., Kruskal, W. H. (1954) Measures of association for cross classifications. *Journal of the American Statistical Association*, **49**, 732-764.

Agresti, A. (2010) *Analysis of Ordinal Categorical Data* (2nd ed.). Wiley.

See Also

[ordAssocs](#), [gkGamma](#), [kendallTauA](#), [kendallTauB](#), [stuartTauC](#), [somersDelta](#)

Other assoc.ordinal: [cStat\(\)](#), [kendallW\(\)](#), [ordAssocs\(\)](#)

Examples

```
# vector input
x <- c(1, 2, 3, 1, 2)
y <- c(2, 1, 3, 2, 1)
conDisPairs(x, y)

# the five counts partition all n(n-1)/2 pairs
sum(conDisPairs(x, y)) == choose(length(x), 2)

# table input gives the same answer
tab <- table(x, y)
conDisPairs(tab)

# the ordinal measures are short formulas on top of it
p <- conDisPairs(x, y)
unname((p["C"] - p["D"]) / (p["C"] + p["D"])) # Goodman-Kruskal's gamma
gkGamma(x, y)
```

```
# vector mode stays usable where a pairwise double loop would not:
# 200'000 observations are 2 * 10^10 pairs
set.seed(1)
n <- 2e5
system.time(conDisPairs(rnorm(n), rnorm(n)))
```

conf

Confusion Matrix and Classification Metrics

Description

Computes confusion matrices and a wide range of performance metrics for classification models or predicted vs. observed labels.

Usage

```
conf(x, ...)

## S3 method for class 'table'
conf(x, pos = NULL, conf.level = 0.95, ...)

## Default S3 method:
conf(x, ref, pos = NULL, na.rm = TRUE, ...)

## S3 method for class 'matrix'
conf(x, pos = NULL, ...)

## S3 method for class 'rpart'
conf(x, ...)

## S3 method for class 'multinom'
conf(x, ...)

## S3 method for class 'glm'
conf(x, cutoff = 0.5, pos = NULL, ...)

## S3 method for class 'randomForest'
conf(x, ...)

## S3 method for class 'svm'
conf(x, ...)

## S3 method for class 'lda'
conf(x, ...)

## S3 method for class 'qda'
conf(x, ...)
```

```
## S3 method for class 'Conf'
print(x, digits = max(3L, getOption("digits") - 3L), ...)

## S3 method for class 'Conf'
plot(x, main = "Confusion Matrix", ...)

sensX(x, ...)

specX(x, ...)
```

Arguments

x	object containing predictions; one of: • a factor or character vector of predicted classes • a confusion matrix (table or matrix) with predictions in the rows and references in the columns • a fitted model object (e.g., glm, rpart)
...	further arguments passed to specific methods
pos	optional character specifying the positive class (binary classification only). If NULL, the second level is used and a message is issued.
conf.level	confidence level for the accuracy interval; defaults to 0.95
ref	optional reference (true labels). Required for the default method.
na.rm	logical; remove missing values before computation. Default TRUE.
cutoff	numeric cutoff for probabilistic models (e.g., glm). Default 0.5.
digits	integer; number of decimal places for printing
main	character string specifying the plot title

Details

This is a generic function with methods for tables, vectors, and several model objects (e.g., glm, rpart, randomForest, svm).

sensX() and specX() are convenience extractors for the sensitivity and specificity values computed by conf().

The orientation of the table matters: rows are read as predictions and columns as references, so the no-information rate is taken from the column margin. conf.default() builds the table accordingly.

Overall statistics:

- Accuracy with confidence interval
- No Information Rate (NIR) and p-value (Accuracy > NIR)
- Cohen's Kappa
- McNemar test p-value

Class-wise statistics (computed one-vs-all for multiclass):

- Sensitivity (Recall)
- Specificity
- Positive Predictive Value (Precision)
- Negative Predictive Value

- Prevalence
- Detection Rate and Detection Prevalence
- Balanced Accuracy
- F-value (harmonic mean of Precision and Recall)
- Matthews Correlation Coefficient (MCC)

Value

`conf()` returns an object of class "Conf" containing:

`table` confusion matrix

`pos` positive class (binary only, else NULL)

`diag` number of correct predictions

`n` total number of observations

`acc, acc.lci, acc.uci` accuracy and CI

`conf.level` confidence level used for the accuracy CI

`nir` no-information rate

`acc.pval` p-value for accuracy greater than the no-information rate

`kappa` Cohen's kappa

`mcnemar.pval` McNemar test p-value

`byclass` matrix of class-wise metrics

`sensX()` and `specX()` return a named numeric vector containing the sensitivity or specificity, respectively, for each reported class.

Examples

```
# vectors
pred <- factor(c("A", "B", "A", "A", "B"))
ref <- factor(c("A", "A", "A", "B", "B"))
conf(pred, ref)

# table
conf(table(pred, ref))

# glm
m <- glm(am ~ hp + wt, data = mtcars, family = binomial)
conf(m)
```

ConfidenceIntervals *Confidence Interval Interface - Common Arguments*

Description

Arguments shared by the confidence interval functions in this package, documented once. Individual functions describe only which values they accept and refer here for what the values mean.

Arguments

<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval. Must be one of "two.sided" (default), "left" or "right". For a one-sided interval, the value names the side with the finite bound. The initial letter is sufficient.
<code>method</code>	character string specifying the interval method. The available methods and the default differ between functions; see the individual help page for the choices.
<code>...</code>	additional arguments for the bootstrap, such as the number of resamples R and the interval type <code>type</code> .

Details

Return value: With `conf.level = NA` the functions return the point estimate as an unnamed scalar. With a confidence level they return a named numeric vector with the elements `est`, `lci` and `uci`, in that order. Reading the result by name rather than by position is the safer habit, since some functions return further elements.

One-sided intervals: `sides` names the side carrying the *finite* bound:

"left" the lower bound is finite, the upper one is open.

"right" the upper bound is finite, the lower one is open.

Thus `sides = "left"` corresponds to `alternative = "greater"` in `t.test`, and `sides = "right"` to `alternative = "less"`. This is also the convention used by the corresponding functions in **DescTools**.

The open side is reported at the boundary of the parameter space, not at infinity - most of the statistics here are bounded, so this is the ordinary case rather than an exception. A correlation opens to ± 1 , an association measure in $[0, 1]$ to 0 or 1, Cramer's V to 1, Pearson's C to $\sqrt{(m-1)/m}$. Where the parameter really is unbounded, $\pm\infty$ is reported: a relative risk opens upwards to Inf but downwards only to 0, a location estimator opens to -Inf and Inf, Cronbach's α to -Inf and 1. An interval never claims a value the statistic cannot take.

A one-sided bound at level γ is the corresponding end of the two-sided interval at level $2\gamma - 1$: a 95% lower end of the two-sided 90% interval requires `conf.level` above 0.5 and is refused below it, where the adjusted level would not be positive.

Choice of method: The available options depend on the statistic and on what is known about its distribution. Classical intervals rely on asymptotic normality or on an analytic variance formula, and are fast and deterministic where such a formula exists. Bootstrap intervals, requested with "boot", need no closed-form variance and are therefore available for statistics that have none - at the price of being random and slower.

Bootstrap intervals are partly computed with the **boot** package (see [boot](#) and [boot.ci](#)). The number of resamples R and the interval type - "perc", "bca" and others - are passed through "bca" corrects for bias and skewness and is the better choice for a smooth statistic whose parameter lies well inside its range. It is the weaker choice near a boundary: both of its ingredients degrade where the parameter sits at the edge of the parameter space, which for an association measure under independence is the ordinary situation rather than a pathology. "perc" is the more robust default there.

Random number generation

Requesting a bootstrap confidence interval draws a seed from R’s global random number generator and therefore advances it. Call [set.seed](#) beforehand for reproducible intervals. This applies to the bootstrap methods only; classical intervals are deterministic.

See Also

[boot](#), [boot.ci](#), [confint](#)

constants	<i>DescToolsX Constants</i>
-----------	-----------------------------

Description

English abbreviated and full weekday names, starting with Monday. Metric prefixes with name, abbreviation and dimension.

Usage

day.abb

day.name

Format

A character vector of length 7.

contCoef	<i>Pearson’s Contingency Coefficient</i>
----------	--

Description

Computes Pearson’s contingency coefficient for a contingency table. If x and y are supplied, the table is constructed first; see [Association](#).

Usage

```
contCoef(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  correct = FALSE,
  ...
)
```

Arguments

<code>x</code>	either a contingency table, a two-column object (matrix, data.frame or list), or a vector of observations (together with <code>y</code>)
<code>y</code>	optional second vector. If <code>x</code> is not a vector, <code>y</code> must be <code>NULL</code> .
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>correct</code>	logical; whether Sakoda's adjusted Pearson's C should be returned; defaults to <code>FALSE</code>
<code>...</code>	further arguments. Named arguments known to normalizeToConfusion or table are used to build the table; <code>R</code> and <code>type</code> configure the bootstrap and are described under Details. Anything else is an error rather than a silent no-op.

Details

Pearson's contingency coefficient ranges from 0 to $\sqrt{(m-1)/m}$, where $m = \min(r, c)$. Consequently, its attainable maximum depends on the dimensions of the table.

Sakoda's correction divides the coefficient by this maximum:

$$C_S = \frac{C}{C_{\max}} = C \sqrt{\frac{m}{m-1}}.$$

The corrected coefficient therefore ranges from 0 to 1.

Since no generally accepted analytical interval is available, only bootstrap intervals are implemented. The interval is obtained from a multinomial bootstrap over the cells of the table. Two arguments configure it, both passed through `...`: `R`, the number of replicates (default 999), and `type`, one of "perc" (default) or "bca".

"perc" is the default deliberately. Under independence the parameter sits *on* the boundary of its range, where the sampling distribution of C is not normal under any monotone transformation - which is precisely what BCa assumes. Both of its ingredients degrade there: the bias correction is read off the share of replicates below the estimate, which collapses when the estimate is at the edge of the bootstrap distribution, and the acceleration is a jackknife, which is not consistent for a functional that is not smooth. "bca" is the better choice for tables with a clearly non-zero association and a reasonable number of observations.

Confidence intervals are restricted to the attainable range. Measures such as [cramerV](#) may be preferable when inference is central.

For further information see [ConfidenceIntervals](#).

Value

if `conf.level = NA`, a numeric scalar containing Pearson's contingency coefficient; otherwise a named numeric vector with elements:

`est` point estimate of the contingency coefficient

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Sakoda, J.M. (1977) Measures of Association for Multivariate Contingency Tables, *Proceedings of the Social Statistics Section of the American Statistical Association* (Part III), 777-780.

Efron, B., Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*, Chapman & Hall, chapter 14.

See Also

[bedrock::pairApply](#)

Other assoc.nominal: [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
tab <- apply(HairEyeColor, c(1, 2), sum)
contCoef(tab)

# just x and y
with(bedrock::untable(tab), contCoef(Hair, Eye))

set.seed(1)
contCoef(tab, conf.level = 0.95)

set.seed(1)
contCoef(tab, conf.level = 0.95, type = "bca", R = 999)
```

corPart

Partial Correlation Matrix

Description

Computes the partial correlation matrix of a set of variables `x` while controlling for another set of variables `y`, based on a covariance/correlation matrix or on raw data.

Usage

```
corPart(m, x, y)
```

Arguments

m	a numeric matrix, either: • a square, symmetric covariance or correlation matrix, or • a data matrix (observations in rows, variables in columns) The two are told apart by symmetry, not by shape alone - a data matrix with as many rows as columns would otherwise be mistaken for a correlation matrix.
x	integer vector of indices specifying the variables of interest for which partial correlations are computed
y	integer vector of indices specifying the control variables (conditioning set)

Details

Partial correlations are read off the precision matrix. Let K be the inverse of the joint covariance matrix of (x, y) ; then

$$\rho_{ij \cdot y} = -K_{ij} / \sqrt{K_{ii}K_{jj}}$$

for i, j in x . This is algebraically equivalent to forming the Schur complement $\Sigma_{xx} - \Sigma_{xy}\Sigma_{yy}^{-1}\Sigma_{yx}$ and scaling it to unit diagonal, but needs a single inversion instead of two.

Because the result is scaled to unit diagonal, it makes no difference whether m is a covariance or a correlation matrix.

Value

a symmetric numeric matrix containing the partial correlations among variables in x , adjusted for variables in y . Row and column names correspond to `colnames(m)[x]`.

Numerical considerations

- The joint submatrix of x and y must be invertible. Near-singularity from collinearity among the control variables is detected via the reciprocal condition number, not merely by a failure of `solve`, which succeeds and returns nonsense well before the matrix is numerically singular.
- x and y must not overlap.
- For raw data, correlations are computed pairwise, which can produce a non-positive-definite matrix when values are missing.

See Also

`cor`, `cov`

Other assoc.continuous: `corPolychor()`, `findCorrX()`, `hoeffdingD()`, `keepSig()`, `pearsonCor()`, `spearmanCor()`

Examples

```
# Simulated data
set.seed(1)
X <- matrix(rnorm(100 * 5), ncol = 5)
colnames(X) <- paste0("V", 1:5)

# Partial correlations of V1, V2 controlling for V3, V4
corPart(X, x = 1:2, y = 3:4)
```

```
# Using a correlation matrix directly
C <- cor(X)
corPart(C, x = 1:2, y = 3:4)

# a single variable of interest is allowed and returns a 1x1 matrix
corPart(C, x = 1, y = 3:4)
```

corPolychor

Polychoric Correlation

Description

Estimates the polychoric correlation between two ordinal variables based on a contingency table. Both a two-step estimator and full maximum likelihood (ml) estimation are supported.

Usage

```
corPolychor(
  x,
  y = NULL,
  method = c("two-step", "ml"),
  se = FALSE,
  control = list(),
  maxcor = 0.9999,
  ...
)
```

Arguments

x	a contingency table or an ordinal vector
y	optional second ordinal vector. If supplied, a contingency table is constructed via <code>table(x, y, ...)</code> .
method	character string specifying the estimation method: "two-step" two-step estimator (default, fast) "ml" full maximum likelihood estimation
se	logical; if TRUE, standard errors are computed via the Hessian matrix. This requires ml estimation, so it is an error to combine it with <code>method = "two-step"</code> .
control	a list of control parameters passed to optim
maxcor	numeric; maximum absolute correlation allowed (default 0.9999) to avoid numerical issues near the boundary
...	further arguments passed to table when y is supplied, for example <code>useNA</code>

Details

The polychoric correlation estimates the correlation between two latent normally distributed variables underlying observed ordinal variables.

The likelihood is based on a discretized bivariate normal distribution, evaluated via [pmvnorm](#).

For numerical stability:

- The correlation parameter is internally transformed using `tanh()` to enforce $|\rho| < 1$. The search range on that scale is derived from `maxcor`, so the estimate is free to approach the documented boundary.
- Cell probabilities are bounded away from zero to avoid $\log(0)$.

Empty rows or columns in the contingency table are removed with a warning.

Value

if `se = FALSE`, a numeric value giving the estimated correlation.

If `se = TRUE`, a list with components:

`type` type of correlation

`rho` estimated polychoric correlation

`rowCuts` estimated row thresholds

`colCuts` estimated column thresholds

`var` variance-covariance matrix of the estimates, on the scale of `rho` and the thresholds. The optimiser works on `atanh(rho)`, so the leading row and column are transformed back with the delta method, factor $(1 - \rho^2)$.

`n` total sample size

`chisq` likelihood-ratio test statistic

`df` degrees of freedom

`method` estimation method actually used

The returned object has class "Polychor".

References

Olsson, U. (1979). Maximum likelihood estimation of the polychoric correlation coefficient. *Psychometrika*, 44(4), 443–460.

Fox, J. (2016). *Applied Regression Analysis and Generalized Linear Models*.

See Also

[pmvnorm](#), [optim](#)

Other `assoc.continuous`: [corPart\(\)](#), [findCorrX\(\)](#), [hoeffdingD\(\)](#), [keepSig\(\)](#), [pearsonCor\(\)](#), [spearmanCor\(\)](#)

Examples

```
# Example with ordinal variables
set.seed(1)
z <- rnorm(200)
x <- factor(cut(z + rnorm(200, sd = 0.6), 3), ordered = TRUE)
y <- factor(cut(z + rnorm(200, sd = 0.6), 3), ordered = TRUE)

# Two-step estimate
corPolychor(x, y)

# ml estimate
corPolychor(x, y, method = "ml")

# With standard errors
res <- corPolychor(x, y, method = "ml", se = TRUE)
res$rho
```

countWorkDays

Count Work Days Between Two Dates

Description

Returns the number of work days between two dates taking into account the provided holiday dates.

Usage

```
countWorkDays(from, to, holiday = NULL, nonworkdays = c("Sat", "Sun"))
```

Arguments

from	initial dates
to	final dates
holiday	a vector of dates (or strings coercible with as.Date) to exclude
nonworkdays	a character vector containing the English three-letter weekday abbreviations to be treated as non-work days, i.e. a subset of c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"). Default is c("Sat", "Sun").

Details

The function is vectorised so that multiple initial and final dates can be supplied. The date vectors are recycled if their lengths differ.

Both endpoints are counted, so countWorkDays(d, d) is 1 for a working day and 0 otherwise. A to that lies before from yields 0.

Holidays outside the interval are ignored, as are holidays falling on a non-work day; duplicated holiday dates are counted once.

Value

an integer vector

See Also

Other date.time: [addMonths\(\)](#), [date-time-predicates](#), [date_functions](#), [generation\(\)](#), [time-conversions](#), [zodiac\(\)](#)

Examples

```
from <- as.Date("2019-01-01") + rep(0, 10)
to   <- as.Date("2019-01-15") + seq(0, 9)

countWorkDays(from, to)

# which weekdays are involved
x <- seq(min(from), max(to), by = "days")
head(data.frame(date = x, day = weekday(x, fmt = "dd", lang = "en")))

# two holidays inside the period reduce the count by two
countWorkDays(from = min(x), to = max(x))
countWorkDays(from = min(x), to = max(x),
               holiday = c("2019-01-07", "2019-01-08"))

# a single day
countWorkDays(as.Date("2019-01-05"), as.Date("2019-01-05")) # Saturday: 0
```

cramerV

*Cramer's V***Description**

Measures the strength of association between two categorical variables. These can be provided as two data vectors *x* and *y*, or as a contingency table (see [Association](#)).

Usage

```
cramerV(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("ncchisq", "ncchisqadj", "fisher", "fisheradj"),
  correct = FALSE,
  ...
)
```

Arguments

<i>x</i>	either a contingency table, a two-column object (matrix, data.frame or list), or a vector of observations (together with <i>y</i>)
<i>y</i>	optional second vector. If <i>x</i> is not a vector, <i>y</i> must be NULL.
<i>conf.level</i>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.

sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
method	character string specifying the confidence interval method: "ncchisq" (default, using the noncentral chi-squared distribution), "ncchisqadj", "fisher" (using fisher z transformation), or "fisheradj" (using the Fisher z transformation and bias correction)
correct	logical; whether to apply the bias correction of Bergsma (2013); defaults to FALSE
...	further arguments, passed on to normalizeToConfusion and table for building the table - useNA is the usual one.

Details

Cramer's V ranges from 0 to 1, with 0 indicating statistical independence.

`correct = TRUE` applies Bergsma's bias correction to the point estimate *and* to the interval: both are put through the same transformation of the chi-squared statistic, so estimate and bounds always live on the same scale. Formerly the two noncentral chi-squared methods returned an uncorrected interval around a corrected estimate.

All four methods are analytical; there is no bootstrap here, and none is needed - unlike [contCoef](#), where no generally accepted analytical interval exists.

For a one-sided interval the bound is computed at the adjusted level $2\gamma - 1$ and the open side is closed at the boundary of the attainable range, 0 or 1, rather than at an infinity V can never reach. `sides = "left"` names the side carrying the *finite* bound and is the analogue of `alternative = "greater"` in a test.

The two Fisher methods rest on $\text{Var}(\text{atanh}(V)) \approx 1/(n - 3)$. Where that approximation has nothing to say - three or fewer observations, or a perfect association, where the transformation is infinite - the bounds are NA and a warning names the reason. The estimate itself is still returned.

Value

if `conf.level = NA`, a numeric scalar containing Cramer's V; otherwise a named numeric vector with elements:

`est` point estimate of Cramer's V.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

Note

Based on code by Michael Smithson (confidence intervals), adapted to conform to package standards.

References

- Cramer, H. (1946) *Mathematical Methods of Statistics*. Princeton University Press
- Agresti, Alan (1996) *Introduction to categorical data analysis*. NY: John Wiley and Sons
- Bergsma, W. (2013) A bias-correction for Cramer's V and Tschuprow's T *Journal of the Korean Statistical Society* 42(3) DOI: 10.1016/j.jkss.2012.10.002

See Also

[base::table](#), [pharos::plotCor](#), [bedrock::pairApply](#), [Association](#)

Other assoc.nominal: [contCoef\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
tab <- table(Pizza$driver, Pizza$wine_delivered)
cramerV(tab)

# just x and y
cramerV(Pizza$driver, Pizza$wine_delivered)

# data.frame
bedrock::pairApply(Pizza[, c("driver", "operator", "area")], cramerV,
  symmetric = TRUE)

# useNA is passed on to table()
bedrock::pairApply(Pizza[, c("driver", "operator", "area")], cramerV,
  useNA = "ifany", symmetric = TRUE)

d.frm <- Pizza[, c("driver", "operator", "area")]
bedrock::pairApply(d.frm[complete.cases(d.frm), ], cramerV, symmetric = TRUE)

# one-sided: "left" carries the finite lower bound, the upper one opens
# to the maximum V can attain
cramerV(tab, conf.level = 0.95, sides = "left")

# Bootstrap confidence intervals for Cramer's V
# http://support.sas.com/documentation/cdl/en/statugfreq/63124/PDF/default/statugfreq.pdf, p. 1821

tab <- as.table(rbind(
  c(26, 26, 23, 18, 9),
  c(6, 7, 9, 14, 23)))
d.frm <- bedrock::untable(tab)

set.seed(1)
n <- 1000
idx <- matrix(sample(nrow(d.frm), size = nrow(d.frm) * n, replace = TRUE),
  ncol = n, byrow = FALSE)
v <- apply(idx, 2, function(x) cramerV(d.frm[x, 1], d.frm[x, 2]))
quantile(v, probs = c(0.025, 0.975))

# compare this to the analytical ones
cramerV(tab, conf.level = 0.95)
```

Description

Cronbach's alpha is a measure of internal consistency and often used for validating psychometric tests. The unstandardized form implemented here is computed from the item variances and the variance of the total score, expressing the proportion of total-score variance not attributable to item-specific variance. This reduces to Kuder-Richardson formula 20 (KR-20) when the columns of the data matrix are dichotomous.

Usage

```
cronbachAlpha(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  returnConditional = FALSE,
  na.rm = FALSE
)
```

Arguments

<code>x</code>	a $n \times m$ matrix or data frame with item responses, n subjects (in rows) and m items (in columns)
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>returnConditional</code>	logical; if TRUE, alpha is additionally calculated for the dataset with each item left out
<code>na.rm</code>	logical; if TRUE, incomplete cases are removed before the computation proceeds

Details

The confidence interval follows Feldt (1965) and is based on the F distribution with $n - 1$ and $(m - 1)(n - 1)$ degrees of freedom, where n is the number of subjects (rows) and m the number of items (columns). It inherits the assumptions of the underlying ANOVA derivation - in particular normally distributed scores and essentially parallel items with homogeneous variances and covariances - and should be read with more caution than the point estimate when these are doubtful, for instance with markedly skewed or heterogeneous items.

`sides` names the side on which the finite bound lies: "left" yields an interval bounded below and "right" one bounded above. Alpha cannot exceed 1, so the open upper side is reported at that boundary rather than as ∞ (design_rules.md 4.1), while the open lower side stays $-\infty$ because alpha is unbounded below.

Missing values are handled according to package conventions: if `na.rm = FALSE` and `x` contains missing values, the usual structure is returned with `NA_real_` in place of every estimate. If `na.rm = TRUE`, complete cases are used. Infinite values leave the variances undefined and are rejected with an error.

Value

a named numeric vector, or a list when `returnConditional = TRUE`.

If `na.rm = FALSE` and `x` contains missing values, the same structure is returned with `NA_real_` throughout.

If `conf.level = NA`, the numeric vector contains only `est`; otherwise it has elements:

`est` point estimate.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

If `returnConditional = TRUE`, a list with the components:

`unconditional` alpha for the full set of items, as above

`conditional` a data frame with one row per item, giving the alpha that would be realized if that item were excluded. `NULL` when `x` has fewer than 3 items, since dropping one would leave too few to compute alpha.

Note

Based on code of Harold C. Doran, adapted to conform to package standards.

References

Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika*, 16(3), 297-334. doi:[10.1007/BF02310555](https://doi.org/10.1007/BF02310555)

Feldt, L. S. (1965). The approximate sampling distribution of Kuder-Richardson reliability coefficient twenty. *Psychometrika*, 30(3), 357-370. doi:[10.1007/BF02289499](https://doi.org/10.1007/BF02289499)

See Also

Other assoc.agreement: [ccc\(\)](#), [cohenKappa\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
set.seed(1234)
tmp <- data.frame(
  item1 = sample(c(0, 1), 20, replace = TRUE),
  item2 = sample(c(0, 1), 20, replace = TRUE),
  item3 = sample(c(0, 1), 20, replace = TRUE),
  item4 = sample(c(0, 1), 20, replace = TRUE),
  item5 = sample(c(0, 1), 20, replace = TRUE)
)

cronbachAlpha(tmp[, 1:4])

cronbachAlpha(tmp[, 1:4], conf.level = 0.95)

# the conditional table is labelled with the column names of x
cronbachAlpha(tmp[, 1:4], returnConditional = TRUE, conf.level = 0.95)

# fewer than 3 items: the conditional component is NULL
cronbachAlpha(tmp[, 1:2], returnConditional = TRUE, conf.level = 0.95)
```

cStat	<i>Concordance Statistic (C-Statistic / AUC)</i>
-------	--

Description

Computes the concordance statistic (C-statistic), equivalent to the area under the ROC curve (AUC), for predicted values and a binary outcome.

Usage

```
cStat(x, ...)

## S3 method for class 'glm'
cStat(x, ...)

## Default S3 method:
cStat(x, resp, conf.level = NA, ...)
```

Arguments

x	an object for which the C-statistic should be computed; for the default method, a numeric vector of predicted values
...	additional arguments passed to methods
resp	a binary response vector (numeric, logical, or factor)
conf.level	confidence level for the interval; NA (default) suppresses interval calculation

Details

The C-statistic is defined as the probability that, for a randomly chosen pair of observations with different outcomes, the observation with the higher predicted value has the higher observed outcome.

Ties in predicted values are handled by assigning a weight of 0.5.

resp is converted with `as.numeric(factor(resp)) - 1`, so the **second** level in sort order counts as the event - 1 for a 0/1 coding, TRUE for a logical, and the second factor level otherwise. Getting this backwards returns $1 - C$ rather than an error, so check the level order when the response is a factor with unusual labels.

This implementation uses:

- $O(n \log n)$ concordance computation
- Parallel bootstrap confidence intervals via **RcppParallel**
- Efficient memory handling

The number of bootstrap samples can be supplied through ... as R; the default is 1000.

Value

if `conf.level = NA`, an unnamed numeric scalar between 0 and 1; otherwise a named numeric vector with elements:

```
est  point estimate of the C-statistic.
lci  lower confidence interval bound.
uci  upper confidence interval bound.
```

Random number generation

A confidence level triggers a bootstrap, which draws a seed from R's global random number generator and therefore advances it. Call `set.seed` beforehand for reproducible intervals.

See Also

Other `assoc.ordinal`: `conDisPairs()`, `kendallW()`, `ordAssocs()`

Examples

```
# Default method
set.seed(1)
x <- runif(100)
y <- rbinom(100, 1, 0.5)
cStat(x, resp = y)

# GLM method
r.mod <- glm(complaint ~ temperature + wrongpizza + wine_ordered,
             data = bedrock::Pizza, family = binomial)
cStat(r.mod, conf.level = 0.95)
```

cut.integer

Cut an Integer Variable into Intervals

Description

A `cut` method for integer vectors. `cut.default()` labels its intervals in the mathematical " $(a,b]$ " notation, which is right for a continuous variable but reads awkwardly for counts: for integers, "11-20" says the same thing more plainly than " $(10,20]$ ".

Usage

```
## S3 method for class 'integer'
cut(
  x,
  breaks,
  labels = NULL,
  include.lowest = FALSE,
  right = TRUE,
  ordered_result = FALSE,
  ...
)
```

Arguments

<code>x</code>	an integer vector
<code>breaks</code>	either a vector of cut points or a single number giving the number of intervals
<code>labels</code>	labels for the levels. The integer-style labels described above are used when this is <code>NULL</code> .
<code>include.lowest</code> , <code>right</code> , <code>ordered_result</code> , ...	passed to <code>cut.default</code>

Details

The integer labels are only constructed when `labels` is `NULL` *and* `breaks` is a vector of whole numbers. A scalar `breaks` (a number of intervals) is passed straight to `cut.default`, which computes the cut points itself, and fractional break points fall back to the default interval notation as well - "34.3-66.6" would suggest an integer range that does not exist.

Infinite outer breaks are rendered as `".."`, so `breaks = c(0, 10, Inf)` yields `"1-10"` and `"11-.."`.

Value

a factor of the same length as `x`

See Also

`cut`, `cutAge`, `cutQ`

Other cut: `cutAge()`, `cutQ()`

Examples

```
x <- as.integer(c(1, 5, 10, 11, 20, 21))

cut(x, breaks = c(0, 10, 20, Inf))

# left-closed intervals shift the labels accordingly
cut(x, breaks = c(0, 10, 20, Inf), right = FALSE)

# a scalar breaks is left to cut.default()
cut(x, breaks = 3)
```

cutAge

Create a Factor Variable by Cutting an Age Variable

Description

Dividing the range of an age variable `x` into intervals is a frequent task in data analysis. The commonly used function `cut` has unfavourable default values for this. `cutAge()` is a convenient wrapper for cutting age variables in groups of e.g. 10 years with more suitable defaults.

Usage

```
cutAge(
  x,
  breaks = c(seq(from = 0, to = 90, by = 10), Inf),
  right = FALSE,
  ordered_result = TRUE,
  full = TRUE,
  labels = NULL,
  ...
)
```

Arguments

x	continuous variable
breaks	either a numeric vector of two or more unique cut points or a single number (greater than or equal to 2) giving the number of intervals into which x is to be cut. Default is 10-year intervals from 0 to 90.
right	logical, indicating if the intervals should be closed on the right (and open on the left) or vice versa. Default is FALSE - unlike in cut !
ordered_result	logical: should the result be an ordered factor? Default is TRUE - unlike in cut !
full	logical; whether to retain empty levels at the edges of the distribution
labels	labels for the levels. When set to TRUE the age ranges will be 00-09, 10-19, 20-29, etc.
...	further arguments passed to cut() , for example to change the labels

Value

a factor, or an integer vector of level codes when labels = FALSE

Values which fall outside the range of breaks are coded as NA, as are NaN and NA values.

See Also

[cut](#), [seq](#)

Other cut: [cut.integer\(\)](#), [cutQ\(\)](#)

Examples

```
set.seed(1)
desc(cutAge(sample(0:100, size = 100, replace = TRUE)))

# readable labels
table(cutAge(c(3, 17, 42, 67, 95), labels = TRUE))

# drop the empty groups at both ends
table(cutAge(c(42, 47, 51), labels = TRUE, full = FALSE))
```

cutQ

Create a Factor Variable Using the Quantiles of a Continuous Variable

Description

Creates a factor variable using the quantiles of a continuous variable.

Usage

```
cutQ(
  x,
  breaks = quantile(x, seq(0, 1, by = 0.25), na.rm = TRUE),
  labels = NULL,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	continuous variable
<code>breaks</code>	cut points used to create groups. By default, quantiles are used. See quantile for details. A single integer specifies the intended number of groups; for example, <code>breaks = 10</code> creates deciles.
<code>labels</code>	labels for the levels of the resulting category. By default, labels are defined as Q1, Q2, and so on. The argument is passed to cut , so <code>labels = FALSE</code> returns integer codes instead of a factor. When quantiles are tied, the levels are built from the observed interval bounds instead and <code>labels</code> is ignored - a warning is issued in that case.
<code>na.rm</code>	logical. Should missing values be removed before the quantiles are computed? Defaults to <code>FALSE</code> . Note that the quantiles themselves are always taken with <code>na.rm = TRUE</code> ; this argument decides whether the missing values are dropped from the returned factor as well.
<code>...</code>	optional arguments passed to cut

Details

This function uses [quantile](#) to obtain the specified quantiles of `x`, then calls [cut](#) to create a factor variable using the intervals specified by these quantiles.

It properly handles cases where more than one quantile obtains the same value, as in the second example below. Note that in this case, there will be fewer generated factor levels than the specified number of quantile intervals.

Value

a factor with one level for each quantile interval, or an integer vector when `labels = FALSE`

Note

Based on code by Gregory R. Warnes, adapted to conform to package standards.

See Also

[cut](#), [quantile](#)

Other cut: [cut.integer\(\)](#), [cutAge\(\)](#)

Examples

```
# create example data
set.seed(1234)
x <- rnorm(1000)

# cut into quartiles
quartiles <- cutQ(x)
table(quartiles)

# cut into deciles
deciles <- cutQ(x, breaks=10, labels=NULL)
table(deciles)

# show handling of 'tied' quantiles.
```

```
x <- round(x) # discretize to create ties
stem(x)       # display the ties
deciles <- cutQ(x, breaks=10)

table(deciles) # note that there are only 5 groups (not 10)
               # due to duplicates
```

date-time-predicates *Date and Time Class Predicates*

Description

Check whether an object contains a date component, a time component, or both, and determine whether its observed times of day vary.

Usage

```
isDate(x)

isTime(x)

isDateTime(x)

hasVaryingTime(x, na.rm = FALSE)
```

Arguments

x	an object to examine
na.rm	logical; if TRUE, missing values are removed before the time components are compared (hasVaryingTime only)

Details

The predicates inspect the class of x; character strings are not parsed or converted. In addition to the base R classes "Date", "POSIXct", and "POSIXlt", the legacy classes "dates", "times", and "chron" are supported.

isDate returns TRUE for objects containing a date component, including date-time objects. isTime returns TRUE for objects containing a time component, including date-time objects. isDateTime identifies classes containing both components, and is by construction equivalent to isDate(x) && isTime(x).

The **chron** classes need care, because "dates" inherits from "times": a pure date object therefore carries the "times" class without having a time component. isTime accounts for this, so that "dates" counts as date-only, "times" as time-only, and "chron" as both.

hasVaryingTime compares the observed times of day while ignoring differences in the date component. It returns FALSE for objects without a time component and when fewer than two non-missing times are available.

For date-time objects the comparison uses local clock time, which is usually what matters when deciding whether times need to be displayed at all. Where a daylight saving transition falls inside x, clock time and elapsed time come apart: two instants an hour apart can share the same clock time in the repeated hour of an autumn transition, and are then reported as not varying.

Value

for `isDate`, `isTime`, and `isDateTime`, a single logical value. `hasVaryingTime` likewise returns a single logical value, or NA if `x` contains missing values and `na.rm = FALSE`.

See Also

[inherits](#), [DateTimeClasses](#)

Other `date.time`: [addMonths\(\)](#), [countWorkDays\(\)](#), [date_functions](#), [generation\(\)](#), [time-conversions](#), [zodiac\(\)](#)

Examples

```
d <- as.Date(c("2024-01-01", "2024-01-02"))
dt <- as.POSIXct(
  c("2024-01-01 08:00:00", "2024-01-02 09:30:00"),
  tz = "UTC"
)

isDate(d)
isTime(d)
isDateTime(d)

isDate(dt)
isTime(dt)
isDateTime(dt)

# character strings are not interpreted as dates
isDate("2024-01-01")

hasVaryingTime(dt)

sameTime <- as.POSIXct(
  c("2024-01-01 08:00:00", "2024-01-02 08:00:00"),
  tz = "UTC"
)
hasVaryingTime(sameTime)

# missing values propagate unless they are removed
withNA <- dt
withNA[2] <- NA
hasVaryingTime(withNA)
hasVaryingTime(withNA, na.rm = TRUE)
```

date_functions

Basic Date Functions

Description

Convenience wrappers for extracting date/time components and performing common date calculations - a readable alternative to [format\(\)](#) and its cryptic format codes.

Usage

```
year(x, ...)

## S3 method for class 'ym'
year(x, ...)

## Default S3 method:
year(x, ...)

month(
  x,
  fmt = c("m", "mm", "mmm"),
  lang = .getOption("lang"),
  stringsAsFactors = TRUE
)

## S3 method for class 'ym'
month(
  x,
  fmt = c("m", "mm", "mmm"),
  lang = .getOption("lang"),
  stringsAsFactors = TRUE
)

## Default S3 method:
month(
  x,
  fmt = c("m", "mm", "mmm"),
  lang = .getOption("lang"),
  stringsAsFactors = TRUE
)

week(x, method = c("iso", "us"))

day(x)

day(x) <- value

weekday(
  x,
  fmt = c("d", "dd", "ddd"),
  lang = .getOption("lang"),
  stringsAsFactors = TRUE
)

quarter(x)

today()

now()

hour(x)
```

```

minute(x)

second(x)

timezone(x)

yearMonth(x)

yearWeek(x, method = c("iso", "us"))

yearDay(x)

diffDays360(startDate, endDate, method = c("eu", "us"))

lastDayOfMonth(x)

yearDays(x)

monthDays(x)

isWeekend(x)

isLeapYear(x)

```

Arguments

<code>x</code>	a Date, POSIXct, POSIXlt, or ym object to evaluate
<code>...</code>	further arguments passed to methods. <code>year()</code> is generic and carries them for the sake of S3 consistency; none of the methods currently uses them.
<code>fmt</code>	format code controlling the output of <code>month()</code> and <code>weekday()</code> . For <code>month()</code> : "m" = integer (default), "mm" = abbreviated name, "mmm" = full name. For <code>weekday()</code> : "d" = integer (default), "dd" = abbreviated name, "ddd" = full name.
<code>lang</code>	language for names returned by <code>month()</code> and <code>weekday()</code> . Either "local" (current system locale, the default) or "en" (English). Falls back to the "lang" option if set; otherwise "local" is used.
<code>stringsAsFactors</code>	logical; if TRUE (default), character results from <code>month()</code> and <code>weekday()</code> are returned as ordered factors whose levels follow calendar order
<code>method</code>	calculation convention. For <code>diffDays360()</code> either "eu" (European, default) or "us" (US); for <code>week()</code> and <code>yearWeek()</code> either "iso" (ISO 8601, default) or "us". The two sets are not interchangeable.
<code>value</code>	replacement value for the <code>day<-</code> assignment function
<code>startDate, endDate</code>	start and end dates for <code>diffDays360()</code>

Value

a vector whose type depends on the function: numeric for integer-valued components, an ordered factor or character vector when `fmt` requests names, logical for `isWeekend()` and `isLeapYear()`,

Date for today() and lastDayOfMonth(), and POSIXct for now()

Date component extractors

Function	Returns	Range / Notes
year	Year of a date or ym object	yyyy
quarter	Quarter of the year	1-4
month	Month of the year (numeric, abbreviated, or full name)	1-12; S3 dispatch for ym
week	Week of the year	ISO 8601 or US convention
day	Day of the month (readable/writable)	1-31
weekday	Day of the week (numeric, abbreviated, or full name)	1 = Mon ... 7 = Sun
yearDay	Day of the year	1-366
yearWeek	Compact year-week integer	yyyyww (ISO or US)
yearMonth	Compact year-month integer	yyyymm

Time component extractors (POSIXct/POSIXlt)

Function	Returns
hour	Hour (0-23)
minute	Minute (0-59)
second	Second (0-60)
timezone	Time zone string
now	Current date and time (Sys.time())
today	Current date (Sys.Date())

Logical tests

Function	Returns
isWeekend	TRUE if x falls on Saturday or Sunday
isLeapYear	TRUE if the year of x is a leap year

Date arithmetic

Function	Description
diffDays360	Days between two dates using the 360-day calendar convention
lastDayOfMonth	Last calendar day of the month of x
yearDays	Total number of days in the year of x (365 or 366)
monthDays	Number of days in the month of x (28-31)

Language for month and weekday names

`month()` and `weekday()` respect the "lang" option. Set `options(lang = "en")` to always get English names, or `options(lang = "local")` for the current system locale. When the option is absent, "local" is used as default.

See Also

[strftime](#), [DateTimeClasses](#), [as.POSIXlt](#), [countWorkDays](#)

Other date.time: [addMonths\(\)](#), [countWorkDays\(\)](#), [date-time-predicates](#), [generation\(\)](#), [time-conversions](#), [zodiac\(\)](#)

Examples

```
x <- today()    # equivalent to Sys.Date()

year(x)
quarter(x)

# month: numeric, abbreviated, full name
month(x)
month(x, fmt = "mm", lang = "en")
month(x, fmt = "mm", lang = "local")
month(x, fmt = "mmm", lang = "en")
month(x, fmt = "mmm", lang = "local")

week(x)
week(x, method = "us")

# day is both readable and writable
day(x)
day(x) <- 20
x

# weekday: numeric, abbreviated, full name
weekday(x)
weekday(x, fmt = "dd", lang = "en")
weekday(x, fmt = "ddd", lang = "local")

yearDay(x)
yearWeek(x)
yearMonth(x)

isWeekend(x)
isLeapYear(x)
isLeapYear(2000L)

# month names for a weekly time sequence
month(seq(Sys.Date(), Sys.Date() + 150, by = "weeks"), fmt = "mm")

# last day of month for several dates
lastDayOfMonth(as.Date(c("2014-10-12", "2013-01-31", "2011-12-05")))

# days in month / year
monthDays(x)
yearDays(x)
```

```
# 360-day calendar difference. The two conventions agree here ...
diffDays360(as.Date("2023-01-31"), as.Date("2023-03-31"))
diffDays360(as.Date("2023-01-31"), as.Date("2023-03-31"), method = "us")

# ... but not here, which is the point of having both
diffDays360(as.Date("2023-01-31"), as.Date("2023-02-28"))
diffDays360(as.Date("2023-01-31"), as.Date("2023-02-28"), method = "us")
```

desc

Describe Data

Description

Produce summaries of various types of variables. Descriptive statistics and plots are chosen automatically depending on the class of *x*. The intention is to provide a fast but rich summary with minimal typing.

Compute descriptive statistics for a dichotomous variable. The plot method displays absolute and relative frequencies in horizontal bar plots.

Usage

```
desc(x, ...)

## S3 method for class 'list'
desc(x, ...)

## S3 method for class 'Desc.list'
print(x, ...)

## S3 method for class 'data.frame'
desc(x, ...)

## S3 method for class 'Desc'
print(x, ...)

## S3 method for class 'Desc'
plot(x, ...)

## S3 method for class 'Desc.AllNA'
print(x, ...)

## S3 method for class 'Desc.AllNA'
plot(x, ...)

## S3 method for class 'Desc.factor'
plot(x, ...)

## S3 method for class 'formula'
desc(
```

```

    formula,
    data,
    subset,
    na.action = na.pass,
    main = NULL,
    verbose = NULL,
    plotit = NULL,
    ...
)

## S3 method for class 'logical'
desc(
  x,
  ord = "level",
  conf.level = 0.95,
  include_x = TRUE,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  digits = NULL,
  ...
)

## S3 method for class 'Desc.logical'
print(x, digits = NULL, ...)

## S3 method for class 'Desc.logical'
plot(x, ...)

## S3 method for class 'Desc.numeric'
print(x, digits = NULL, ...)

## S3 method for class 'Desc.numeric'
plot(x, main = x$meta$main, ...)

```

Arguments

<code>x</code>	a dichotomous numeric, integer, factor, character, or logical vector
<code>...</code>	further arguments passed to methods
<code>formula</code>	formula of the form <code>lhs ~ rhs</code> , where <code>lhs</code> gives the response values and <code>rhs</code> the corresponding groups or explanatory variables
<code>data</code>	optional matrix or data frame (or similar; see model.frame) containing the variables in the formula. By default the variables are taken from <code>environment(formula)</code> .
<code>subset</code>	optional vector specifying a subset of observations to be used in the analysis
<code>na.action</code>	function indicating what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
<code>main</code>	character string, NULL, or NA, defining the main title. By default (<code>main = NULL</code>) the title will be composed as: <code><class(es)></code> . If NA, no title is printed.
<code>verbose</code>	integer controlling verbosity of table output. One of 1 (minimal), 2 (default), 3 (extensive). Applies to tables only.

<code>plotit</code>	logical. Should a plot be created? The plot type depends on the classes of the variables. Default can be defined by the option <code>plotit</code> , if it does not exist then it's set to <code>TRUE</code> .
<code>ord</code>	order of the levels
<code>conf.level</code>	confidence level of the interval (default 0.95). If set to <code>NA</code> , no confidence interval is calculated.
<code>include_x</code>	logical; if <code>TRUE</code> , the original vector is retained in the result
<code>digits</code>	number of digits used to format relative frequencies; the default can be set with <code>setDescToolsXOption(digits = x)</code>

Details

`desc()` is an S3 generic that computes basic descriptive statistics depending on the class of its input. The result is an object of class "desc" with a more specific subclass such as "desc.numeric", "desc.factor" or "desc.data.frame".

For numeric vectors, summary statistics such as mean and standard deviation are computed. For factors, frequency tables are returned. For data frames, `desc()` is applied column-wise.

`desc` is a **generic function**. It dispatches to the method of the class of its first argument.

Typing `?desc + TAB` at the prompt lists all available methods. You usually call `desc(x)`, but direct calls like `desc.numeric(x)` are also possible.

Univariate descriptions

- Numeric variables: `desc.numeric`
- Factors and character vectors: `desc.factor`
- Boolean variables: `desc.logical`
- Contingency tables: `desc.table`
- Dates: `desc.Date`
- Time series: `desc.ts`

Bivariate descriptions

- numeric ~ numeric: `desc.nn`
- numeric ~ qualitative: `desc.nq`
- qualitative ~ numeric: `desc.qn`
- qualitative ~ qualitative: `desc.qq` (wrapper around `desc.table`)

Design The desc system separates:

- computation (internal `.desc_*` functions)
- printing (`print.Desc.*`)
- visualization (`plot.Desc.*`)

Description of a **dichotomous variable**. This can either be a logical vector, a factor with two levels or a numeric variable with only two unique values. The confidence levels for the relative frequencies are calculated by `binomCI()`, method "Wilson" on a confidence level defined by `conf.level`.

Dichotomous variables can be condensed into a compact graphical representation. The method calculates frequencies and binomial confidence intervals and can display them as a dot plot with error bars.

Value

an object of class "Desc" with a subclass determined by the input, such as "Desc.numeric" or "Desc.qn"

an object of class `c("Desc.logical", "Desc")` with components:

`afrq` absolute frequencies

`rfrq` matrix of binomial estimates with columns:

`est` point estimate of the binomial proportion

`lci` lower confidence interval bound

`uci` upper confidence interval bound

See Also

[summary](#), [plot](#)

[plotPropCI](#) for graphical display

Other desc: [desc.Date\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.qq](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.Date

Descriptive statistics for calendar date variables

Description

Computes a structured descriptive summary for objects of class "Date". The description focuses on time-axis characteristics (range, span, coverage, quantiles) and distributional structure over week-days and months.

Usage

```
## S3 method for class 'Date'
desc(
  x,
  main = NULL,
  plotit = NULL,
  verbose = NULL,
  wprobs = rep(1/7, 7),
  mprobs = rep(1/12, 12),
  ...
)
```

Arguments

<code>x</code>	a dichotomous numeric, integer, factor, character, or logical vector
<code>main</code>	character string, NULL, or NA, defining the main title. By default (<code>main = NULL</code>) the title will be composed as: <code><class(es)></code> . If NA, no title is printed.
<code>plotit</code>	logical. Should a plot be created? The plot type depends on the classes of the variables. Default can be defined by the option <code>plotit</code> , if it does not exist then it's set to TRUE.

verbose	integer controlling verbosity of table output. One of 1 (minimal), 2 (default), 3 (extensive). Applies to tables only.
wprobs	numeric vector of length 7 specifying expected probabilities for weekdays (Monday to Sunday). The default is a uniform distribution <code>rep(1/7, 7)</code> .
mprobs	numeric vector of length 12 specifying expected probabilities for months (January to December). If NULL (default), probabilities proportional to the number of days per month in a non-leap year are used.
...	further arguments passed to methods

Details

In addition to core time-axis statistics, observed and expected frequencies for weekdays and months are calculated together with standardized residuals and chi-square p-values. The function also performs heuristic detection of suspicious sentinel dates (e.g., extreme future or implausibly early values) to highlight potential data-quality issues.

The core time-axis summary includes:

- Number of observations and missing values
- Minimum and maximum date
- Span in days (max - min)
- Number of unique observed days
- Coverage: proportion of observed days relative to the total number of calendar days within the observed range
- Fundamental quantiles (5\
- Interquartile range (IQR) in days

Weekday and month distributions are compared to their expected probabilities using chi-square goodness-of-fit tests.

Standardized residuals are defined as

$$(Observed - Expected) / \sqrt{Expected}$$

. They describe the magnitude and direction of deviation from the expected distribution.

Sentinel detection is based on simple heuristics such as extremely large future dates or implausibly early calendar dates. It is meant as a diagnostic aid rather than a formal validation procedure.

Value

an object of class `c("Desc.Date", "Desc")` with components:

core time-axis statistics

weekday observed and expected weekday counts, standardized residuals, and p-value

month observed and expected month counts, standardized residuals, and p-value

sentinel heuristic data-quality diagnostics

meta metadata

See Also

[desc](#), [print.Desc.Date](#)

Other desc: [desc\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.qq](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.factor

*Describe a Factor***Description**

Compute descriptive statistics for a factor or character vector. The plot method displays absolute and relative frequencies in horizontal bar plots.

Usage

```
## S3 method for class 'factor'
desc(
  x,
  maxrows = NULL,
  ord = NULL,
  conf.level = 0.95,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  digits = NULL,
  ...
)

## S3 method for class 'character'
desc(
  x,
  maxrows = NULL,
  ord = NULL,
  conf.level = 0.95,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  digits = NULL,
  ...
)

## S3 method for class 'Desc.factor'
print(x, digits = NULL, ...)
```

Arguments

x	factor, ordered factor, or character vector to describe
maxrows	numeric; defines the maximum number of rows in a frequency table to be reported. For factors with many levels it is often not interesting to see all of them. Default is set to 12 most frequent ones (resp. the first ones if ord is set to "levels" or "names"). For a numeric argument x maxrows is the minimum number of unique values needed for a numeric variable to be treated as continuous. If left to its default NULL, x will be regarded as continuous if it has more than 12 single values. In this case the list of extreme values will be displayed and the frequency table else.

If `maxrows` is < 1 it will be interpreted as percentage. In this case just as many rows, as the `maxrows` most frequent levels will be shown. Say, if `maxrows` is set to 0.8, then the number of rows is fixed so, that the highest cumulative relative frequency is the first one going beyond 0.8.

Setting `maxrows` to `Inf` will unconditionally report all values and also produce a plot with type "h" instead of a histogram.

<code>ord</code>	character out of "name" (alphabetical order), "level", "asc" (by frequencies ascending), "desc" (by frequencies descending) defining the order for a frequency table as used for factors, numerics with few unique values and logicals. Factors (and character vectors) are by default ordered by their descending frequencies, ordered factors by their natural order.
<code>conf.level</code>	confidence level of the interval (default 0.95). If set to NA, no confidence interval is calculated.
<code>main</code>	character string, NULL, or NA, defining the main title. By default (<code>main = NULL</code>) the title will be composed as: <code><class(es)></code> . If NA, no title is printed.
<code>verbose</code>	integer controlling verbosity of table output. One of 1 (minimal), 2 (default), 3 (extensive). Applies to tables only.
<code>plotit</code>	logical. Should a plot be created? The plot type depends on the classes of the variables. Default can be defined by the option <code>plotit</code> , if it does not exist then it's set to TRUE.
<code>digits</code>	number of digits used to format relative frequencies; the default can be set with <code>setDescToolsXOption(digits = x)</code>
<code>...</code>	further arguments passed to methods

Details

This function produces a rich description of a **factor**, containing length, number of NAs, number of levels and detailed frequencies of all levels. The order of the frequency table can be chosen between descending/ascending frequency, labels or levels. For ordered factors the order default is "level". Character vectors are treated as unordered factors. `desc.character()` converts `x` to a factor and then uses the factor method. Ordered factors use their intrinsic level order rather than descending frequency order.

Value

an object of class `c("Desc.factor", "Desc")` containing counts, frequency information, and meta-data

See Also

[plotCatDist](#) for graphical display

Other desc: [desc\(\)](#), [desc.Date\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.qq](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.nn

*Describe a Numeric-Numeric Relationship***Description**

Computes, prints and plots a comprehensive bivariate description for two quantitative variables. The function is dispatched automatically by `desc(y ~ x, data)` when both `y` and `x` are numeric.

Usage

```
.descNN(x, y, conf.level = 0.95)

## S3 method for class 'Desc.nn'
print(x, verbose = NULL, abs.sty = NULL, per.sty = NULL, ...)

## S3 method for class 'Desc.nn'
plot(x, main = x$meta$main, which = 1, verbose = NULL, ...)
```

Arguments

<code>x</code>	numeric predictor for <code>.descNN()</code> , or an object of class "Desc.nn" for the print and plot methods
<code>verbose</code>	integer controlling the amount of output (1, 2, or 3). NULL (default) falls back to <code>x\$meta\$verbose % % getOption("DescTools.verbose", 2)</code> .
<code>abs.sty</code>	format style for counts. NULL falls back to <code>getOption("DescTools.abs.sty")</code> .
<code>per.sty</code>	format style for proportions. NULL falls back to <code>getOption("DescTools.per.sty")</code> .
<code>...</code>	further arguments passed to the underlying plot functions
<code>main</code>	main title for the plot. Defaults to the title stored in <code>x\$meta\$main</code> .
<code>which</code>	integer vector selecting which plots to draw. See Details. NULL (default) selects plots automatically based on <code>verbose</code> .
<code>y</code>	numeric response variable
<code>conf.level</code>	confidence level for interval estimates (default 0.95)

Details**Print output by verbose level:**

`verbose = 1` Summary (n, missings), Pearson r and Spearman r each with confidence interval and effect size label, linear regression coefficients (estimate, CI, significance) and R^2 .

`verbose = 2` (**default**) All of the above, plus residual standard error and Shapiro-Wilk test on residuals.

`verbose = 3` All of the above, plus Breusch-Pagan test for heteroscedasticity and Cook's distance summary.

Confidence intervals are reported throughout instead of standard errors and t-values, using `confint()` for regression coefficients and `corCI()` (Fisher z-transform) for correlations.

Effect size labels for correlations follow Cohen (1988):

negligible	$ rl < 0.10$
small	$0.10 \leq rl < 0.30$
moderate	$0.30 \leq rl < 0.50$
large	$ rl \geq 0.50$

Plot options via which:

which = 1 Scatterplot with linear regression line and confidence band.

which = 2 Scatterplot with Loess smoother and confidence band (via `lines.loess()`).

which = 3 Residual plot: residuals vs. fitted values.

which = 4 Q-Q plot of residuals.

Default which by verbose level:

- verbose = 1: which = 1
- verbose = 2: which = 1:2
- verbose = 3: which = 1:4

Value

`.descNN()` returns an object of class `c("Desc.nn", "Desc")`. Its `lm$intercept` and `lm$slope` components contain:

`est` point estimate of the coefficient

`lci` lower confidence interval bound

`uci` upper confidence interval bound

`p` p-value

The print and plot methods return x invisibly.

References

Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.

Breusch, T.S. and Pagan, A.R. (1979). A simple test for heteroscedasticity and random coefficient variation. *Econometrica*, 47, 1287–1294.

See Also

`desc` for the generic entry point, `desc.nq` for numeric ~ categorical, `desc.qn` for categorical ~ numeric, `desc.qq` for categorical ~ categorical, `corCI`, `bpTest`, `lm`, `cor.test`

Other desc: `desc()`, `desc.Date()`, `desc.factor()`, `desc.nq`, `desc.numeric()`, `desc.qn`, `desc.qq`, `desc.ts()`, `print.Desc.qq()`

Examples

```
# basic usage via desc()
desc(mpg ~ wt, mtcars)

# more detail
desc(mpg ~ wt, mtcars, verbose = 3)

# store result and plot separately
d <- desc(mpg ~ wt, mtcars, plotit = FALSE)
print(d, verbose = 1)
plot(d, which = 1:2)

# pipe
desc(mpg ~ wt, mtcars) |> plot(which = 3)
```

desc.nq

*Describe Relationship: Numeric x by Categorical g***Description**

Computes descriptive statistics for a numeric variable *x* grouped by a categorical variable *g*.

Usage

```
.descNQ(x, g, ...)

## S3 method for class 'Desc.nq'
print(x, digits = NULL, ...)

## S3 method for class 'Desc.nq'
plot(x, main = x$meta$main, which = NULL, ...)
```

Arguments

<i>x</i>	a numeric variable
<i>digits</i>	number of digits used to format relative frequencies
<i>...</i>	further arguments, currently unused
<i>main</i>	main title for the plot; defaults to the title stored in <i>x\$meta\$main</i>
<i>which</i>	integer vector selecting which plots to draw. See Details. NULL (default) selects plots automatically based on verbose.
<i>g</i>	a categorical grouping variable (factor or coercible to factor)

Details

The function summarizes the distribution of *x* across levels of *g* and performs nonparametric tests of group differences.

Computed statistics

- Group-wise descriptive statistics (mean, median, SD, IQR, counts)

- Kruskal-Wallis test
- Effect size (η^2) based on the Kruskal-Wallis statistic
- Levene's test for homogeneity of variance

Interpretation The Kruskal-Wallis test evaluates whether the distribution of x differs between groups defined by g . The effect size η^2 provides a standardized measure of group differences.

Value

an object of class `c("Desc.nq", "Desc")` with components:

`tab` group-wise summary table

`test` result of the Kruskal-Wallis test

`vtest` result of Levene's test

`eta` effect size

See Also

[desc](#), [desc.qn](#), [desc.nn](#), [pharos::plot.Desc.qn](#) [kruskal.test](#), [lumen::leveneTest](#)

Other desc: [desc\(\)](#), [desc.Date\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.qq](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.numeric

Describe a Numeric Variable

Description

Compute descriptive statistics for a numeric vector. The plot method may display a histogram, density curve, box plot, and empirical distribution.

Usage

```
## S3 method for class 'numeric'
desc(
  x,
  maxrows = NULL,
  conf.level = 0.95,
  include_x = TRUE,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  digits = NULL,
  ...
)
```

Arguments

<code>x</code>	numeric vector to describe, or an object of class "Desc.numeric" for the print and plot methods
<code>maxrows</code>	numeric; defines the maximum number of rows in a frequency table to be reported. For factors with many levels it is often not interesting to see all of them. Default is set to 12 most frequent ones (resp. the first ones if <code>ord</code> is set to "levels" or "names").
<code>conf.level</code>	confidence level for interval estimates (default 0.95)
<code>include_x</code>	logical; if TRUE, the original vector is retained in the result
<code>main</code>	character string, NULL, or NA, defining the main title. By default (<code>main = NULL</code>) the title will be composed as: <code><class(es)></code> . If NA, no title is printed.
<code>verbose</code>	integer controlling verbosity of table output. One of 1 (minimal), 2 (default), 3 (extensive). Applies to tables only.
<code>plotit</code>	logical. Should a plot be created? The plot type depends on the classes of the variables. Default can be defined by the option <code>plotit</code> , if it does not exist then it's set to TRUE.
<code>digits</code>	number of digits used to format numeric values
<code>...</code>	further arguments passed to methods

Details

This function is an S3 method for `plot`. It is automatically dispatched when calling `plot(x)` on a `Desc.numeric` object.

For a numeric argument `x` `maxrows` is the minimum number of unique values needed for a numeric variable to be treated as continuous. If left to its default NULL, `x` will be regarded as continuous if it has more than 12 single values. In this case the list of extreme values will be displayed and the frequency table else.

If `maxrows` is < 1 it will be interpreted as percentage. In this case just as many rows, as the `maxrows` most frequent levels will be shown. Say, if `maxrows` is set to 0.8, then the number of rows is fixed so, that the highest cumulative relative frequency is the first one going beyond 0.8.

Setting `maxrows` to `Inf` will unconditionally report all values and also produce a plot with type "h" instead of a histogram.

Named colors defined by `DescToolsX` (e.g. "hred", "hblue") can be used directly.

Value

an object of class `c("Desc.numeric", "Desc")` containing descriptive statistics, frequency information, and metadata

See Also

`base::summary()`, `base::plot()`

Other Statistical summary functions: `abstract()`

Other desc: `desc()`, `desc.Date()`, `desc.factor()`, `desc.nn`, `desc.nq`, `desc.qn`, `desc.qq`, `desc.ts()`, `print.Desc.qq()`

Examples

```
desc(Pizza$delivery_min)           # numeric
```

desc.qn

*Describe Relationship: Categorical y vs Numeric x***Description**

Computes descriptive statistics for the relationship between a categorical variable *y* and a numeric variable *x*.

Usage

```
.descQN(y, x, conf.level = 0.95, breaks, right)
```

```
## S3 method for class 'Desc.qn'
print(x, verbose = NULL, ...)
```

Arguments

<i>x</i>	a numeric variable
<i>verbose</i>	amount of printed output
<i>...</i>	further arguments passed to methods
<i>y</i>	a categorical variable (factor or coercible to factor)
<i>conf.level</i>	confidence level for interval estimates (default 0.95)
<i>breaks</i>	numeric vector defining cut points for <i>x</i> . If not supplied, quartiles of <i>x</i> are used.
<i>right</i>	logical; passed to <code>cut()</code> , defining interval closure

Details

The function summarizes how a numeric variable *x* differs across levels of a categorical variable *y*.

Computed statistics

- Group-wise descriptive statistics (median, IQR, counts)
- Kruskal-Wallis test with effect size (η^2)
- Levene's test for homogeneity of variance
- Kendall's Tau-b with confidence interval and p-value
- Spearman correlation (reported for higher verbosity levels)

Binary outcomes If *y* has two levels:

- Area under the curve (AUC)
- Prevalence across quantile groups of *x*
- Cochran-Armitage trend test

Quantile grouping The numeric variable *x* is optionally discretized using *breaks*. By default, quartiles are used.

Value

an object of class `c("Desc.qn", "Desc")` with components:

`grpTable` group-wise summary table

`kw` result of the Kruskal-Wallis test

`eta2` effect size

`levene` result of Levene's test

`tauB` estimate, confidence interval, and p-value for Kendall's tau-b

`spearman` estimate, confidence interval, and p-value for Spearman's correlation

`auc` area under the curve for a binary outcome

`prevTable` prevalence table for a binary outcome with columns:

`quantile` quantile group

`n` number of complete cases in the group

`est` point estimate of the prevalence

`lci` lower confidence interval bound

`uci` upper confidence interval bound

`caTest` result of the Cochran-Armitage test for a binary outcome

See Also

[desc](#), [desc.nn](#), [desc.nq](#), [kruskal.test](#), [leveneTest](#)

Other desc: [desc\(\)](#), [desc.Date\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qq](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.qq

Describe Relationship: Categorical x by Categorical y

Description

Computes descriptive statistics for the relationship between two categorical variables `x` and `y`.

Usage

```
.descQQ(x, y, ...)
```

Arguments

`x` a categorical variable

`y` a categorical variable

`...` further arguments, currently unused

Details

This function is a wrapper around [desc.table](#) applied to the contingency table `table(x, y)`.

It summarizes the joint distribution of two categorical variables and provides association measures and visualizations.

Computed statistics

- Contingency table
- Row and column percentages
- Association measures (e.g., Cramer's V, Phi)
- Optional statistical tests depending on configuration

Implementation note Internally, `desc.qq(x, y)` is equivalent to:

```
desc(table(x, y))
```

Value

an object of class `c("Desc.qq", "Desc")`

See Also

[desc](#), [desc.table](#), [desc.qn](#), [desc.nn](#), [pharos::plot.Desc.table](#)

Other desc: [desc\(\)](#), [desc.Date\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.ts\(\)](#), [print.Desc.qq\(\)](#)

desc.ts

Diagnostic Summary for Time Series Objects

Description

Provides a compact diagnostic summary for univariate ts objects, extending classical descriptive statistics with key time series diagnostics.

Usage

```
## S3 method for class 'ts'
desc(x, maxLag = 12, main = NULL, plotit = NULL, verbose = NULL, ...)

## S3 method for class 'Desc.ts'
print(x, digits = NULL, ...)

## S3 method for class 'Desc.ts'
plot(x, ...)
```

Arguments

<code>x</code>	a univariate object of class "ts"
<code>maxLag</code>	number of lags used in the Ljung-Box test; defaults to 12
<code>main</code>	character string, NULL, or NA, defining the main title. By default (<code>main = NULL</code>) the title will be composed as: <code>(<class(es)>)</code> . If NA, no title is printed.
<code>plotit</code>	logical. Should a plot be created? The plot type depends on the classes of the variables. Default can be defined by the option <code>plotit</code> , if it does not exist then it's set to TRUE.
<code>verbose</code>	integer controlling verbosity of table output. One of 1 (minimal), 2 (default), 3 (extensive). Applies to tables only.
<code>...</code>	further arguments passed to methods
<code>digits</code>	number of digits used to format numeric values

Details

The function reports:

- Lag-1 autocorrelation
- Ljung-Box test for overall autocorrelation
- Augmented Dickey-Fuller (ADF) test
- KPSS test
- Linear trend estimation (slope and p-value)
- Suggested Box-Cox transformation parameter

The goal is to provide quick diagnostic guidance before model fitting (e.g., ARIMA specification).

Stationarity is evaluated using both the Augmented Dickey-Fuller (ADF) and KPSS tests. A combined decision rule is used: the series is considered stationary if the ADF test rejects the null hypothesis of a unit root ($p < 0.05$) and the KPSS test does not reject the null hypothesis of stationarity ($p > 0.05$).

The Box-Cox transformation parameter is estimated using `boxCoxLambda()`.

Value

an object of class `c("Desc.ts", "Desc")` containing the computed statistics

References

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control.

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice.

See Also

`stats::acf`, `stats::Box.test`, `boxCoxLambda`, `lumen::adfTest`, `lumen::kpssTest`, `pharos::plotTimeSeries`

Other desc: `desc()`, `desc.Date()`, `desc.factor()`, `desc.nn`, `desc.nq`, `desc.numeric()`, `desc.qn`, `desc.qq`, `print.Desc.qq()`

Examples

```
desc(AirPassengers)
desc(Nile, maxLag = 10)
```

DescToolsX	<i>DescToolsX: Descriptive Statistics and Exploratory Data Analysis</i>
------------	---

Description

DescToolsX is the sequel to DescTools which provides a modern, consistent, and extensible framework for descriptive statistics, statistical tests, confidence intervals, and exploratory data analysis.

Details

The package is a deliberate redesign inspired by DescTools, with a strong emphasis on naming consistency, predictability, and avoidance of name clashes with base R and other packages.

Statistical summary functions that would otherwise mask base R functions are suffixed with X (e.g. meanX(), sdX(), medianX()). So functions ending in X are extended versions (typically supporting weights or implementing confidence intervals) provided by DescToolsX and are designed to coexist with base R functions without masking them.

DescToolsX follows a strict and consistent naming scheme to ensure predictability and to avoid name clashes with base R and other packages.

Consistency and predictability take precedence over historical base R naming conventions. This design choice is a key difference between DescToolsX and DescTools.

Design principles

DescToolsX follows a set of strict design principles to ensure consistency, usability, and performance across the entire package.

Further principles

Following section explain further principles valid throughout **DescToolsX**:

Formulas	Handling formulas
Association	Association
Agreement	Interrater agreement
ConfidenceIntervals	Confidence intervals
Association	Measures
Association	Plots

Function names

All functions use **lower camelCase**.

Statistical summary functions

Statistical functions that would otherwise mask base R functions are suffixed with X. This explicitly signals an extended or modified implementation.

Examples: meanX(), medianX(), sdX(), madX(), iqrX(), varX(), quantileX(), skew(), kurt()

Confidence interval functions

Functions computing confidence intervals use the suffix `CI`, following established R conventions.

Examples: `meanCI()`, `medianCI()`, `sdCI()`, `varCI()`, `quantileCI()`

Confidence interval functions follow the same argument order as their corresponding point estimators. In particular, `conf.level` is always used to specify the confidence level and appears explicitly as a named argument.

Examples: `meanCI(x, conf.level = 0.95)`, `medianCI(x, conf.level = 0.95)`

Statistical tests

Statistical tests use lower camelCase and end with `Test`.

Examples: `shapiroFranciaTest()`, `andersonDarlingTest()`, `leveneTest()`, `jarqueBeraTest()`

Plot functions

Plotting functions start with the prefix `plot` and use lower camelCase.

Examples: `plotQQ()`, `plotECDF()`, `plotCor()`, `plotViolin()`

Plotting functions follow the same data-first argument convention. The data object is always the first argument, followed by plot-specific parameters and graphical options. This ensures intuitive usage and consistent behaviour across different plot types.

Examples: `plotQQ(x)`, `plotECDF(x)`, `plotCor(x, method = "spearman")`

Classes and S3 methods

Classes use **UpperCamelCase**. S3 methods follow standard R conventions.

Examples: `desc.numeric`, `percTable`, `print.PercTable`, `plot.Desc.numeric`

Argument order

Function arguments follow a consistent and predictable order:

1. `x` (primary data object)
2. method-specific parameters
3. confidence-related parameters (e.g. `conf.level`)
4. formatting and display options
5. ... (additional arguments)

This ordering is applied uniformly across statistical summary functions, confidence interval functions, and plotting functions.

Performance and implementation

Computationally intensive functionality is systematically reimplemented using **Rcpp**. This replaces former pure R implementations and results in substantially improved runtime performance while preserving numerical accuracy and user-facing behaviour.

Performance improvements are a core design goal of DescToolsX and a key motivation for the package redesign.

`pharos` is listed in `Depends` because this package extends its user-facing API and expects it to be attached. Functions used internally are explicitly imported via the `NAMESPACE`.

Author(s)

Maintainer: Andri Signorell <andri@signorell.net> ([ORCID](#))

Authors:

- Andri Signorell <andri@signorell.net> ([ORCID](#))

Other contributors:

- R-Core [contributor]
- Andreas Alfons [contributor]
- Nanina Anderegg [contributor]
- Antonio [contributor]
- Tomas Aragon [contributor]
- Antti Arppe [contributor]
- Markus Brueckl [contributor]
- Leanne Chhay [contributor]
- Michael Dewey [contributor]
- Harold C. Doran [contributor]
- Romain Francois [contributor]
- Matthias Gamer [contributor]
- Rob J. Hyndman [contributor]
- Max Kuhn [contributor]
- Jim Lemon [contributor]
- Martin Maechler [contributor]
- Arni Magnusson [contributor]
- David Meyer [contributor]
- Yongyi Min [contributor]
- Cyril F. Moser [contributor]
- Markus Naepflin [contributor]
- Danielle Navarro [contributor]
- Sandrine Pavoine [contributor]
- Roland Rapold [contributor]
- William Revelle [contributor]
- Tyler Rinker [contributor]
- Nathan Russell [contributor]
- Luis Gustavo Schuck [contributor]
- Michael Smithson [contributor]
- Werner A. Stahel [contributor]
- Mark Stevenson [contributor]
- Ralf Stubner [contributor]
- Matthias Templ [contributor]
- Luis Torgo [contributor]

- Gregory R. Warnes [contributor]
- Daniel Wollschlaeger [contributor]
- Joseph Wood [contributor]
- Achim Zeileis [contributor]
- OpenAI Codex (AI-assisted code and documentation review; all suggestions reviewed by the maintainer) [contributor]
- Anthropic Claude (AI-assisted code and documentation review; all suggestions reviewed by the maintainer) [contributor]

See Also

Useful links:

- <https://andrisignorell.github.io/DescToolsX/>
- <https://github.com/AndriSignorell/DescToolsX/>
- Report bugs at <https://github.com/AndriSignorell/DescToolsX/issues>

divCoef	<i>Compute a diversity coefficient</i>
---------	--

Description

Computes Rao's quadratic diversity coefficient for each column of a data frame, optionally using a provided distance matrix.

Usage

```
divCoef(x, dis = NULL, normalize = FALSE, na.rm = FALSE, tol = 0.00000001)
```

Arguments

<code>x</code>	a data frame or matrix of non-negative values (e.g. abundances). Rows correspond to entities, columns to samples.
<code>dis</code>	optional object of class <code>dist</code> . If <code>NULL</code> , a default Euclidean distance is used, in which case the coefficient reduces to the Gini-Simpson index $1 - \sum p_i^2$.
<code>normalize</code>	logical; if <code>TRUE</code> , the diversity is scaled by its theoretical maximum
<code>na.rm</code>	logical; if <code>TRUE</code> , columns containing missing values yield NA instead of aborting
<code>tol</code>	numeric convergence tolerance for the iterative maximisation used by <code>normalize</code> = <code>TRUE</code>

Details

The diversity coefficient is defined as

$$D = \frac{x^T D^2 x}{2(\sum x)^2}$$

where x is a column of x and D is the distance matrix.

If `normalize = TRUE`, values are divided by the maximum achievable diversity under the given distance matrix. That maximum is found by a fixed-point iteration over the simplex, which is a heuristic: it is not guaranteed to reach the global optimum for an arbitrary distance matrix. A warning is issued when the iteration has not converged within `tol`.

Value

a numeric vector of diversity coefficients, one per column

See Also

Other inequality: [atkinson\(\)](#), [gini\(\)](#), [lc\(\)](#), [rosenbluth\(\)](#), [theil\(\)](#)

Examples

```
set.seed(1)
x <- matrix(runif(20), ncol = 4)
d <- dist(matrix(rnorm(10), ncol = 2))

divCoef(x, d)
divCoef(x, d, normalize = TRUE)

# without a distance matrix this is the Gini-Simpson index
divCoef(matrix(c(1, 1, 1, 1, 0, 0), ncol = 2))
```

entropy	<i>Shannon Entropy</i>
---------	------------------------

Description

Computes the Shannon entropy of a categorical vector, contingency table, or matrix.

Usage

```
entropy(x, y = NULL, base = 2, normalize = FALSE, na.rm = FALSE, ...)
```

Arguments

x	a table, matrix or array of counts, or a categorical vector (factor, character or logical), which is tabulated first
y	an optional second variable used together with x to create a contingency table via <code>table(x, y, ...)</code>
base	logarithm base; defaults to 2 (bits)
normalize	logical. If TRUE, entropy is normalized to the interval $[0, 1]$.
na.rm	logical; if TRUE, missing counts are dropped. A categorical x is tabulated with table , which excludes NA by default in any case.
...	additional arguments passed to <code>table()</code>

Details

Entropy is defined as:

$$H(X) = - \sum_i p_i \log_b(p_i)$$

where p_i are empirical probabilities and b is the logarithm base.

Common logarithm bases:

base = 2	entropy in bits
base = exp(1)	entropy in nats
base = 10	entropy in bans

Zero probabilities are ignored in the summation.

normalize = TRUE divides by $\log_b k$, with k the number of *occupied* categories rather than the number of possible ones. The maximum of 1 is therefore reached whenever the observed categories are equally frequent, independently of how many empty levels the input carries. With a single occupied category the normalized value is 0.

Value

a numeric scalar containing the entropy

References

Shannon CE (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379-423.

See Also

Other diversity.concentration: [herfindahl\(\)](#), [simpson\(\)](#)

Examples

```
x <- c("A", "A", "B", "B", "C")

entropy(x)

tab <- matrix(c(10, 20,
                30, 40), nrow = 2)

entropy(tab)
entropy(tab, normalize = TRUE)

# a fair coin carries exactly one bit
entropy(c(50, 50))
```

Description

Calculates eta-squared, partial eta-squared and generalized eta-squared

Usage

```
etaSq(fit, type = 2, anova = FALSE)

## S3 method for class 'lm'
etaSq(fit, type = 2, anova = FALSE)

## S3 method for class 'aovlist'
etaSq(fit, type = 2, anova = FALSE)

aovlDetails(fit)

aovlErrorTerms(fit)
```

Arguments

<code>fit</code>	an analysis of variance object of class "aov" or "aovlist"
<code>type</code>	type of sums of squares to calculate. <code>etaSq.aovlist()</code> requires <code>type = 1</code> .
<code>anova</code>	logical; whether to include the full ANOVA table with the effect sizes

Details

Calculates the eta-squared, partial eta-squared, and generalized eta-squared measures of effect size that are commonly used in analysis of variance. The input `fit` should be the analysis of variance object itself. For between-subjects designs, generalized eta-squared equals partial eta-squared. The reported generalized eta-squared for repeated-measures designs assumes that all factors are manipulated, i.e., that there are no measured factors like gender (see references).

For unbalanced designs, the default in `etaSq` is to compute Type II sums of squares (`type=2`), in keeping with the `Anova` function in the `car` package. It is possible to revert to the Type I SS values (`type=1`) to be consistent with `anova`, but this rarely tests hypotheses of interest. Type III SS values (`type=3`) can also be computed. `etaSq.aovlist` requires `type=1`.

Value

for `etaSq.lm()`, a numeric matrix with one row per model term and columns `eta.sq` and `eta.sq.part`; for `etaSq.aovlist()`, a numeric matrix that additionally contains `eta.sq.gen`. If `anova = TRUE`, ANOVA statistics are included in additional columns. `aovlDetails()` returns a data frame of effect terms and their ANOVA statistics, while `aovlErrorTerms()` returns a list with components:

```
SS  error sums of squares
MS  error mean squares
DF  error degrees of freedom
```

Note

Based on code by Danielle Navarro, and Daniel Wollschlaeger.

References

Bakeman, R. (2005). Recommended effect size statistics for repeated measures designs. *Behavior Research Methods* 37(3), 379-384.

Olejnik, S. and Algina, J. (2003). Generalized Eta and Omega Squared Statistics: Measures of Effect Size for Some Common Research Designs. *Psychological Methods* 8(4), 434-447.

See Also

[aov](#), [anova](#), [Anova](#)

Other effect.size: [cohenD\(\)](#), [cohenH\(\)](#), [glassDelta\(\)](#), [oddsRatio\(\)](#), [relRisk\(\)](#)

Examples

```
#### Example 1: one-way ANOVA ####

outcome <- c(1.4,2.1,3.0,2.1,3.2,4.7,3.5,4.5,5.4) # data
treatment1 <- factor(c(1,1,1,2,2,2,3,3,3)) # grouping variable
anova1 <- aov(outcome ~ treatment1) # run the ANOVA
summary(anova1) # print the ANOVA table
etaSq(anova1) # effect size

#### Example 2: two-way ANOVA ####

treatment2 <- factor(c(1,2,3,1,2,3,1,2,3)) # second grouping variable
anova2 <- aov(outcome ~ treatment1 + treatment2) # run the ANOVA
summary(anova2) # print the ANOVA table
etaSq(anova2) # effect size

#### Example 3: two-way ANOVA unbalanced cell sizes ####
#### data from Maxwell & Delaney, 2004 #####
#### Designing experiments and analyzing data #####

dfMD <- data.frame(IV1=factor(rep(1:3, c(3+5+7, 5+6+4, 5+4+6))),
                  IV2=factor(rep(rep(1:3, 3), c(3,5,7, 5,6,4, 5,4,6))),
                  DV=c(c(41, 43, 50), c(51, 43, 53, 54, 46), c(45, 55, 56, 60, 58, 62, 62),
                      c(56, 47, 45, 46, 49), c(58, 54, 49, 61, 52, 62), c(59, 55, 68, 63),
                      c(43, 56, 48, 46, 47), c(59, 46, 58, 54), c(55, 69, 63, 56, 62, 67)))

# use contr.sum for correct sum of squares type 3
dfMD$IV1s <- C(dfMD$IV1, "contr.sum")
dfMD$IV2s <- C(dfMD$IV2, "contr.sum")
dfMD$IV1t <- C(dfMD$IV1, "contr.treatment")
dfMD$IV2t <- C(dfMD$IV2, "contr.treatment")

etaSq(aov(DV ~ IV1s*IV2s, data=dfMD), type=3)
etaSq(aov(DV ~ IV1t*IV2t, data=dfMD), type=1)

#### Example 4: two-way split-plot ANOVA -> etaSq.aovlist ####

set.seed(1)
DV_t1 <- round(rnorm(3*10, -0.5, 1), 2)
DV_t2 <- round(rnorm(3*10, 0, 1), 2)
DV_t3 <- round(rnorm(3*10, 0.5, 1), 2)
dfSPF <- data.frame(id=factor(rep(1:(3*10), times=3)),
                  IVbtw=factor(rep(LETTERS[1:3], times=3*10)),
                  IVwth=factor(rep(1:3, each=3*10)),
                  DV=c(DV_t1, DV_t2, DV_t3))
spf <- aov(DV ~ IVbtw*IVwth + Error(id/IVwth), data=dfSPF)
etaSq(spf, type=1, anova=TRUE)
```

expFreq	<i>Expected Frequencies</i>
---------	-----------------------------

Description

Calculate the expected frequencies of an n-way table assuming independence.

Usage

```
expFreq(x, freq = c("abs", "rel"))
```

Arguments

x	a table
freq	whether absolute or relative frequencies are computed. Must be "abs" or "rel"; partial matching is supported.

Details

Expected values for a 2-dimensional table can be easily calculated with `res <- chisq.test(x)` and accessed by `res$expected`. This approach fails for higher dimensional tables, for which there is no support. `expFreq()` also returns expected frequencies for higher dimensional tables.

Value

a table with either the absolute or the relative expected frequencies, of the same dimension and dimnames as x

Note

Based on code by David Meyer previously published as `independence_table` in **vcd**, adapted to conform to package standards.

See Also

[chisq.test](#)

Other frequency: [freq\(\)](#), [freq2D\(\)](#), [percTable\(\)](#), [tOne\(\)](#)

Examples

```
expFreq(Titanic)

expFreq(UCBAdmissions, freq="r")
```

extremes

*Kth Smallest/Largest Values***Description**

Find the kth smallest, resp. largest values from a vector `x` and return the values and their frequencies.

Usage

```
large(x, k = 5L, unique = FALSE, na.last = NA)
```

```
small(x, k = 5L, unique = FALSE, na.last = NA)
```

```
highLow(x, nlow = 5L, nhigh = nlow, na.last = NA)
```

Arguments

<code>x</code>	a numeric vector
<code>k</code>	a positive integer defining how many extreme values are returned. Default is <code>k = 5</code> . If <code>k > length(x)</code> , all values will be returned.
<code>unique</code>	logical, defining if unique values should be considered or not. If this is set to <code>TRUE</code> , a list with the <code>k</code> extreme values and their frequencies is returned. Default is <code>FALSE</code> (as <code>unique</code> is a rather expensive function).
<code>na.last</code>	for controlling the treatment of NAs. If <code>TRUE</code> , missing values in the data are put last; if <code>FALSE</code> , they are put first; if <code>NA</code> , they are removed.
<code>nlow</code>	number of smallest values included in the formatted output; defaults to 5
<code>nhigh</code>	number of largest values included in the formatted output; defaults to <code>nlow</code>

Details

This does not seem to be a difficult problem at first sight. We could simply tabulate and sort the vector and finally take the first or last `k` values. However sorting and tabulating the whole vector when we're just interested in the few smallest values is a considerable waste of resources. This approach becomes already impracticable for medium vector lengths ($\sim 10^5$). There are several points and solutions of this problem discussed out there. The present implementation is based on highly efficient C++ code and proved to be very fast.

`highLow()` combines the two upper functions and reports the `k` extreme values on both sides together with their frequencies in parentheses. It is used for describing univariate variables and is interesting for checking the ends of the vector, where in real data often wrong values accumulate. In essence this is nothing more than a combine printing routine for `large()` and `small()`.

Value

for `large()` and `small()`, a vector of extreme values when `unique = FALSE`, otherwise a list with components:

value extreme values

frequency corresponding frequencies

`highLow()` returns a character scalar containing formatted lowest and highest values with frequencies.

Note

Based on C++ code by Nathan Russell and Romain Francois, adapted to conform to package standards.

References

[StackOverflow: Largest n unique values](#)

[Rcpp Gallery article](#)

See Also

[max](#), [min](#), [sort](#), [rank](#)

Other quantile: [quantileX\(\)](#)

Examples

```
x <- sample(1:10, 1000, rep=TRUE)
large(x, 3)
large(x, k=3, unique=TRUE)

# works fine up to x ~ 1e6
x <- runif(1000000)
small(x, 3, unique=TRUE)
small(x, 3, unique=FALSE)

# Both ends
cat(highLow(bedrock::Pizza$temperature, na.last=NA))
```

findCorrX

Identify Highly Correlated Variables

Description

Identifies variables in a correlation matrix that should be removed due to high pairwise correlations above a specified cutoff. The algorithm uses a greedy approach similar to `caret::findCorrelation()`, but extends it with flexible scoring methods and multiple output formats.

Usage

```
findCorrX(
  x,
  cutoff = 0.9,
  method = c("mean", "max", "median"),
  output = c("index", "names", "logical", "report"),
  verbose = FALSE
)
```

Arguments

<code>x</code>	a symmetric correlation matrix
<code>cutoff</code>	numeric threshold in (0, 1). Pairs with absolute correlation above this value are considered too highly correlated.
<code>method</code>	character string specifying how variable importance is scored. One of "mean", "max", or "median". Default is "mean".
<code>output</code>	character string specifying the return format: • "index" for indices of variables to remove (default) • "names" for column names of variables to remove • "logical" for a logical vector indicating removed variables • "report" for a detailed list of removed and retained variables and the decision log
<code>verbose</code>	logical; if TRUE, progress information is printed

Details

The function iteratively examines pairs of variables with correlations above `cutoff`. For each such pair, the variable with the higher overall correlation (based on `method`) is removed.

The scoring is computed once at the beginning and kept fixed throughout, ensuring deterministic and efficient behavior. Because the columns are then processed in decreasing score order, the variable removed from a pair is always the earlier - i.e. the higher-scoring - one.

This is a greedy heuristic and does not guarantee a globally optimal solution.

Value

depending on output:

"index" integer vector of column indices to remove

"names" character vector of column names

"logical" logical vector with one element per column

"report" list with elements removed, kept, and log

See Also

Other `assoc.continuous`: [corPart\(\)](#), [corPolychor\(\)](#), [hoeffdingD\(\)](#), [keepSig\(\)](#), [pearsonCor\(\)](#), [spearmanCor\(\)](#)

Examples

```
set.seed(123)
m <- matrix(rnorm(100), ncol = 5)
colnames(m) <- paste0("V", 1:5)
cmat <- cor(m)

findCorrX(cmat, cutoff = 0.8)
findCorrX(cmat, cutoff = 0.8, method = "max", output = "names")
```

Formulas

*Formula Interface - Common Arguments***Description**

Common formula-based interface shared by multiple functions.

Arguments

formula	formula of the form lhs ~ rhs, where lhs gives the response values and rhs the corresponding groups or explanatory variables
data	optional matrix or data frame (or similar; see model.frame) containing the variables in the formula. By default the variables are taken from <code>environment(formula)</code> .
subset	optional vector specifying a subset of observations to be used in the analysis
na.action	function indicating what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .

Details

Formula interfaces are evaluated using [model.frame](#), following standard R conventions. The left-hand side of the formula must contain the response variable. The right-hand side typically specifies a grouping or explanatory variable. Only formulas with a single response and at least one explanatory variable are supported.

See also:

- [formula](#)
- [model.frame](#)
- [terms](#)

freq

*Frequency Table for a Single Variable***Description**

Calculates absolute and relative frequencies of a vector `x`. Continuous (numeric) variables will be cut using the same logic as used by the function [hist](#). Categorical variables will be aggregated by [table](#). The result will contain single and cumulative frequencies for both, absolute values and percentages.

Usage

```
freq(
  x,
  breaks = hist(x, plot = FALSE)$breaks,
  include.lowest = TRUE,
  ord = c("level", "desc", "asc", "name"),
  useNA = c("no", "ifany", "always"),
  ...
)
```

```
)

## S3 method for class 'Freq'
print(x, digits = NULL, ...)
```

Arguments

<code>x</code>	variable to be described; may be any atomic type
<code>breaks</code>	either a numeric vector of two or more cut points or a single number (greater than or equal to 2) giving the number of intervals into which <code>x</code> is to be cut. Default taken from the function <code>hist()</code> . If set to <code>FALSE</code> , a numeric <code>x</code> will not be classed at all, but tabulated directly by its distinct values. This is useful for discrete numeric variables such as counts or scores. The argument is ignored if <code>x</code> is not of numeric type.
<code>include.lowest</code>	logical, indicating if an <code>x[i]</code> equal to the lowest (or highest, for <code>right = FALSE</code>) "breaks" value should be included. Ignored if <code>x</code> is not of numeric type or if <code>breaks = FALSE</code> .
<code>ord</code>	how should the result be ordered? Default is "level", other choices are 'by frequency' ("desc" or "asc") or 'by name of the levels' ("name"). The argument can be abbreviated. This is ignored if <code>x</code> is numeric and classed (<code>breaks</code> not <code>FALSE</code>).
<code>useNA</code>	one of "no", "ifany", or "always". Defines whether to include extra NA levels in the table. Defaults to "no" which is the <code>table()</code> default too.
<code>...</code>	further arguments are passed to the function <code>cut()</code> . Use <code>dig.lab</code> to control the format of numeric group names. Use the argument <code>right</code> to define if the intervals should be closed on the right (and open on the left) or vice versa. In <code>print.Freq</code> the dots are not used.
<code>digits</code>	number of digits used to display relative frequencies

Details

By default only the valid cases are considered for the frequencies, say NA values are excluded. (This is in accordance with the default behavior of the R function `table`, which seemed a reasonable reference.) If the NAs should be included you can set the `useNA` argument to either "ifany" or "always".

For numeric variables, if `breaks` is specified as a single number, the range of the data is divided into breaks pieces of equal length, and then the outer limits are moved away by 0.1\ extreme values both fall within the break intervals. (If `x` is a constant vector, equal-length intervals are created that cover the single value.) See `cut`.

With `breaks = FALSE` no classing takes place and the distinct values of a numeric `x` are tabulated directly (in ascending order of the values). In this case the `ord` argument applies as for categorical variables.

Value

an object of class "Freq" inheriting from `data.frame`, with columns:

```
level levels or intervals of the tabulated variable
freq absolute frequencies
perc relative frequencies
cumfreq cumulative absolute frequencies
cumperc cumulative relative frequencies
```

See Also

[cut](#), [hist](#), [cumsum](#), [table](#), [prop.table](#)

Other frequency: [expFreq\(\)](#), [freq2D\(\)](#), [percTable\(\)](#), [tOne\(\)](#)

Examples

```
data(Pizza)

# result is a data.frame
d.freq <- freq(Pizza$price)
d.freq

# it is printed by default with 3 digits for the percent values,
# but the number of digits can be defined in the print function
print(d.freq, digits=5)

# sorted by frequency
freq(Pizza$driver, ord="desc")

# sorted by name using all the observations, say including NAs
freq(Pizza$driver, ord="name", useNA="ifany")

# percentages and cumulative frequencies for a vector of count data
freq(as.table(c(2,4,12,8)))

# tabulate a discrete numeric variable directly, without classing
freq(Pizza$count, breaks = FALSE)

# ... which also allows ordering by frequency
freq(Pizza$count, breaks = FALSE, ord = "desc")
```

freq2D

Bivariate (Two-Dimensional) Frequency Distribution

Description

Calculate a frequency distribution for two continuous variables.

Usage

```
freq2D(x, ...)
```

S3 method for class 'formula'

```
freq2D(x, data, subset, na.action, n = 20, pad = 0, dnn = NULL, ...)
```

Default S3 method:

```
freq2D(x, y, n = 20, pad = 0, dnn = NULL, ...)
```

Arguments

<code>x</code>	a vector of x values, or a data frame whose first two columns contain the x and y values
<code>...</code>	named arguments passed to the default method
<code>data</code>	a <code>data.frame</code> , <code>matrix</code> , or <code>list</code> from which the variables in <code>formula</code> should be taken
<code>subset</code>	an optional vector specifying a subset of observations to be used
<code>na.action</code>	a function which indicates what should happen when the data contain missing values. Defaults to <code>getOption("na.action")</code> .
<code>n</code>	the desired number of bins for the output, a scalar or a vector of length 2
<code>pad</code>	number of rows and columns to add to each margin, containing only zeros
<code>dnn</code>	names for the dimensions in the result
<code>y</code>	a vector of y values
<code>formula</code>	a formula , such as <code>y ~ x</code>

Details

The exact number of bins is determined by the [pretty](#) function, based on the value of `n`.

Padding the margins with zeros can be helpful for subsequent analysis, such as smoothing.

Value

a frequency matrix whose rows represent the y bins in descending order and whose columns represent the x bins

Note

Based on code by Arni Magnusson, adapted to conform to package standards.

See Also

[cut](#), [table](#), and [print.table](#) are the basic underlying functions.

Other frequency: [expFreq\(\)](#), [freq\(\)](#), [percTable\(\)](#), [tOne\(\)](#)

Examples

```
freq2D(quakes$long, quakes$lat, dnn="")
freq2D(lat ~ long, quakes, n=c(10, 20), pad=1)
```

`generation`*Generation by Birth Year*

Description

Yields the generation of a person based on the year of birth.

Usage

```
generation(year)
```

Arguments

<code>year</code>	year of birth
-------------------	---------------

Details

The generations are defined as:

birth year	label
1946–1964	Babyboomer
1965–1979	Generation X
1980–1995	Generation Y - also known as Millennials
1996–2010	Generation Z
2011 and later	Generation Alpha

The last class is left open at the top. The table formerly gave it as 1946-2025 while the code used `Inf`, so a birth year of 2026 was documented as NA and returned as "Gen Alpha". Naming the successor generation is not settled enough to hard-code an upper bound.

Value

ordered factor with levels `c("Babyboomer", "Gen X", "Millennial", "Gen Z", "Gen Alpha")`
Values which fall outside the range of breaks are coded as NA, as are `NaN` and `NA` values.

See Also

[cutAge](#)

Other `date.time`: [addMonths\(\)](#), [countWorkDays\(\)](#), [date-time-predicates](#), [date_functions](#), [time-conversions](#), [zodiac\(\)](#)

Examples

```
generation(c(1946, 1964, 1972, 2001, 2003, 2018, 2026))
```

gini

*Gini Coefficient***Description**

Computes the Gini coefficient, a widely used measure of inequality, optionally with bootstrap confidence intervals.

Usage

```
gini(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  unbiased = TRUE,
  weights = NULL,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	numeric vector of non-negative values
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>unbiased</code>	logical; whether to apply the small-sample bias correction factor $n/(n-1)$, with n the effective sample size (the sum of the weights). For unweighted data this is the usual $1/(1 - \sum w_i^2)$.
<code>weights</code>	optional non-negative numeric vector with the same length as <code>x</code>
<code>na.rm</code>	logical; whether to remove missing values before computation
<code>...</code>	additional arguments passed to the bootstrap procedure: <code>type</code> confidence interval type (default "bca") <code>R</code> number of bootstrap replications (default 999) <code>parallel</code> parallelization mode ("no", "multicore", or "snow") <code>ncpus</code> number of CPUs

Details

The Gini coefficient ranges from 0 (perfect equality) to 1 (maximal inequality). For finite samples, the uncorrected estimator is biased; setting `unbiased = TRUE` applies a standard correction factor.

Weights are interpreted as frequency (replication) weights, meaning that each observation contributes proportionally to its weight in the empirical distribution.

Confidence intervals are obtained via bootstrap resampling using the **boot** package. The default interval type is bias-corrected and accelerated ("bca").

The implementation uses a numerically stable formulation based on the Lorenz curve:

$$G = \frac{1}{\mu} \sum_i w_i x_i (2F_i - 1)$$

where F_i are midpoints of cumulative weights and μ is the weighted mean.

sides names the side on which the finite bound lies: "left" yields an interval bounded below, "right" one bounded above. The coefficient is bounded, so the open side is reported at the range boundary (0 or 1) rather than as $\pm\infty$ (design_rules.md 4.1).

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of the Gini coefficient

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Random number generation

A confidence level triggers a bootstrap and therefore advances R's global random number generator. Call [set.seed](#) beforehand for reproducible intervals.

See Also

Other inequality: [atkinson\(\)](#), [divCoef\(\)](#), [lc\(\)](#), [rosenbluth\(\)](#), [theil\(\)](#)

Examples

```
x <- c(10, 20, 30, 40)
gini(x)

# weighted example
gini(c(10, 0), weights = c(2, 3))

# with confidence interval
gini(x, conf.level = 0.95, R = 499)
```

Description

Calculate Goodman-Kruskal's tau, a measure of association for nominal variables in a two-way table. The function accepts either a contingency table or two vectors.

Usage

```
gkTau(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  direction = c("row", "column"),
  ...
)
```

Arguments

<code>x</code>	numeric vector or contingency table. A matrix is treated as a table.
<code>y</code>	NULL (default) or a vector with compatible dimensions to <code>x</code> . If supplied, <code>table(x, y, ...)</code> is calculated.
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>direction</code>	direction of the calculation. Must be "row" (default) or "column". "row" gives tau (RIC), i.e. the row variable is the dependent one and is predicted from the column variable; "column" gives tau (CIR).
<code>...</code>	further arguments are passed to the function table , allowing i.e. to set <code>useNA</code> . This refers only to the vector interface.

Details

Goodman-Kruskal tau measures association for cross tabulations of nominal level variables. Goodman-Kruskal tau is based on random category assignment. It measures the percentage improvement in predictability of the dependent variable (column or row variable) given the value of other variables (row or column variables). Goodman-Kruskal tau is the same as Goodman-Kruskal lambda except the calculations of the tau statistic are based on assignment probabilities specified by marginal or conditional proportions. Misclassification probabilities are based on random category assignment with probabilities specified by marginal or conditional proportion.

Goodman Kruskal tau reduces to ϕ^2 (see: [phi](#)) in the 2x2-table case.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of Goodman-Kruskal's tau

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Note

Based on code by Antti Arppe, adapted to conform to package standards.

References

- Agresti, A. (2002) *Categorical Data Analysis*. John Wiley & Sons, pp. 57-59.
- Goodman, L. A., & Kruskal, W. H. (1954) Measures of association for cross classifications. *Journal of the American Statistical Association*, 49, 732-764.
- Somers, R. H. (1962) A New Asymmetric Measure of Association for Ordinal Variables, *American Sociological Review*, 27, 799-811.
- Goodman, L. A., & Kruskal, W. H. (1963) Measures of association for cross classifications III: Approximate sampling theory. *Journal of the American Statistical Association*, 58, 310-364.
- Liebetrau, A. M. (1983) *Measures of Association*, Sage University Papers Series on Quantitative Applications in the Social Sciences, 07-004. Newbury Park, CA: Sage, pp. 24-30

See Also

[lambda](#), [cramerV](#), [Association](#)

Other assoc.nominal: [contCoef\(\)](#), [cramerV\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
# example in:
# http://support.sas.com/documentation/cdl/en/statugfreq/63124/PDF/default/statugfreq.pdf
# pp. S. 1821

tab <- as.table(rbind(c(26,26,23,18,9),c(6,7,9,14,23)))

# Goodman Kruskal's tau C|R
gkTau(tab, direction="column", conf.level=0.95)
# Goodman Kruskal's tau R|C
gkTau(tab, direction="row", conf.level=0.95)

# http://support.sas.com/documentation/cdl/en/statugfreq/63124/PDF/default/statugfreq.pdf
# pp. 1814 (143)
tab <- as.table(cbind(c(11,2),c(4,6)))

gkTau(tab, direction="row", conf.level=0.95)
gkTau(tab, direction="column", conf.level=0.95)
# reduce both to:
phi(tab)^2

# example 1 in Liebetrau (1983)

tt <- matrix(c(549,93,233,119,225,455,402,
               212,124,78,42,41,12,132,
               54,54,33,13,46,7,153), ncol=3,
             dimnames=list(rownames=c("Gov", "Mil", "Edu", "Eco", "Intel", "Rel", "For"),
                           colnames=c("One", "Two", "Multi")))

gkTau(tt, direction = "row", conf.level = 0.95)
gkTau(tt, direction = "column", conf.level = 0.95)

# SPSS
ttt <- matrix(c(225,53,206,3,1,12), nrow=3,
```

```

dimnames=list(rownames=c("right","center", "left"),
               colnames=c("us","ussr")))

round(gkTau(ttt, direction = "r", con=0.95), d=3)
round(gkTau(ttt, direction = "c"), d=3)

```

glassDelta

Glass' Delta Effect Size

Description

Computes Glass' delta, a standardized mean difference that uses the standard deviation of the control group only, along with a noncentral-t based confidence interval and an optional small-sample bias correction.

Usage

```

glassDelta(
  x,
  y,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  useControlSd = TRUE,
  correct = FALSE,
  na.rm = FALSE
)

```

Arguments

<code>x</code>	numeric vector containing the treatment group
<code>y</code>	numeric vector containing the control group
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>useControlSd</code>	logical, if TRUE (default) the standard deviation of the control group y is used for standardization, otherwise the one of x
<code>correct</code>	logical, if TRUE the exact small-sample bias correction (Hedges' correction with $df = n_C - 1$) is applied. Requires at least 3 observations in the standardizing group. Default is FALSE.
<code>na.rm</code>	logical, should missing values be removed? Default is FALSE. If FALSE and any of the groups contains missing values, NA is returned.

Details

Glass' delta is defined as:

$$\Delta = \frac{\bar{x} - \bar{y}}{s_y}$$

where s_y is the standard deviation of the control group. It is preferred over Cohen's d when the treatment is expected to affect the variance, so that the control group's variability is the natural reference scale.

The confidence interval is obtained by inverting the noncentral t-distribution with $df = n_C - 1$ degrees of freedom, where n_C is the size of the group supplying the standard deviation (Kelley, 2007). Note that this interval assumes equal population variances in both groups. Since Glass' delta is typically chosen precisely when the variances are expected to differ, the interval should be regarded as approximate under heteroscedasticity.

With `correct = TRUE` the exact correction factor

$$J(df) = \frac{\Gamma(df/2)}{\sqrt{df/2} \Gamma((df-1)/2)}$$

is applied to the estimate and both confidence limits.

Value

a named numeric vector. If `conf.level = NA`, only `est` is returned; otherwise the vector has elements:

`est` point estimate of Glass' delta

`lci` lower confidence interval bound

`uci` upper confidence interval bound

In both cases the result carries the attribute "magnitude" with the conventional interpretation of the estimate's absolute size ("`negligible`" < 0.2 ≤ "`small`" < 0.5 ≤ "`medium`" < 0.8 ≤ "`large`"), analogous to `cohenD()`.

Note

The confidence interval method follows Ken Kelley's approach previously published in the **MBESS** package, reimplemented to conform to package standards.

References

Glass, G. V. (1976) Primary, secondary, and meta-analysis of research. *Educational Researcher*, 5(10), 3-8.

Hedges, L. V., Olkin, I. (1985) *Statistical Methods for Meta-Analysis*. Orlando: Academic Press.

Kelley, K. (2007) Confidence intervals for standardized effect sizes: Theory, application, and implementation. *Journal of Statistical Software*, 20(8), 1-24.

See Also

[cohenD](#)

Other effect.size: [cohenD\(\)](#), [cohenH\(\)](#), [etaSq\(\)](#), [oddsRatio\(\)](#), [relRisk\(\)](#)

Examples

```
set.seed(5)
x <- rnorm(30, mean = 1)
y <- rnorm(30, mean = 0)

glassDelta(x, y)
```

```

glassDelta(x, y, conf.level = 0.95)

# one-sided: "right" bounds the interval from ABOVE
glassDelta(x, y, conf.level = 0.95, sides = "right")

# ... and "left" from below
glassDelta(x, y, conf.level = 0.95, sides = "left")

# small-sample bias correction
glassDelta(x, y, conf.level = 0.95, correct = TRUE)

# standardize by the treatment group instead
glassDelta(x, y, useControlSd = FALSE)

```

gmean

*Geometric Mean and Standard Deviation***Description**

Calculates the geometric mean, its confidence interval and the geometric standard deviation of a vector `x`.

Usage

```

gmean(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  na.rm = FALSE,
  ...
)

gsd(x, na.rm = FALSE)

```

Arguments

<code>x</code>	a positive numeric vector. An object which is not a vector is coerced, if possible, by <code>as.vector()</code> .
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of <code>"two.sided"</code> (default), <code>"left"</code> or <code>"right"</code>). See ConfidenceIntervals .
<code>method</code>	a vector of character strings representing the type of intervals required. The value should be any subset of the values <code>"classic"</code> , <code>"boot"</code> . See boot.ci .
<code>na.rm</code>	logical, indicating whether <code>NA</code> values should be stripped before the computation proceeds. Defaults to <code>FALSE</code> .

... further arguments are passed to the `boot` function. Supported arguments are `type` ("norm", "basic", "stud", "perc", "bca"), `parallel` and the number of bootstrap replicates `R`. If not defined those will be set to their defaults, being "basic" for `type`, option "boot.parallel" (and if that is not set, "no") for `parallel` and 999 for `R`.

Details

The geometric mean is defined as:

$$\sqrt[n]{x_1 \cdot x_2 \cdot x_3 \dots \cdot x_n}$$

The geometric mean and geometric standard deviation are restricted to positive inputs (because otherwise the answer can have an imaginary component). Hence if any argument is negative, the result will be NA. If any argument is zero, then the geometric mean is zero.

For strict positive values the geometric mean is computed as `exp(meanCI(log(x)))`.

Considerations (Roefeldt 2018) "The calculation of the geometric mean requires that all values are non-zero and positive. So what should you do if you have data that do not meet this requirement? If you have values that equal zero, you have a few options:

- Adjust your scale so that you add 1 to every number in the data set, and then subtract 1 from the resulting geometric mean.
- Ignore zeros or missing data in your calculations.
- Convert zeros to a very small number (often called "below the detection limit") that is less than the next smallest number in the data set.

If you have negative numbers, you will need to convert those numbers to a positive value before calculating the geometric mean. You can then assign the resulting geometric mean a negative value. If your data set contains both positive and negative values, you will have to separate them and find the geometric means for each group, and you can then find the weighted average of their individual geometric means to find the total geometric mean for the full data set. If none of these options appeals to you, you are not alone! There is controversy among statisticians about what is the best method for dealing with these values. You may want to calculate several types of averages and decide what makes the most sense for you and the results you are trying to report."

Value

for `gsd()`, a numeric scalar (NA if fewer than two strictly positive values remain). For `gmean()`, a numeric scalar if `conf.level = NA`; otherwise a named numeric vector with elements:

`est` point estimate of the geometric mean

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Snedecor, G. W., Cochran, W. G. Cochran (1989) Statistical Methods, 8th ed. Ames, IA: Iowa State University Press

Roefeldt K. (2018) Better than Average: Calculating Geometric Means Using SAS, Henry M. Jackson Foundation for the Advancement of Military Medicine,
https://www.lexjansen.com/wuss/2018/56_Final_Paper_PDF.pdf

See Also

Other location: [hmean\(\)](#), [hodgesLehmann\(\)](#), [huberM\(\)](#), [meanX\(\)](#), [medianX\(\)](#), [modeX\(\)](#), [tukeyBiweight\(\)](#)

Examples

```
set.seed(1)
x <- runif(5)
gmean(x)

m <- matrix(runif(50), nrow = 10)
apply(m, 2, gmean)

sapply(as.data.frame(m), gmean)

# example in https://www.stata.com/manuals13/rameans.pdf
x <- c(5,4,-4,-5,0,0,NA,7)

# positives only
gmean(x[x>0], na.rm=TRUE, conf.level=0.95)

# shift by 5 so that everything is positive, then drop the zeros:
# naReplace() puts zeros IN (turning NA into 0), which makes the
# geometric mean collapse to 0 - the opposite of what is wanted here
z <- x + 5
gmean(z[!is.na(z) & z > 0], conf.level = 0.95)
```

herfindahl

Herfindahl Index

Description

Computes the Herfindahl (or Herfindahl-Hirschman) index as a measure of concentration or inequality.

Usage

```
herfindahl(x, n = rep(1, length(x)), parameter = 1, na.rm = FALSE)
```

Arguments

x	numeric vector of non-negative values, such as market shares, incomes, or frequencies
n	optional frequency weights. Each element of x is replicated n times.
parameter	parameter m controlling sensitivity to concentration; must be strictly positive, default is 1. $m = 0$ is rejected: it degenerates to a constant 1 for every input.
na.rm	logical; whether to remove missing values

Details

The index is defined as the power mean of order $m + 1$ of the relative shares. For parameter = 1, the classical Herfindahl-Hirschman Index (HHI) is obtained.

Larger values indicate higher concentration. If negative values or missing values (when `na.rm = FALSE`) are present, NA is returned.

Value

numeric scalar containing the Herfindahl index

Note

Based on code by Achim Zeileis, rewritten to conform to package standards.

References

- Cowell, F. A. (2000) Measurement of Inequality, in Atkinson, A. B., Bourguignon, F. *Handbook of Income Distribution*. (Eds) Amsterdam
- Cowell, F. A. (1995) *Measuring Inequality*. Prentice Hall/Harvester Wheatsheaf
- Hall, M., Tidemann, N. (1967) *Measures of Concentration*, JASA 62, 162-168.
- Hirschman, A. O. (1964). The paternity of an index.

See Also

[gini](#), [atkinson](#)

Other diversity.concentration: [entropy\(\)](#), [simpson\(\)](#)

Examples

```
# generate vector (of sales)
x <- c(541, 1463, 2445, 3438, 4437, 5401, 6392, 8304, 11904, 22261)

# compute Herfindahl coefficient with parameter 1
herfindahl(x)

# Some more examples
herfindahl(c(261,29,33,15,39,28,95,5,6,28,69,8,105,38,15))
herfindahl(c(783,121,112,70,201,153,425,19,37,126,325,51,442,193,41))
```

hmean

Harmonic Mean and Its Confidence Interval

Description

Calculates the harmonic mean and its confidence interval of a vector x.

Usage

```
hmean(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("classic", "boot"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a positive numeric vector. An object which is not a vector is coerced, if possible, by <code>as.vector()</code> .
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of <code>"two.sided"</code> (default), <code>"left"</code> or <code>"right"</code>). See ConfidenceIntervals .
<code>method</code>	a vector of character strings representing the type of intervals required. The value should be any subset of the values <code>"classic"</code> , <code>"boot"</code> . See boot.ci .
<code>na.rm</code>	logical, indicating whether <code>NA</code> values should be stripped before the computation proceeds. Defaults to <code>FALSE</code> .
<code>...</code>	further arguments are passed to the boot function. Supported arguments are <code>type</code> (<code>"norm"</code> , <code>"basic"</code> , <code>"stud"</code> , <code>"perc"</code> , <code>"bca"</code>), <code>parallel</code> and the number of bootstrap replicates <code>R</code> . If not defined those will be set to their defaults, being <code>"basic"</code> for <code>type</code> , option <code>"boot.parallel"</code> (and if that is not set, <code>"no"</code>) for <code>parallel</code> and 999 for <code>R</code> .

Details

To compute the harmonic mean, $1/x$ is first calculated, before the arithmetic mean and its confidence interval are computed by [meanCI](#). The harmonic mean is then the reciprocal of the arithmetic mean of the reciprocals of the values. The same applies to the confidence interval.

The harmonic mean is restricted to strictly positive inputs. Non-positive values are turned into `NA` and therefore make the result `NA` unless `na.rm = TRUE`, in which case they are dropped. If the lower bound of the confidence interval is not greater than zero, then the confidence interval is not defined, and thus `NA` will be reported.

`sides` names the side on which the finite bound lies: `"left"` yields an interval bounded below, `"right"` one bounded above. The harmonic mean of positive values is itself positive, so the open lower side is reported as 0 rather than as `NA` or $-\infty$.

Use [sapply](#) to calculate the measures from data frame, resp. from a matrix.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

```
est  point estimate of the harmonic mean
lci  lower confidence interval bound
uci  upper confidence interval bound
```

References

Snedecor, G. W., Cochran, W. G. (1989) Statistical Methods, 8th ed. Ames, IA: *Iowa State University Press*

See Also

Other location: [gmean\(\)](#), [hodesLehmann\(\)](#), [huberM\(\)](#), [meanX\(\)](#), [medianX\(\)](#), [modeX\(\)](#), [tukeyBiweight\(\)](#)

Examples

```
x <- runif(5)
hmean(x)

m <- matrix(runif(50), nrow = 10)
apply(m, 2, hmean)

sapply(as.data.frame(m), hmean)
```

hodesLehmann

Hodges-Lehmann Estimator of Location

Description

Function to compute the Hodges-Lehmann estimator of location in the one and two sample case following a clever fast algorithm by John Monahan (1984).

Usage

```
hodesLehmann(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	numeric vector
<code>y</code>	optional numeric vector
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>na.rm</code>	logical; whether to remove missing values
<code>...</code>	additional arguments passed to bootstrap procedures

Details

The Hodges-Lehmann estimator is the median of the combined data points and Walsh averages.

It is the same as the pseudo median returned as a by-product of `wilcox.test` (which however does not calculate correctly as soon as ties are present).

Note that in the two-sample case the estimator for the difference in location parameters does not estimate the difference in medians (a common misconception) but rather the median of the difference between a sample from x and a sample from y .

`sides` names the side on which the finite bound lies: "left" yields $[lci, \infty)$, "right" yields $(-\infty, uci]$. The estimator is unbounded, so the open side is reported as $\pm\infty$.

`x` and `y` are not modified.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of the Hodges-Lehmann location

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Random number generation

A confidence level triggers a bootstrap and therefore advances R's global random number generator. Call `set.seed` beforehand for reproducible intervals. The point estimate itself is deterministic: the compiled routine picks its pivots from a local generator and does not touch R's stream.

Note

C++ port of Monahan's algorithm by Cyril Flurin Moser

See Also

`wilcox.test`

Other location: `gmean()`, `hmean()`, `huberM()`, `meanX()`, `medianX()`, `modeX()`, `tukeyBiweight()`

Examples

```
x <- c(1.83, 0.50, 1.62, 2.48, 1.68, 1.88, 1.55, 3.06, 1.30)
hodgesLehmann(x)

# the input is left alone
v <- c(3, 1, 2)
hodgesLehmann(v)
v

# two-sample: median of the pairwise differences, NOT the difference
# of the medians
y <- c(0.878, 0.647, 0.598, 2.05, 1.06, 1.29, 1.06, 3.14, 1.29)
hodgesLehmann(x, y)

set.seed(1)
hodgesLehmann(x, conf.level = 0.95)
```

hoeffdingD	<i>Hoeffding's D Statistic</i>
------------	--------------------------------

Description

Computes Hoeffding's D statistic for testing independence between two variables. Two algorithms are available: an $O(n \log n)$ one based on rank statistics and Fenwick trees (Even-Zohar & Leng, 2020), and the classical $O(n^2)$ one, which is slower but handles ties exactly.

Usage

```
hoeffdingD(
  x,
  y,
  engine = c("fast", "exact"),
  R = 999,
  jitter = FALSE,
  eps = NULL,
  seed = NULL,
  output = c("def", "test")
)
```

Arguments

x	numeric vector
y	numeric vector with the same length as x
engine	character string selecting the algorithm: "fast" (default) or "exact". Both compute the same quantity; on data without ties they agree to floating point accuracy. With ties they differ - see Details.
R	number of permutations used for the test of independence. Only used when output = "test"; defaults to 999.
jitter	logical. If TRUE, small random noise is added to both x and y to break ties. This is useful when the data contain ties, since the fast algorithm assumes continuous data.
eps	optional numeric half-width of the uniform jitter noise, which is drawn from $U(-eps, eps)$. Defaults to $1e-10$ times the standard deviation of the affected variable. A variable with zero variance cannot be jittered and raises an error.
seed	optional integer random seed for reproducibility when jitter = TRUE. The state of R's random number generator is restored afterwards, so passing a seed does not disturb the calling session.
output	output format, either "def" (default), which returns the statistic, or "test", which returns an object of class "htest" with a permutation P value for the hypothesis of independence.

Details

Choice of engine: engine = "fast" runs in $O(n \log n)$ and is what makes six-figure sample sizes practical. It needs a strict ordering and therefore assumes no ties; with tied values it is biased, and the size of the bias grows with the number of ties.

engine = "exact" is the classical formulation of Hollander and Wolfe. It resolves ties through midranks and the fractional counts Q_i , so it is exact for tied data - the same quantity `Hmisc::hoeffd()` reports. It costs $O(n^2)$ time, which is a few milliseconds at $n = 1000$ and roughly a second at $n = 10000$.

On data without ties the two agree to floating point accuracy, so the choice only matters when ties are present. Then there are three options, in descending order of preference: engine = "exact" answers the question exactly, jitter = TRUE answers it approximately but quickly, and doing neither answers a slightly different question. Only the last one is a mistake.

Ties and jittering: jitter = TRUE breaks ties by adding small random noise, which makes the fast engine applicable at the cost of a small random perturbation of the result. It applies to the fast engine only; with engine = "exact" it is unnecessary and is refused rather than silently ignored.

Test of independence: output = "test" returns an "htest" object carrying a permutation P value for the null hypothesis that x and y are independent. Under that hypothesis every pairing of the two samples is equally likely, so the null distribution is obtained by recomputing the statistic on R random pairings. The P value is $(1 + \#\{D^* \geq D\})/(R + 1)$, which is never exactly zero - with R permutations no evidence stronger than $1/(R + 1)$ has been gathered, and reporting 0 would claim otherwise.

The permutation route was chosen over the tabulated asymptotic distribution on purpose: it needs neither a table nor a scaling constant, it is exact by construction, and it costs R evaluations of a statistic that is fast - which is the whole point of the fast engine. With engine = "exact" the permutations are applied to y itself, so the tie structure of both variables is preserved and the null distribution belongs to the same tied data.

Why there is no confidence interval: D is a U-statistic whose kernel is degenerate exactly under independence and non-degenerate otherwise. Away from independence $\sqrt{n}(\hat{D} - D)$ is asymptotically normal and an interval is in principle available. At $D = 0$ the limit is not a normal distribution but a weighted sum of chi-squares (Blum, Kiefer and Rosenblatt, 1961), and the variance of the first-order projection vanishes - so an interval built on the non-degenerate asymptotics collapses precisely where most data sit.

The obvious escape does not work either. For a degenerate U-statistic the ordinary bootstrap is not consistent; a valid resampling scheme has to be built on the second-order term of the Hoeffding decomposition (Arcones and Gine, 1992). A percentile or BCa interval from the usual machinery would therefore be serviceable for strongly dependent data and wrong exactly at independence, which is the one place a reader would look.

What is well founded here is the test, and that is what output = "test" provides. An interval over the first-order projection could be constructed for data that are clearly far from independence, but it is deliberately not offered: its coverage cannot be relied on in the situation the statistic is most often used for.

Missing values are an error rather than a silent approximation: `order` and `rank` place NAs at the end by default, which yields a formally valid permutation and hence a number that looks like an answer. Remove or impute them before calling.

Value

numeric scalar containing Hoeffding's D statistic on the conventional scale: 1 under perfect monotone or antitone dependence, 0 under independence, with a lower bound that approaches -0.5 for large n (for $n = 7$ the minimum over all permutations is -0.26).

Note that the raw statistic of Hollander and Wolfe lies in $[-1/60, 1/30]$; this function returns 30 times that value, which is the scale `Hmisc::hoeffd()` reports. The two differ by exactly the factor 30, so results are comparable only after accounting for it.

Random number generation

`jitter = TRUE` draws from R's random number generator and therefore advances it, unless `seed` is supplied - in which case the previous state is restored.

References

- Even-Zohar, C. and Leng, C. (2020). Fast computation of Hoeffding's D statistic.
- Hollander, M., Wolfe, D. A. and Chicken, E. (2013). Nonparametric Statistical Methods (3rd ed.).
- Blum, J. R., Kiefer, J. and Rosenblatt, M. (1961). Distribution free tests of independence based on the sample distribution function. *Annals of Mathematical Statistics* **32**, 485-498.
- Arcones, M. A. and Gine, E. (1992). On the bootstrap of U and V statistics. *Annals of Statistics* **20**, 655-674.

See Also

[spearmanCor](#), [kendallTauB](#)

Other `assoc.continuous`: [corPart\(\)](#), [corPolychor\(\)](#), [findCorrX\(\)](#), [keepSig\(\)](#), [pearsonCor\(\)](#), [spearmanCor\(\)](#)

Examples

```
set.seed(1)
x <- rnorm(200)
y <- x^2 + rnorm(200)

# fast computation
hoeffdingD(x, y)

# with ties
y2 <- round(y, 1)
hoeffdingD(x, y2)           # warning: the fast engine is biased
hoeffdingD(x, y2, engine = "exact") # exact, and the recommended answer
hoeffdingD(x, y2, jitter = TRUE)    # fast approximation

# without ties the two engines agree
all.equal(hoeffdingD(x, y),
          hoeffdingD(x, y, engine = "exact"))

# perfect monotone dependence is 1 on this scale, antitone likewise
hoeffdingD(1:50, (1:50)^3)
hoeffdingD(1:50, 50:1)

# test of independence - the quadratic relation is invisible to
# correlation but not to D
set.seed(3)
hoeffdingD(x, y, output = "test")
cor.test(x, y)$p.value
```

huberM

*Safe (Generalized) Huber M-Estimator of Location***Description**

A (generalized) Huber M-estimator of location with MAD scale that handles the degenerate case of zero scale gracefully, where `huber()` would return an error.

Usage

```
huberM(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("wald", "boot"),
  k = 1.345,
  mu = NULL,
  s = NULL,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	numeric vector of data values
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>method</code>	confidence interval method: "wald" (default) or "boot".
<code>k</code>	positive tuning constant; the algorithm winsorizes at k standard deviations. Default is 1.345.
<code>mu</code>	initial location estimate. NULL (default) uses <code>median(x)</code> , computed after <code>na.rm</code> is applied.
<code>s</code>	scale estimate held constant through the iterations. NULL (default) uses <code>mad(x, center = mu)</code> , computed after <code>na.rm</code> is applied.
<code>na.rm</code>	logical; whether to remove missing values before computation; default is FALSE
<code>...</code>	further arguments passed to the bootstrap engine when <code>method = "boot"</code> : <code>R</code> , <code>type</code> , <code>parallel</code> , and <code>ncpus</code> ; see Details

Details**Wald interval** (`method = "wald"`)

Uses the τ correction factor of Ruckstuhl following

$$\hat{\mu} \pm t_{\alpha/2, n-1} \sqrt{\hat{\tau}(x, \hat{\mu})} \frac{\hat{s}}{\sqrt{n}}$$

No finite-sample correction is applied beyond the t-quantile.

Bootstrap interval (method = "boot")

The statistic $\hat{\mu}$ is resampled R times. Note that μ and s are fixed at their initial values (computed from the full data before resampling) and are not re-estimated on each resample. The bootstrap therefore targets the variability of the location estimator with fixed scale initialization, not a fully re-estimated robust estimator. Bootstrap arguments are passed through `...` and extracted via `.extractBootArgs()`:

`R` number of bootstrap replicates (default 999)

`type` confidence interval type: "perc" or "bca" (default)

`parallel` parallelization mode: "no", "multicore", or "snow" (default "no")

`ncpus` number of CPUs for parallel bootstrap (default `getOption("boot.ncpus", 1L)`)

The original internal estimator is accessible as `DescToolsX:::huberM`.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` location estimate from Huber's M-estimator

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Random number generation

`method = "boot"` resamples and therefore advances R's global random number generator. Call `set.seed` beforehand for reproducible intervals.

Note

Adapted from code by Martin Maechler to conform to package standards

References

Huber, P. J. (1981). *Robust Statistics*. Wiley.

See Also

`huber`, `mad`, `tukeyBiweight`

Other location: `gmean()`, `hmean()`, `hodgesLehmann()`, `meanX()`, `medianX()`, `modeX()`, `tukeyBiweight()`

Examples

```
huberM(c(1:9, 1000))
mad(c(1:9, 1000))
```

```
set.seed(7)
x <- c(round(rnorm(1000), 1), round(rnorm(50, mean = 10, sd = 10)))
```

```
huberM(x, conf.level = 0.95)
huberM(x, conf.level = 0.95, method = "boot", R = 499, type = "bca")
```

```
# degenerate case: scale zero
huberM(rep(9, 100))
```

icc

*Intraclass Correlation Coefficient (ICC)***Description**

Computes intraclass correlation coefficients (ICC) according to Shrout and Fleiss (1979) and McGraw and Wong (1996).

Usage

```
icc(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("anova", "reml", "boot"),
  model = c("twoway", "oneway"),
  type = c("agreement", "consistency"),
  unit = c("single", "average"),
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	numeric matrix or data frame with subjects in rows and raters in columns
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>method</code>	character string specifying the estimation and confidence interval method; defaults to "anova"
<code>model</code>	character string, either "oneway" or "twoway"
<code>type</code>	character string, either "agreement" or "consistency"
<code>unit</code>	character string, either "single" or "average"
<code>na.rm</code>	logical; if TRUE, complete cases are used
<code>...</code>	additional arguments. For <code>method = "boot"</code> , the number of bootstrap resamples can be specified via <code>R</code> .

Details

The ICC is a measure of reliability for ratings of n subjects by k raters. The specific coefficient depends on three design decisions:

- **model:** one-way or two-way ANOVA design
- **type:** agreement or consistency
- **unit:** single rating or average of k ratings

The six classical Shrout–Fleiss cases are:

model	type	unit
oneway	agreement	single (ICC1)
oneway	agreement	average (ICC1k)
twoway	agreement	single (ICC2)
twoway	agreement	average (ICC2k)
twoway	consistency	single (ICC3)
twoway	consistency	average (ICC3k)

For model = "oneway" only type = "agreement" is meaningful.

Confidence intervals can be computed using different inference methods:

- "anova": exact F-based intervals following Shrout and Fleiss (1979)
- "reml": variance components estimated via REML. Point estimate only; no confidence interval is available for this method.
- "boot": nonparametric percentile bootstrap

ICC(1) is based on a one-way random effects ANOVA and measures absolute agreement. ICC(2) assumes raters are randomly sampled and generalizable, while ICC(3) assumes a fixed set of raters.

The average forms (k) reflect the reliability of the mean of k raters and correspond to the Spearman–Brown adjusted reliability.

The ANOVA-based confidence intervals follow the exact formulas of Shrout and Fleiss (1979), including the variance approximation for ICC(2).

Value

if `conf.level` = NA, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of the intraclass correlation

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Random number generation

`method` = "boot" resamples subjects and therefore advances R's global random number generator. Call [set.seed](#) beforehand for reproducible intervals.

References

Shrout, P. E., Fleiss, J. L. (1979). Intraclass correlations: uses in assessing rater reliability. *Psychological Bulletin*, 86, 420–428.

McGraw, K. O., Wong, S. P. (1996). Forming inferences about some intraclass correlation coefficients. *Psychological Methods*, 1, 30–46.

See Also

Other `assoc.agreement`: [ccc\(\)](#), [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
#example from Shrout and Fleiss (1979)
sf <- matrix(c( 9, 2, 5, 8,    6, 1, 3, 2,    8, 4, 6, 8,
               7, 1, 2, 6,   10, 5, 6, 9,    6, 2, 4, 7),
            ncol=4, byrow=TRUE,
            dimnames=list(c("S1","S2","S3","S4","S5","S6"),
                          c("J1","J2","J3","J4"))) )

icc(sf)

# get all versions
args <- formals(icc)[c("model","type","unit")]
grid <- expand.grid(lapply(args, eval),
                   stringsAsFactors = FALSE)[-c(4,8),]

out <- apply(grid, 1, function(row)
  icc(sf,
    model = row["model"],
    type  = row["type"],
    unit  = row["unit"],
    method = "anova",
    conf.level = 0.95) )

t(simplify2array(out))
```

impute	<i>Impute Missing Values in a Vector</i>
--------	--

Description

Replaces missing values (NA) in a vector by either a summary statistic computed from the data (e.g. mean, median) or a user-supplied scalar value.

Usage

```
impute(x, FUN = median, na.rm = TRUE, ...)
```

Arguments

x	atomic vector
FUN	function used to compute the imputation value, or a scalar value; default is median
na.rm	logical; whether to remove missing values before computing the summary statistic
...	additional arguments passed to FUN

Details

If FUN is a function, it is applied to x to compute a single imputation value. If the function accepts an argument named `na.rm`, it will be passed automatically.

Alternatively, FUN can be a single scalar value, which will directly replace all missing values.

If FUN is a function, it must return a single value. An error is thrown if the returned value is not scalar.

Whether FUN accepts `na.rm` is decided from its formal arguments; a function taking `...` is assumed to pass it on. Functions without either are called without it.

Value

vector of the same length as x, with missing values replaced. The type follows R's usual coercion rules: imputing a double (the median of an integer vector, say) into an integer vector returns a double.

See Also

For direct value replacement see [bedrock::naReplace\(\)](#).

Other impute: [imputeKnn\(\)](#)

Examples

```
x <- c(2, 3, NA, 5, 9)

# Default: median(x, na.rm=TRUE)
impute(x)

# Using mean(x, na.rm=TRUE)
impute(x, mean)

# Using trimmed mean
impute(x, function(x) meanX(x, trim=0.3, na.rm = TRUE))

# Constant replacement
impute(x, 99)

# an integer vector stays integer only if the value is one too
impute(c(2L, 3L, NA, 5L), FUN = 4L)
```

imputeKnn

K-Nearest Neighbors Imputation

Description

Imputes missing values from the k nearest complete observations. Numeric and factor variables are both supported.

Usage

```
imputeKnn(
  x,
  k = 10,
  scale = TRUE,
  method = c("weighted", "median"),
  distData = NULL
)
```

Arguments

<code>x</code>	a data frame with missing values.
<code>k</code>	positive integer, the number of nearest neighbours to draw on.
<code>scale</code>	logical; if TRUE (default), numeric variables are standardized before distances are computed. This affects the distances only, not the imputed values.
<code>method</code>	the aggregation applied to the neighbours' values, either "weighted" for a distance-weighted mean (numeric) or weighted mode (factor), or "median" for a median (numeric) or mode (factor).
<code>distData</code>	optional data frame supplying the pool of potential neighbours. If given, neighbours are drawn from <code>distData</code> only, while <code>x</code> alone is imputed. It must have the same variables as <code>x</code> .

Details

Distance is Gower-style: numeric variables contribute their squared differences and factor variables a 0/1 mismatch penalty, summed and square-rooted. Numeric variables are standardized beforehand when `scale = TRUE`, so that no variable dominates through its unit alone. The standardization affects distances only; imputed values are always taken from the original data.

Continuous and categorical contributions are combined without further weighting, so a factor mismatch counts as much as a one-standard-deviation numeric gap. Whether that balance suits the data is for the caller to judge.

Each incomplete observation is compared with the complete ones on the variables it does observe, so rows missing different variables use different distances. Observations are therefore grouped by their pattern of missingness and each pattern is solved in one pass.

With `method = "weighted"` the neighbours are weighted by $\exp(-d)$. Because that decays on the scale of the distances themselves, it is meaningful only when those are of moderate size, which is another reason to keep `scale = TRUE`. Weights are normalized against their maximum before exponentiation, so that widely separated neighbours cannot all underflow to zero.

When **dbscan** is installed it is used to find the neighbours via a kd-tree, which is markedly faster on large data. The result is the same either way, apart from the resolution of exact ties.

Value

a data frame of the same shape as `x`, with missing values replaced.

See Also

Other impute: [impute\(\)](#)

Examples

```

set.seed(123)
dat <- data.frame(
  x = c(1, 2, 3, 4, 5, 6),
  y = c(1, 2, 3, 4, 5, 6),
  z = factor(c("a", "b", "a", "b", "a", "b"))
)

dat[c(1, 3), "x"] <- NA
dat[c(2, 5), "y"] <- NA

imputeKnn(dat, k = 2)

# neighbours drawn from a separate reference set
ref <- data.frame(
  x = c(1.5, 2.5, 3.5, 4.5),
  y = c(1.5, 2.5, 3.5, 4.5),
  z = factor(c("a", "b", "a", "b"), levels = c("a", "b"))
)

imputeKnn(dat, k = 2, distData = ref)

```

iqrX

*The (weighted) Interquartile Range***Description**

computes interquartile range of the x values. Weights are supported.

Usage

```
iqrX(x, weights = NULL, na.rm = FALSE, type = NULL)
```

Arguments

x	numeric vector
weights	optional numeric vector giving the sample weights
na.rm	logical; whether to remove missing values
type	an integer selecting one of the many quantile algorithms, see quantileX()

Details

This implementation is based on [quantileX\(\)](#) function, which allows to define weights.

Value

numeric scalar containing the interquartile range

See Also

[medianX](#), [quantileX](#), [IQR](#), [quantile](#)

Other dispersion: [coefVar\(\)](#), [madX\(\)](#), [meanAD\(\)](#), [meanSE\(\)](#), [rangeX\(\)](#), [varX\(\)](#)

Examples

```
x <- c(3.7, 3.3, 3.5, 2.8)
w <- c(5, 5, 4, 1)/15

iqrX(x=x, weights=w)
```

isConfusionTable	<i>Detect Whether an Object Looks Like a Confusion/Coincidence Matrix</i>
------------------	---

Description

Checks if `x` behaves like a rater-by-rater contingency table: square 2D numeric (integer-like) counts (or, optionally, proportions), non-negative, finite, and (optionally) with matching row/column names.

Usage

```
isConfusionTable(
  x,
  requireDimnames = TRUE,
  requireSameLevels = TRUE,
  integerTol = sqrt(.Machine$double.eps),
  acceptProportions = TRUE,
  requireSquare = TRUE
)
```

Arguments

<code>x</code>	object to check, typically a table, matrix, or numeric data.frame
<code>requireDimnames</code>	logical; if TRUE, both row and column names must be present. Defaults to TRUE.
<code>requireSameLevels</code>	logical; if TRUE and dimnames are present, row and column names must be the same set (order ignored). Defaults to TRUE.
<code>integerTol</code>	numeric tolerance for integer-like counts; defaults to <code>sqrt(.Machine\$double.eps)</code>
<code>acceptProportions</code>	logical; if TRUE, proportion tables are accepted when all entries are in $[0, 1]$ and their sum is approximately 1. Defaults to TRUE.
<code>requireSquare</code>	logical; whether to require a square table; defaults to TRUE

Value

TRUE if `x` looks like a confusion or coincidence matrix, otherwise FALSE

Examples

```

tab <- table(sample(letters[1:3], 100, TRUE),
             sample(letters[1:3], 100, TRUE))
isConfusionTable(tab)           # TRUE

M <- as.matrix(tab)
isConfusionTable(M)             # TRUE (dimnames present)
isConfusionTable(unname(M), requireDimnames = FALSE) # TRUE without names

df <- as.data.frame.matrix(tab)
isConfusionTable(df)           # TRUE (numeric data.frame)

# Two-column raw ratings are NOT a confusion table:
ratings <- cbind(r1 = sample(0:1, 50, TRUE), r2 = sample(0:1, 50, TRUE))
isConfusionTable(ratings)      # FALSE (not square)

```

kappaM	<i>Kappa for m Raters</i>
--------	---------------------------

Description

Computes agreement among $m \geq 2$ raters on categorical items, using Fleiss' kappa (the default), Conger's exact kappa, or Light's kappa.

Usage

```

kappaM(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  estimator = c("fleiss", "conger", "light"),
  use = c("complete.obs", "everything", "pairwise.complete.obs")
)

```

Arguments

x	a $n \times m$ matrix or data frame, n subjects in rows and m raters in columns
conf.level	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
estimator	a character string specifying the coefficient to compute. One of "fleiss" (default), "conger", or "light". These are three different coefficients for the same quantity, not three interval methods - hence estimator and not method.
use	a character string giving the treatment of missing values. One of "complete.obs" (default), which drops any subject rated incompletely; "everything", which returns NA if any value is missing; or "pairwise.complete.obs", which uses all subjects rated by both members of each rater pair. The last is available for estimator = "light" only, since Fleiss' and Conger's coefficients require a complete row per subject.

Details

All three coefficients contrast the mean observed agreement with the agreement expected by chance, and differ in how that chance agreement is derived.

"fleiss" Fleiss (1971) bases chance agreement on the category proportions pooled across all raters. It generalises Scott's pi (Scott, 1955), and for $m = 2$ it equals Scott's pi - not Cohen's kappa.

"conger" Conger (1980) derives chance agreement from each rater's own marginal distribution, and reduces exactly to Cohen's kappa when $m = 2$.

"light" Light (1971) is the unweighted mean of all $\binom{m}{2}$ pairwise Cohen kappas.

Confidence intervals are of Wald type. For "fleiss" and "conger" the standard error is the analytic one given by the respective author. For "light" no closed-form variance exists; since the estimate is the mean of the pairwise kappas, its standard error is estimated as their standard deviation divided by $\sqrt{\binom{m}{2}}$. This treats the pairwise kappas as independent, which they are not - they share raters - so the interval is approximate and tends to be too narrow. A bootstrap interval is preferable when the assumption is doubtful.

Kappa lies in $[-1, 1]$, so the interval is restricted to that range and the open side of a one-sided interval is reported at the boundary rather than at $\pm\infty$. See [ConfidenceIntervals](#).

Value

a named numeric vector. If `conf.level = NA`, only `est` is returned; otherwise the vector has elements:

`est` point estimate of the selected kappa coefficient
`lci` lower confidence interval bound
`uci` upper confidence interval bound

Note

Based on code by Matthias Gamer previously published as `kappaM.fleiss()` in the **irr** package, rewritten to conform to package standards.

References

- Conger, A. J. (1980). Integration and generalisation of Kappas for multiple raters. *Psychological Bulletin*, 88(2), 322-328.
- Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 378-382.
- Fleiss, J. L., Levin, B., & Paik, M. C. (2003). *Statistical Methods for Rates and Proportions* (3rd ed.). New York: John Wiley & Sons.
- Light, R. J. (1971). Measures of response agreement for qualitative data: Some generalizations and alternatives. *Psychological Bulletin*, 76(5), 365-377.
- Scott, W. A. (1955). Reliability of content analysis: The case of nominal scale coding. *Public Opinion Quarterly*, 19(3), 321-325.

See Also

Other assoc.agreement: [ccc\(\)](#), [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
statement <- data.frame(
  A = c(2,3,1,3,1,2,1,2,3,3,3,3,2,1,3,3,2,2,1,
        2,1,3,3,2,2,1,2,1,1,2,3,3,3,3,1,2,1,1),
  B = c(2,2,2,1,1,2,1,2,3,3,2,3,1,3,1,1,3,2,1,2,
        2,1,3,2,2,2,3,2,1,1,2,2,3,3,3,2,2,2,3),
  C = c(2,2,2,1,1,2,1,2,3,3,2,3,3,3,3,2,2,2,2,3,
        2,2,3,3,2,2,3,2,2,2,2,3,3,3,3,3,2,2,2),
  D = c(2,2,2,1,1,2,1,2,3,3,2,3,3,3,3,3,2,2,2,2,
        3,1,3,2,2,2,1,2,2,1,2,3,3,3,3,3,2,2,1),
  E = c(2,2,2,3,3,2,3,1,3,3,2,3,3,3,3,3,2,2,2,3,
        2,3,3,2,2,2,3,2,1,3,2,3,3,1,3,3,3,2,2,1)
)

kappaM(statement)

# Conger's exact kappa
kappaM(statement, estimator = "conger")

# Light's kappa, the mean of the pairwise Cohen kappas
kappaM(statement, estimator = "light")

# Fleiss' kappa with a confidence interval
kappaM(statement, conf.level = 0.95)
```

keepSig

Keep Only Significant Values in a Symmetric Matrix

Description

Replaces entries in a symmetric matrix (typically a correlation matrix) with NA wherever the corresponding p-value exceeds a significance threshold - retaining only the statistically supported associations. Designed as a pre-processing step for [plotWeb](#) and [plotCor](#).

Usage

```
keepSig(
  m,
  p = NULL,
  data = NULL,
  sig.level = 0.05,
  method = c("pearson", "spearman", "kendall"),
  diag = TRUE
)
```

Arguments

- m** a symmetric numeric matrix. Typically the output of [cor](#), but any symmetric matrix of effect sizes or association measures is accepted.
- p** a matrix of p-values of the same dimension as m. If NULL (default), p-values are computed from pairwise correlation tests on the columns of data (requires data).

data	an optional numeric data frame or matrix. Used to compute p when p = NULL. Ignored if p is supplied.
sig.level	numeric; significance threshold. Entries where $p > \text{sig.level}$ are replaced by NA. Default 0.05.
method	character; the correlation test method passed to <code>cor.test</code> when computing p-values from data. One of "pearson" (default), "spearman", or "kendall".
diag	logical; if TRUE (default), the diagonal is kept as-is (typically 1 for correlation matrices). If FALSE, the diagonal is also set to NA.

Value

matrix with the same dimensions and dimnames as m, with NA wherever $p > \text{sig.level}$

See Also

[pharos::plotWeb](#), [pharos::plotCor](#), [lumen::corTest](#), [stats::cor.test](#)

Other assoc.continuous: [corPart\(\)](#), [corPolychor\(\)](#), [findCorrX\(\)](#), [hoeffdingD\(\)](#), [pearsonCor\(\)](#), [spearmanCor\(\)](#)

Examples

```
# compute p-values on the fly from the raw data
keepSig(cor(mtcars), data = mtcars)

# stricter threshold, and drop the diagonal as well
keepSig(cor(swiss), data = swiss, sig.level = 0.01, diag = FALSE)

# the intended use is as a pre-processing step for the plots
pharos::plotWeb(keepSig(cor(mtcars), data = mtcars))

# supply a pre-computed p-value matrix
m <- cor(mtcars)
p <- outer(
  (vars <- colnames(mtcars)), vars,
  Vectorize(function(v1, v2)
    cor.test(mtcars[[v1]], mtcars[[v2]])$p.value)
)
dimnames(p) <- list(vars, vars)
plotWeb(keepSig(m, p = p))
```

kendallW

Kendall's Coefficient of Concordance W

Description

Computes Kendall's coefficient of concordance, a popular measure of association. It is an index of interrater reliability of ordinal data. The coefficient could be corrected for ties within raters.

Usage

```
kendallW(x, correct = FALSE, test = FALSE, na.rm = NULL)
```

Arguments

<code>x</code>	$n \times m$ matrix or dataframe, k subjects (in rows) m raters (in columns)
<code>correct</code>	a logical indicating whether the coefficient should be corrected for ties within raters (default FALSE)
<code>test</code>	a logical indicating whether the test statistic and p-value should be reported (default FALSE)
<code>na.rm</code>	deprecated and ignored

Details

The test for Kendall's W is completely equivalent to [friedman.test](#). The only advantage of this test over Friedman's is that Kendall's W has an interpretation as the coefficient of concordance. The test itself is only valid for large samples.

Kendall's W should be corrected for ties, if raters did not use a true ranking order for the subjects. The function warns if ties are present and no correction has been required.

In the presence of NAs the algorithm is switched to a generalized form for randomly incomplete datasets introduced in Brueckl (2011). This approach uses the mean Spearman ρ of all pairwise comparisons (see Kendall, 1962):

$$W = (1 + \text{mean}(\rho) * (k - 1)) / k$$

where k is the mean number of (pairwise) ratings per object and $\text{mean}(\rho)$ is calculated weighted, according to Taylor (1987), since the pairwise are possibly based on a different number of ratings, what must be reflected in weights. In case of complete datasets, it yields the same results as usual implementations of Kendall's W, except for tied ranks. In case of tied ranks, the (pairwise) correction of s used, which (already with complete datasets) results in slightly different values than the tie correction explicitly specified for W.

Value

if `test = FALSE`, a numeric scalar containing Kendall's W. Otherwise an object of class "htest" with components:

`statistic` chi-squared test statistic

`p.value` p-value for the test

`method` description of the test and coefficient variant

`data.name` name of the data

`estimate` coefficient of concordance W

`parameter` degrees of freedom and numbers of subjects and raters

Note

Based on code by Matthias Gamer and Markus Brueckl, adapted to conform to package standards.

References

- Kendall, M.G. (1948) *Rank correlation methods*. London: Griffin.
- Kendall, M.G. (1962). *Rank correlation methods* (3rd ed.). London: Griffin.
- Brueckl, M. (2011). Statistische Verfahren zur Ermittlung der Urteileruebereinstimmung. in: Altersbedingte Veraenderungen der Stimme und Sprechweise von Frauen, Berlin: Logos, 88-103.
- Taylor, J.M.G. (1987). Kendall's and Spearman's correlation coefficients in the presence of a blocking variable. *Biometrics*, 43, 409-416.

See Also

[stats::cor](#), [stats::friedman.test](#)

Other assoc.ordinal: [cStat\(\)](#), [conDisPairs\(\)](#), [ordAssocs\(\)](#)

Examples

```
anxiety <- data.frame(rater1=c(3,3,3,4,5,5,2,3,5,2,2,6,1,5,2,2,1,2,4,3),
                     rater2=c(3,6,4,6,2,4,2,4,3,3,2,3,3,3,2,2,1,3,3,4),
                     rater3=c(2,1,4,4,3,2,1,6,1,1,1,2,3,3,1,1,3,3,2,2))

kendallW(anxiety, TRUE)

# with test results
kendallW(anxiety, TRUE, test=TRUE)

# example from Siegel and Castellan (1988)
d.att <- data.frame(
  id      = c(4,21,11),
  airfare = c(5,1,4),
  climate = c(6,7,5),
  season  = c(7,6,1),
  people  = c(1,2,3),
  program = c(2,3,2),
  publicity = c(4,5,7),
  present = c(3,4,6),
  interest = c(8,8,8)
)

kendallW(t(d.att[, -1]), test = TRUE)

# which is perfectly the same as
friedman.test(y=as.matrix(d.att[, -1]), groups = d.att$id)
```

krippAlpha

Krippendorff's Alpha for Wide Data

Description

Computes Krippendorff's alpha coefficient of interrater reliability from data in wide format (with m raters). The function supports nominal, ordinal, interval, and ratio measurement levels.

Usage

```
krippAlpha(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  metric = c("nominal", "ordinal", "interval", "ratio"),
  levels = NULL,
  raters = NULL,
  output = c("def", "ext"),
  ...
)
```

Arguments

<code>x</code>	a data frame, matrix, or similar wide-format object containing ratings (columns = raters, rows = subjects/items)
<code>conf.level</code>	confidence level for bootstrap confidence intervals of Krippendorff's alpha. If NA (default), no bootstrap is computed.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See details in ConfidenceIntervals . Alpha lies in $[-1, 1]$, so the open side is reported at that boundary rather than at an infinity it cannot reach.
<code>metric</code>	character string specifying the measurement level, i.e. the difference function δ^2 used to compare two categories. One of "nominal", "ordinal", "interval", or "ratio". This selects <i>which</i> alpha is computed and has nothing to do with the confidence interval - the bootstrap interval type travels as type through ... It is called metric and not method because method means the interval method everywhere else in the suite.
<code>levels</code>	optional vector specifying the possible categories or scale values (required for the "interval" and "ratio" metrics). If NULL, levels are inferred from the data.
<code>raters</code>	optional vector specifying which columns of x are the raters. If NULL, all columns are assumed to be raters.
<code>output</code>	output format, either "def" (default) or "ext" for extended results
<code>...</code>	further arguments passed to boot . Supported arguments are type ("norm", "basic", "stud", "perc", "bca"), parallel and the number of bootstrap replicates R. Defaults are "basic" for type, option "boot.parallel" (or "no" if unset) for parallel, and 999 for R.

Details

The function constructs the coincidence matrix from the wide-format data using `CoincidenceFromWide` and then calculates Krippendorff's alpha based on the chosen distance metric:

- "nominal": Disagreement is 0 if equal, 1 otherwise.
- "ordinal": Squared difference of cumulative proportions.
- "interval": Squared Euclidean distances of scale values.
- "ratio": Squared relative differences of scale values.

Value

if output = "def" and conf.level = NA, a numeric scalar. If output = "def" and a confidence interval is requested, a named numeric vector with elements:

est point estimate of Krippendorff's alpha

lci lower confidence interval bound

uci upper confidence interval bound

If output = "ext", a list with elements:

alpha point estimate of Krippendorff's alpha

Do observed disagreement

De expected disagreement under chance

O coincidence matrix

nV category totals in coincidence space

delta2 pairwise distance matrix for the selected metric

ci named numeric vector with est, lci, and uci, or NA if no interval is requested

References

Krippendorff, K. (2018). *Content Analysis: An Introduction to Its Methodology*. Sage Publications.

See Also

[bootCI](#)

Other assoc.agreement: [ccc\(\)](#), [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#), [randolphKappa\(\)](#)

Examples

```
# Example with nominal data (3 raters, 5 subjects)
dat <- data.frame(
  r1 = c(1, 2, 1, 3, 2),
  r2 = c(1, 2, 2, 3, 2),
  r3 = c(1, 2, 1, 3, 1)
)
krippAlpha(dat, metric = "nominal")

# Interval-scaled example
dat2 <- data.frame(
  r1 = c(1, 4, 5, 7, 2),
  r2 = c(2, 5, 6, 7, 1),
  r3 = c(1, 4, 6, 6, 2)
)
krippAlpha(dat2, metric = "interval", levels = 1:7)
```

kurt	<i>Kurtosis</i>
------	-----------------

Description

`kurt()` returns the excess kurtosis, therefore the kurtosis calculates as `kurt(x) + 3` if required.

Usage

```
kurt(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("boot", "classic"),
  estimator = 3,
  weights = NULL,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a numeric vector. An object that is not a vector is coerced by <code>as.vector</code> if possible.
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of <code>"two.sided"</code> (default), <code>"left"</code> or <code>"right"</code>). See ConfidenceIntervals .
<code>method</code>	character string specifying the confidence interval method. <code>"boot"</code> (default) uses a nonparametric bootstrap, with BCa intervals unless another bootstrap type is supplied through <code>...</code> ; <code>"classic"</code> uses a Wald interval based on the asymptotic standard error. See Details and ConfidenceIntervals .
<code>estimator</code>	integer, either 1, 2 or 3 (default) defining the algorithm used for calculation. See Details .
<code>weights</code>	a numerical vector of weights the same length as <code>x</code> giving the weights to use for elements of <code>x</code>
<code>na.rm</code>	logical, indicating whether NA values should be stripped before the computation proceeds. Defaults to <code>FALSE</code> .
<code>...</code>	further arguments passed to boot when confidence intervals are calculated

Details

If `na.rm` is `TRUE` then missing values are removed before computation proceeds.

The estimator for calculating kurtosis can either be:

- 1: $g_2 = m_4 / m_2^2 - 3$
- 2: $G_2 = ((n+1) g_2 + 6) * (n-1) / ((n-2)(n-3))$
- 3: $b_2 = m_4 / s^4 - 3 = (g_2 + 3) (1 - 1/n)^2 - 3$

- 1 is the typical definition used in Stata and in many older textbooks.
- 2 is used in SAS and SPSS.
- 3 is used in MINITAB and BMDP.

Cramer (1997) mentions the asymptotic standard error of the kurtosis:

$$\text{ASE.kurt} = \sqrt{(24 * n * (n - 1)^2) / ((n - 3) * (n - 2) * (n + 3) * (n + 5))}$$

to be used for calculating the confidence intervals. This is implemented here with `method="classic"`. However, Joanes and Gill (1998) advise against this approach, pointing out that the normal assumptions would virtually always be violated. They suggest using the bootstrap method. That's why the default method for the confidence interval type is set to "boot". If not further specified the boot ci type will be chosen as "bca".

This implementation is comparably fast, as the expensive sums are coded in C.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` kurtosis estimate

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Random number generation

`method = "boot"` - the default - resamples and therefore advances R's global random number generator. Call `set.seed` beforehand for reproducible intervals.

References

Cramer, D. (1997): *Basic Statistics for Social Research* Routledge.

Joanes, D. N., Gill, C. A. (1998): Comparing measures of sample skewness and kurtosis. *The Statistician*, 47, 183-189.

See Also

[meanX](#), [sdX](#), similar code in [e1071](#)

Other shape: [skew\(\)](#)

Examples

```
kurt(Pizza$price, na.rm=TRUE)

# use sapply to calculate skewness for a data.frame
sapply(Pizza[,c("temperature", "price", "delivery_min")], kurt, na.rm=TRUE)

# or apply to do that columnwise with a matrix
apply(as.matrix(Pizza[,c("temperature", "price", "delivery_min")]), 2,
      kurt, na.rm=TRUE)
```

lambda

*Goodman Kruskal Lambda***Description**

Calculate symmetric and asymmetric Goodman Kruskal lambda and their confidence intervals. Lambda is a measure of proportional reduction in error in cross tabulation analysis. For any sample with a nominal independent variable and dependent variable (or ones that can be treated nominally), it indicates the extent to which the modal categories and frequencies for each value of the independent variable differ from the overall modal category and frequency, i.e. for all values of the independent variable together

Usage

```
lambda(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  direction = c("symmetric", "row", "column"),
  ...
)
```

Arguments

x	either a contingency table, a two-column object (matrix, data.frame or list), or a vector of observations (together with y)
y	optional second vector. If x is not a vector, y must be NULL.
conf.level	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See details in ConfidenceIntervals .
direction	type of lambda. Can be one out of "symmetric" (default), "row", "column" (abbreviations are allowed). If direction is set to "row" then lambda(RIC) (column dependent) will be reported. See Details.
...	further arguments, passed on to normalizeToConfusion and table for building the table - useNA is the usual one.

Details

Asymmetric lambda is interpreted as the probable improvement in predicting the column variable Y given knowledge of the row variable X.

The nondirectional lambda is the average of the two asymmetric lambdas, lambda(CIR) and lambda(RIC). lambda (asymmetric and symmetric) has a scale ranging from 0 to 1.

Value

if conf.level = NA, a numeric scalar. Otherwise a named numeric vector with elements:

est point estimate of Goodman–Kruskal lambda

lci lower confidence interval bound

uci upper confidence interval bound

Note

Based on code by Antti Arppe and Nanina Anderegg (confidence interval symmetric lambda), adapted to conform to package standards.

References

- Agresti, A. (2002) *Categorical Data Analysis*. John Wiley & Sons
- Goodman, L. A., Kruskal W. H. (1979) Measures of Association for Cross Classifications. New York: Springer-Verlag (contains articles appearing in *J. Amer. Statist. Assoc.* in 1954, 1959, 1963, 1972).
- http://www.nssl.noaa.gov/users/brooks/public_html/feda/papers/goodmankruskal1.pdf (might be out-dated)
- Liebetrau, A. M. (1983) *Measures of Association*, Sage University Papers Series on Quantitative Applications in the Social Sciences, 07-004. Newbury Park, CA: Sage, pp. 17–24

See Also

[Association](#)

Other assoc.nominal: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
# example from Goodman Kruskal (1954)
m <- as.table(cbind(c(1768,946,115), c(807,1387,438), c(189,746,288), c(47,53,16)))
dimnames(m) <- list(paste("A", 1:3), paste("B", 1:4))
m

# direction default is "symmetric"
lambda(m)
lambda(m, conf.level=0.95)

lambda(m, direction="row")
lambda(m, direction="column")
```

lc

Lorenz Curve

Description

Computes the empirical Lorenz curve for a numeric vector, optionally with weights and grouped via formula interface. Returns an object of class "Lc" (or "LcList" for grouped data) that can be visualized with `plot()`, `lines()`, and `points()` from the **pharos** package.

Usage

```
## S3 method for class 'formula'
lc(formula, data, subset, na.action = na.pass, ...)

## Default S3 method:
```

```
lc(x, n = rep(1, length(x)), na.rm = FALSE, ...)

## S3 method for class 'Lc'
predict(object, newdata, conf.level = NA, general = FALSE, ...)
```

Arguments

formula	a formula of the form $y \sim \text{group}$ specifying the response and grouping variable
data	optional data frame in which to evaluate formula
subset	optional expression indicating which rows of data to use
na.action	function for handling missing values in the model frame. Default is na.pass .
...	further arguments passed to <code>lc.default()</code> from <code>lc.formula()</code> . In <code>predict.Lc()</code> , the argument <code>R</code> (positive integer, default 999) controls the number of bootstrap replications when <code>conf.level</code> is supplied; it is extracted via <code>.extractBootArgs()</code> and ignored otherwise.
x	numeric vector of non-negative values
n	numeric vector of non-negative weights of the same length as <code>x</code> . Defaults to equal weights (<code>rep(1, length(x))</code>).
na.rm	logical. If TRUE, observations with NA in <code>x</code> or <code>n</code> are removed before computation. Default is FALSE.
object	object of class "Lc" as returned by <code>lc()</code>
newdata	optional numeric vector of values in $[0, 1]$ at which to evaluate the Lorenz curve via linear interpolation. If omitted, the original grid points are returned.
conf.level	numeric scalar in $(0, 1)$. If supplied, bootstrap confidence intervals at level <code>conf.level</code> are added as columns <code>lci</code> and <code>uci</code> . Set to NA (default) to suppress intervals.
general	logical. If TRUE, the generalized Lorenz curve is used. Default is FALSE.

Details

The Lorenz curve is defined as

$$L(p) = \frac{\sum_{i=1}^k w_i x_i}{\sum_{i=1}^n w_i x_i}$$

where observations are sorted in increasing order and p denotes the cumulative proportion of weights up to rank k .

The generalized Lorenz curve scales the standard curve by the weighted mean μ :

$$L_{\text{general}}(p) = L(p) \cdot \mu$$

For formula input of the form $y \sim \text{group}$, the data are split by group and a separate Lorenz curve is computed for each level. A single "Lc" object is returned when there is only one group; otherwise an "LcList".

Bootstrap confidence intervals in `predict.Lc()` are based on resampling with replacement from the (weighted) empirical distribution, followed by pointwise quantiles across bootstrap replicates. The number of replications is controlled by `R` passed via `...` and extracted by `.extractBootArgs()` (default `R = 999`).

Value

`lc.default()` an object of class "Lc", a list with components:

- `p` numeric vector of cumulative population shares starting at 0
- `L` numeric vector of Lorenz curve values at `p`
- `L.general` generalized Lorenz curve values
- `Gini` estimated Gini coefficient
- `x` original unsorted data vector
- `n` original weight vector

`lc.formula()` a single "Lc" object if the formula specifies one group, otherwise an object of class "LcList" (a named list of "Lc" objects, one per group level)

`predict.Lc()` a data frame with columns `p` and `L` (interpolated curve values at `newdata`). If `conf.level` is supplied, columns `lci` and `uci` are appended.

References

Lorenz, M. O. (1905). Methods of measuring the concentration of wealth. *Publications of the American Statistical Association*, **9**, 209–219.

See Also

[pharos::plot.Lc](#) for visualization.

Other inequality: [atkinson\(\)](#), [divCoef\(\)](#), [gini\(\)](#), [rosenbluth\(\)](#), [theil\(\)](#)

Examples

```
set.seed(1)
x <- rlnorm(100)

# default method
lc_obj <- lc(x)
lc_obj$Gini

# with weights
w <- runif(100, 0.5, 2)
lc(x, n = w)

# formula interface: grouped Lorenz curves
g <- sample(letters[1:3], 100, replace = TRUE)
d <- data.frame(x = x, g = g)
lc_grp <- lc(x ~ g, data = d)

# prediction on a regular grid
predict(lc_obj, newdata = seq(0, 1, by = 0.1))

# with 95% bootstrap confidence intervals (R = 200 for speed)
predict(lc_obj, newdata = seq(0, 1, by = 0.25),
       conf.level = 0.95, R = 200)

# plotting routines from package pharos
set.seed(1)
x <- rlnorm(100)
lc_obj <- lc(x)
```

```

# basic plot
plot(lc_obj)

# overlay confidence band
lines(lc_obj, cbandArgs = list(conf.level = 0.95))

# add points
points(lc_obj, pch = 16)

# generalized Lorenz curve
plot(lc_obj, general = TRUE)

# grouped Lorenz curves
g <- sample(letters[1:3], 100, replace = TRUE)
lc_grp <- lc(x ~ g)
plot(lc_grp)
lines(lc_grp)
points(lc_grp, pch = 16)

```

lof

Local Outlier Factor

Description

Computes the local outlier factor (LOF) of Breunig et al. (2000) for each observation. The LOF compares the local density around an observation with the densities around its k nearest neighbours, so that an observation in a sparse region surrounded by dense ones scores high even when it is not globally extreme.

Usage

```
lof(x, k)
```

Arguments

<code>x</code>	a numeric matrix or data frame, observations in rows.
<code>k</code>	the number of neighbours defining the local neighbourhood, not counting the observation itself.

Details

Scores near 1 indicate that the density around the observation matches that of its neighbours. Substantially larger values indicate a locally sparse neighbourhood and thus an outlier; the user decides on the threshold, as its scale depends on the data. Values below 1 arise in unusually dense regions and are not normally of interest.

The computation uses `lof` from the **dbscan** package, which builds a kd-tree and therefore scales to large data. Note that its `minPts` counts the observation itself, whereas k here counts neighbours only, following Breunig et al.; the translation $\text{minPts} = k + 1$ is applied internally.

Duplicated observations need care. Where an observation has more than k exact duplicates, every neighbour distance is zero, the local reachability density is infinite and the LOF is formally $0/0$.

dbscan reports 1 in that case, on the grounds that the coincident points already supply enough density for the observation not to be an outlier. Breunig et al. instead assume genuine duplicates have been removed beforehand; do that if you need results matching the paper exactly.

Note that the infinite density is not confined to the duplicates themselves. Any observation that has one of them among its own k neighbours inherits an infinite score, so a block of duplicates lying inside the data can render its whole surroundings Inf. This follows from the definition rather than from the implementation, and is a further reason to remove exact duplicates first.

Only complete observations can be scored, since the distances are otherwise undefined.

Value

a numeric vector with the local outlier factor of each observation, in the order of the rows of x .

References

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 93-104. doi:[10.1145/335191.335388](https://doi.org/10.1145/335191.335388)

See Also

[lof](#)

Examples

```
if(requireNamespace("dbscan", quietly = TRUE)) {

  res <- lof(iris[, -5], k = 10)

  # the ten most outlying observations
  head(order(res, decreasing = TRUE), 10)

  # scores near 1 are unremarkable, large ones are not
  summary(res)
}
```

logSt

Started Logarithmic Transformation and Its Inverse

Description

Transforms the data by a log transformation, modifying small and zero observations such that the transformation is linear for $x \leq \text{threshold}$ and logarithmic for $x > \text{threshold}$. So the transformation yields finite values and is continuously differentiable.

Usage

```
logSt(x, base = 10, calib = x, threshold = NULL, mult = 1)

logStInv(x, base = NULL, threshold = NULL)
```

Arguments

<code>x</code>	a vector or matrix of data to transform
<code>base</code>	a positive or complex number: the base with respect to which logarithms are computed. Defaults to 10. Use <code>base = exp(1)</code> for natural log.
<code>calib</code>	a vector or matrix of data used to calibrate the transformation and determine the required constant c
<code>threshold</code>	constant c that determines the transformation. The inverse function <code>logStInv</code> will look for an attribute named "threshold" if the argument is set to NULL.
<code>mult</code>	a tuning constant affecting the transformation of small values, as described in Details

Details

In order to avoid $\log(x) = -\infty$ for $x = 0$ in log-transformations there's often a constant added to the variable before taking the *log*. This is not always a pleasurable strategy. The function `logSt` handles this problem based on the following ideas:

- The modification should only affect the values for "small" arguments.
- What "small" is should be determined in connection with the non-zero values of the original variable, since it should behave well (be equivariant) with respect to a change in the "unit of measurement".
- The function must remain monotone, and it should remain (weakly) convex.

These criteria are implemented here as follows: The shape is determined by a threshold c at which - coming from above - the log function switches to a linear function with the same slope at this point.

This is obtained by

$$g(x) = \begin{cases} \log_{10}(x) & \text{for } x \geq c \\ \log_{10}(c) - \frac{c-x}{c \log(10)} & \text{for } x < c \end{cases}$$

Small values are determined by the threshold c . If not given by the argument `threshold`, it is determined by the quartiles q_1 and q_3 of the non-zero data as those smaller than $c = \frac{q_1^{1+r}}{q_3^r}$ where r can be set by the argument `mult`. The rationale is, that, for lognormal data, this constant identifies 2 percent of the data as small.

Beyond this limit, the transformation continues linear with the derivative of the log curve at this point.

The function chooses $\log_{10}$ rather than natural logs by default because they can be backtransformed relatively easily in mind.

Value

the transformed data. The value c used for the transformation and needed for inverse transformation is returned as `attr(, "threshold")` and the used base as `attr(, "base")`.

Note

A further idea in this context can be found in (Rocke 2003). A generalized log in order to stabilize the variance is presented as: $f(x, a) = \log(0.5 * (x + \sqrt{x^2 + a^2}))$

Based on code by Werner A. Stahel, adapted to conform to package standards.

References

Rocke, D M, Durbin B (2003): Approximate variance-stabilizing transformations for gene-expression microarray data, *Bioinformatics*. 22;19(8):966-72.

See Also

[log](#), [log10](#)

Other transform: [boxCox\(\)](#), [boxCoxLambda\(\)](#), [scaleX\(\)](#), [yeoJohnson\(\)](#)

Examples

```
dd <- c(seq(0,1,0.1), 5 * 10^rnorm(100, 0, 0.2))
dd <- sort(dd)
r.dl <- logSt(dd)
plot(dd, r.dl, type="l")
abline(v=attr(r.dl, "threshold"), lty=2)

x <- rchisq(df=3, n=100)
# should give 0 (or at least something small):
logStInv(logSt(x)) - x
```

madX

Median Absolute Deviation

Description

Compute the median absolute deviation, i.e., the (lo-/hi-) median of the absolute deviations from the median, and (by default) adjust by a factor for asymptotically normal consistency. This function wraps the specific base R function [mad](#) and extends it for the use of weights.

Usage

```
madX(
  x,
  weights = NULL,
  center = medianX,
  constant = 1.4826,
  medianType = c("standard", "low", "high"),
  na.rm = FALSE
)
```

Arguments

x	a numeric vector
weights	a numerical vector of weights the same length as x giving the weights to use for elements of x
center	a numeric center or a function applied to x. When weights are supplied, the function must support a weights argument. Defaults to medianX.
constant	scale factor (default is 1.4826)

medianType	character string selecting the "standard", "low", or "high" median for even sample sizes
na.rm	if TRUE then NA values are stripped from x before computation takes place

Details

The actual value calculated is $\text{constant} * \text{cMedian}(\text{abs}(x - \text{center}))$ with the default value of center being $\text{median}(x)$, and cMedian being the usual, the 'low' or 'high' median, see the arguments description for low and high above.

The default constant = 1.4826 (approximately $1/\Phi^{-1}(\frac{3}{4}) = 1/\text{qnorm}(3/4)$) ensures consistency, i.e.,

$$E[\text{mad}(X_1, \dots, X_n)] = \sigma$$

for X_i distributed as $N(\mu, \sigma^2)$ and large n .

If na.rm is TRUE then NA values are stripped from x before computation takes place. If this is not done then an NA value in x will cause madX to return NA.

Confidence intervals are provided by `lumen::madCI()`.

Value

a numeric scalar containing the scaled median absolute deviation

See Also

`IQR` which is simpler but less robust, `iqrX` for the same using weights, `mad`, `median`, `var` the base R equivalents `madCI` (confidence intervals).

Other dispersion: `coefVar()`, `iqrX()`, `meanAD()`, `meanSE()`, `rangeX()`, `varX()`

Examples

```
madX(c(1:9))
print(madX(c(1:9), constant = 1)) ==
      madX(c(1:8, 100), constant = 1)      # = 2 ; TRUE
x <- c(1,2,3,5,7,8)
sort(abs(x - median(x)))
c(madX(x, constant = 1, medianType="standard"),
  madX(x, constant = 1, medianType="low"),
  madX(x, constant = 1, medianType="high"))

# use weights
x <- sample(20, 30, replace = TRUE)
z <- as.numeric(names(w <- table(x)))

(m1 <- madX(z, weights=w))
(m2 <- madX(x))
stopifnot(identical(m1, m2))
```

mae	<i>Mean Absolute Error</i>
-----	----------------------------

Description

Computes the mean absolute error (MAE) between predicted and reference values.

Usage

```
mae(x, ...)
```

```
## S3 method for class 'lm'
mae(x, ...)
```

```
## Default S3 method:
mae(x, ref, na.rm = FALSE, ...)
```

Arguments

x	an object. Methods are available for numeric vectors and model objects (e.g. <code>lm</code>).
...	additional arguments passed to methods
ref	numeric vector of reference (true) values
na.rm	logical; whether to remove missing values

Details

The mean absolute error is defined as:

$$\frac{1}{n} \sum |ref - x|$$

Value

a numeric scalar containing the MAE

See Also

[mean](#), [abs](#)

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)
```

```
mae(x, ref)
```

```
# with linear model
fit <- lm(mpg ~ hp, data = mtcars)
mae(fit)
```

mape	<i>Mean Absolute Percentage Error</i>
------	---------------------------------------

Description

Computes the mean absolute percentage error (MAPE) between predicted and reference values.

Usage

```
mape(x, ...)  
  
## S3 method for class 'lm'  
mape(x, ...)  
  
## Default S3 method:  
mape(x, ref, na.rm = FALSE, ...)
```

Arguments

x	an object. Methods are available for numeric vectors and model objects (e.g. <code>lm</code>).
...	additional arguments passed to methods
ref	numeric vector of reference (true) values
na.rm	logical; whether to remove missing values

Details

The MAPE is defined as:

$$\frac{1}{n} \sum \left| \frac{ref - x}{ref} \right|$$

Note that values where `ref = 0` lead to division by zero and result in NA.

Value

a numeric scalar containing the MAPE, as a **fraction**, not a percentage: a mean absolute relative error of six percent is returned as `0.06`. Multiply by 100 for the percentage form. The name is conventional; the definition below is the one implemented.

See Also

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)

mape(x, ref)

# with linear model
fit <- lm(mpg ~ hp, data = mtcars)
```

```
mape(fit)
```

```
meanAD
```

Mean Absolute Deviation From a Center Point

Description

Calculates the mean absolute deviation from a center point, typically the sample mean or the median.

Usage

```
meanAD(x, weights = NULL, center = meanX, na.rm = FALSE)
```

Arguments

<code>x</code>	a vector containing the observations
<code>weights</code>	a numerical vector of weights the same length as <code>x</code> giving the weights to use for elements of <code>x</code>
<code>center</code>	a numeric center or a function applied to <code>x</code> . User-defined functions must accept <code>weights</code> when <code>weights</code> are supplied. Defaults to meanX() .
<code>na.rm</code>	logical; whether to remove missing values. Defaults to <code>FALSE</code> .

Details

The `meanAD` function calculates the mean absolute deviation from the mean value (or from another supplied center point) of `x`, after having removed NA values (if requested):

$$\frac{1}{n} \cdot \sum_{i=1}^n |x_i - c| \quad \text{where } c = \text{mean}(x) \text{ or } c = \text{med}(x)$$

The function supports the use of weights. The default function for the center value [meanX\(\)](#) has a `weights` argument, too. If a user defined function is used it must be assured that it has a `weights` argument.

Value

a numeric scalar containing the mean absolute deviation

Note

Rewritten following an idea of Danielle Navarro (aad in the **lsr** package).

See Also

[mad](#)

Other dispersion: [coefVar\(\)](#), [iqrX\(\)](#), [madX\(\)](#), [meanSE\(\)](#), [rangeX\(\)](#), [varX\(\)](#)

Examples

```

x <- runif(100)
meanAD(x)

speed <- c(58, 88, 40, 60, 72, 66, 80, 48, NA)
meanAD(speed)
meanAD(speed, na.rm=TRUE)

# using the median as centerpoint
x <- c(2,3,5,3,1,15,23)

meanAD(x, center=mean)
meanAD(x, center=median)

# define a fixed center
meanAD(x, center=4)

# use of weights
meanAD(x=0:6, weights=c(21,46,54,40,24,10,5))

```

meanSE	<i>Standard Error of Mean</i>
--------	-------------------------------

Description

Calculates the standard error of mean.

Usage

```
meanSE(x, sd = NULL, na.rm = FALSE)
```

Arguments

x	a non-empty numeric vector of data values
sd	the standard deviation of x. If supplied, it is interpreted as the population standard deviation. If NULL (default), the sample standard deviation sd(x) is used.
na.rm	logical. Should missing values be removed? Defaults to FALSE.

Details

meanSE() calculates the standard error of the mean defined as:

$$\frac{\sigma}{\sqrt{n}}$$

σ being standard deviation of x and n the length of x.

Value

the standard error as a numeric scalar

See Also

[lumen::meanCI](#)

Other dispersion: [coefVar\(\)](#), [iqrX\(\)](#), [madX\(\)](#), [meanAD\(\)](#), [rangeX\(\)](#), [varX\(\)](#)

Examples

```
meanSE(Pizza$price, na.rm=TRUE)

# evaluate data.frame
sapply(Pizza[, 1:4], meanSE, na.rm=TRUE)
```

meanX	<i>(Weighted) Arithmetic Mean</i>
-------	-----------------------------------

Description

Generic function for the (trimmed) arithmetic mean, possibly with given weights.

Usage

```
meanX(x, ...)

## S3 method for class 'Freq'
meanX(x, breaks, ...)

## Default S3 method:
meanX(x, weights = NULL, trim = 0, na.rm = FALSE, ...)
```

Arguments

x	an object. Currently there are methods for numeric/logical vectors and date , date-time and time interval objects. Complex vectors are allowed for trim = 0 only.
...	further arguments passed to or from other methods
breaks	breaks for calculating the mean for classified data as composed by freq
weights	non-negative numeric vector of weights of the same length as x, interpreted as frequency (replication) weights. Observations with larger weights contribute more strongly to the empirical distribution.
trim	the fraction (0 to 0.5) of observations to be trimmed from each end of x before the mean is computed. Values of trim outside that range are taken as the nearest endpoint.
na.rm	a logical value indicating whether NA values should be stripped before the computation proceeds

Details

The argument `weights` is interpreted as frequency (replication) weights. Conceptually, this corresponds to computing the statistic on a reweighted empirical distribution, where observations with larger weights represent a higher frequency in the population.

Note: Analytic (precision) weights, which assume observation-specific error variances or likelihood-based weighting, are deliberately not supported. This design ensures that all weighted statistics remain well-defined for ordinal, robust, and distribution-based measures such as medians, quantiles, and measures of dispersion.

`trim` and `weights` can't be used together at the same time.

Value

if `trim` is zero (the default), the arithmetic mean of the values in `x` is computed, as a numeric or complex vector of length one. If `x` is not logical (coerced to numeric), numeric (including integer) or complex, `NA_real_` is returned, with a warning.

If `trim` is non-zero, a symmetrically trimmed mean is computed with a fraction of `trim` observations deleted from each end before the mean is computed.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

[weighted.mean](#), [mean.POSIXct](#), [colMeans](#) for row and column means.

Other location: [gmean\(\)](#), [hmean\(\)](#), [hodgesLehmann\(\)](#), [huberM\(\)](#), [medianX\(\)](#), [modeX\(\)](#), [tukeyBiweight\(\)](#)

Examples

```
x <- c(0:10, 50)
xm <- meanX(x)
c(xm, meanX(x, trim = 0.10))
```

medianX

(Weighted) Median Value

Description

Compute the sample median. The function basically wraps the function [quantileX\(\)](#), which offers the option to define weights.

For grouped data the median can be estimated by linear interpolation within the class containing the median, which is implemented in the interface for `Freq`-objects.

Usage

```

medianX(x, ...)

## Default S3 method:
medianX(x, weights = NULL, na.rm = FALSE, ...)

## S3 method for class 'factor'
medianX(x, na.rm = FALSE, ...)

## S3 method for class 'Freq'
medianX(x, breaks, ...)

```

Arguments

<code>x</code>	an object for which a method has been defined, or a numeric vector containing the values whose median is to be computed
<code>...</code>	further arguments passed to or from other methods
<code>weights</code>	a numerical vector of weights the same length as <code>x</code> giving the weights to use for elements of <code>x</code>
<code>na.rm</code>	a logical value indicating whether NA values should be stripped before the computation proceeds
<code>breaks</code>	breaks for calculating the mean for classified data as composed by freq

Details

This is a generic function for which methods can be written. However, the default method makes use of `is.na`, `sort` and `mean` from package **base** all of which are generic, and so the default method will work for most classes (e.g., "[Date](#)") for which a median is a reasonable concept.

Calculating the median for ordered factors is not implemented in standard R, as it's not well defined (it is not clear what to do if the median sits between two levels in factors of even length). This function returns the high median and prints a warning if the low median would be different (which is supposed to be a rare event). There's a vivid discussion between experts going on whether this should be defined or not. We'll wait for definitive results and enjoy the function's comfort so far...

Value

the default method returns a length-one object of the same type as `x`, except when `x` is integer of even length, when the result will be double.

If there are no values or if `na.rm = FALSE` and there are NA values the result is NA of the same type as `x` (or more generally the result of `x[FALSE][NA]`).

Note

There are alternative approaches for calculating weighted median (e.g. `matrixstats::weightedMedian`).

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

[quantile](#) for general quantiles,

[Maechler on R-help mailing list, 2003-Nov](#)

[StackOverflow discussion on median](#)

Other location: [gmean\(\)](#), [hmean\(\)](#), [hodgesLehmann\(\)](#), [huberM\(\)](#), [meanX\(\)](#), [modeX\(\)](#), [tukeyBiweight\(\)](#)

Examples

```
medianX(1:4)           # = 2.5 [even number]
medianX(c(1:3, 100, 1000)) # = 3 [odd, robust]

# Approximation for classified data
breaks <- seq(10,70, 10)
medianX(
  freq(cut(Pizza$temperature, breaks=breaks)),
  breaks=breaks)

# compared to
medianX(Pizza$temperature)

# starting from a classified table
# from    to  income
#    0   4000    20
# 4000   6000    42
# 6000   8000    31
# 8000  10000    12

# freq(as.table(c(20,42,31,12)))
#   level freq  perc cumfreq cumperc
# 1     A   20 19.0%     20   19.0%
# 2     B   42 40.0%     62   59.0%
# 3     C   31 29.5%     93   88.6%
# 4     D   12 11.4%    105  100.0%

medianX(freq(as.table(c(20,42,31,12))), breaks=c(0,4000,6000,8000,10000))

# use weights
x <- sample(20, 30, replace = TRUE)
z <- as.numeric(names(w <- table(x)))

(m1 <- medianX(z, weights=w))
(m2 <- medianX(x))
stopifnot(identical(m1, m2))
```

modeX

Mode (most Frequent Value(s))

Description

Calculate the mode, the most frequent value, of a numeric or character vector x.

Usage

```
modeX(x, na.rm = FALSE)
```

Arguments

<code>x</code>	a non-empty numeric vector of data values
<code>na.rm</code>	logical. Should missing values be removed? Defaults to FALSE.

Details

The mode is mainly useful for qualitative data, sometimes still for integer vectors.

For numeric vectors, the interest lies less in central tendency than in conspicuous accumulation points, which can indicate data errors. `desc()` therefore reports it within the numeric description once the frequency of a single value exceeds a threshold, since ties are generally unexpected in numeric data unless the generating process explains them.

NA handling follows the package standard: a single NA yields NA. This is conservative, as the mode is sometimes determined unambiguously despite missing values. For `x = c(1, 1, 1, 1, 2, 2, NA)` the mode is 1 whatever the missing value is, and the modal frequency lies between 4 and 5. Exploiting this is left to the user.

The mode is elsewhere often obtained by tabulating every element and returning the most frequent. This function uses a dedicated C++ data structure and determines only the most frequent element, making it orders of magnitude faster, especially for large numeric vectors with many distinct values.

Note: There are other approaches for determining the mode, e.g. one might use `density(x)$x[which.max(density(x)$y)]` for quantitative data, resp. `hist()`.

Another interesting idea for a more robust estimation of the mode:

```
peak <- optimize(function(x, model)
  predict(model, data.frame(x = x)),
  c(min(x), max(x)), maximum = TRUE, model = y.loess)
points(peak$maximum, peak$objective)
```

Value

the most frequent value as a number or character, depending on `class(x)`. If there is more than one, all are returned in a vector.

The modal frequency is attached as attribute named "freq".

Note

Great Rcpp part contributed by Joseph Wood and Ralf Stubner.

References

[rcpp-fast-statistical-mode](#)

See Also

Other location: [gmean\(\)](#), [hmean\(\)](#), [hodgesLehmann\(\)](#), [huberM\(\)](#), [meanX\(\)](#), [medianX\(\)](#), [tukeyBiweight\(\)](#)

Examples

```
# normal mode
modeX(c(0:5, 5))

modeX(5)
modeX(NA)
modeX(c(NA, NA))
modeX(c(NA, 0:5))
modeX(c(NA, 0:5), na.rm=TRUE)
modeX(c(NA, 0:5, 5), na.rm=TRUE)

# returns all encountered modes, if several exist
modeX(c(0:5, 4, 5, 6))

modeX(Pizza$driver)
modeX(Pizza$driver, na.rm=TRUE)
modeX(as.character(Pizza$driver), na.rm=TRUE)

# use sapply for evaluating data.frames (resp. apply for matrices)
sapply(Pizza[,c("driver", "temperature", "date")], modeX, na.rm=TRUE)
```

mse	<i>Mean Squared Error</i>
-----	---------------------------

Description

Computes the mean squared error (MSE) between predicted and reference values.

Usage

```
mse(x, ...)
```

```
## S3 method for class 'lm'
mse(x, ...)
```

```
## Default S3 method:
mse(x, ref, na.rm = FALSE, ...)
```

Arguments

x	an object. Methods are available for numeric vectors and model objects (e.g. <code>lm</code>).
...	additional arguments passed to methods
ref	numeric vector of reference (true) values
na.rm	logical; whether to remove missing values

Details

The mean squared error is defined as:

$$\frac{1}{n} \sum (ref - x)^2$$

Value

a numeric scalar containing the MSE

See Also

[mean](#)

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)

mse(x, ref)

# with linear model
fit <- lm(mpg ~ hp, data = mtcars)
mse(fit)
```

mutInf

Mutual Information

Description

Computes the mutual information (MI) between two variables from a contingency table.

Usage

```
mutInf(x, y = NULL, base = 2, normalize = FALSE, ...)
```

Arguments

x	a contingency table, matrix, or vector that can be coerced into a contingency table
y	an optional second variable used together with x to create a contingency table via <code>table(x, y, ...)</code>
base	logarithm base. Defaults to 2 (bits).
normalize	logical. If TRUE, returns normalized mutual information (NMI).
...	additional arguments passed to <code>table()</code>

Details

Mutual information quantifies the amount of information obtained about one variable through observing the other.

It is defined as:

$$I(X; Y) = H(X) + H(Y) - H(X, Y)$$

where $H(X)$ and $H(Y)$ are marginal entropies and $H(X, Y)$ is the joint entropy.

Mutual information is always nonnegative:

$$I(X; Y) \geq 0$$

Larger values indicate stronger dependence.

If `normalize = TRUE`, the returned value is:

$$\frac{I(X; Y)}{\sqrt{H(X)H(Y)}}$$

which approximately scales the measure to $[0, 1]$.

Value

a numeric scalar containing the mutual information

References

Cover TM, Thomas JA (2006). Elements of Information Theory (2nd ed.). Wiley.

See Also

[entropy](#)

Other assoc.nominal: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
tab <- matrix(
  c(10, 20,
    30, 40),
  nrow = 2
)

mutInf(tab)

mutInf(tab, normalize = TRUE)

x <- sample(letters[1:3], 100, TRUE)
y <- sample(LETTERS[1:2], 100, TRUE)

mutInf(x, y)
```

nmae*Normalized Mean Absolute Error*

Description

Computes the normalized mean absolute error (NMAE) between predictions and reference values.

Usage

```
nmae(x, ref, trainY, na.rm = FALSE)
```

Arguments

<code>x</code>	numeric vector of predicted values
<code>ref</code>	numeric vector of reference (true) values
<code>trainY</code>	numeric vector used as the normalization baseline
<code>na.rm</code>	logical; whether to remove incomplete cases before the computation. Defaults to FALSE, in which case a missing value anywhere makes the result NA.

Details

The normalized mean absolute error is defined as:

$$\frac{\sum |ref - x|}{\sum |ref - mean(trainY)|}$$

The denominator represents the absolute deviation from the mean of the training response, providing a baseline for comparison.

If the denominator is zero, NA is returned.

Value

a numeric scalar containing the normalized mean absolute error

See Also

[mean](#), [abs](#)

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmse\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)
trainY <- c(2, 3, 4, 3)

nmae(x, ref, trainY)
```

nmse*Normalized Mean Squared Error*

Description

Computes the normalized mean squared error (NMSE) between predictions and reference values.

Usage

```
nmse(x, ref, trainY, na.rm = FALSE)
```

Arguments

<code>x</code>	numeric vector of predicted values
<code>ref</code>	numeric vector of reference (true) values
<code>trainY</code>	numeric vector used as the normalization baseline
<code>na.rm</code>	logical; whether to remove incomplete cases before the computation. Defaults to FALSE, in which case a missing value anywhere makes the result NA.

Details

The normalized mean squared error is defined as:

$$\frac{\sum (ref - x)^2}{\sum (ref - \text{mean}(\text{trainY}))^2}$$

The denominator represents the squared deviation from the mean of the training response, providing a baseline for comparison.

If the denominator is zero, NA is returned.

Value

a numeric scalar containing the normalized mean squared error

See Also

[mean](#), [sum](#)

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [rmse\(\)](#), [smape\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)
trainY <- c(2, 3, 4, 3)

nmse(x, ref, trainY)
```

normalizeToConfusion *Normalize Input to a Contingency or Agreement Table*

Description

Converts diverse input formats into a numeric contingency matrix suitable for agreement measures (e.g. Cohen's Kappa) or association measures (e.g. Cramer's V). Accepted input formats include tables, matrices, data frames, lists, and raw rating vectors.

Usage

```
normalizeToConfusion(
  x,
  y = NULL,
  levels = NULL,
  useNA = "no",
  mode = c("agreement", "association")
)
```

Arguments

<code>x</code>	input object. Accepted formats: a table representing a pre-computed contingency table; a square numeric matrix representing a pre-computed contingency table; a numeric matrix with exactly 2 columns (one per rater); a list or data.frame with exactly 2 elements; or a vector if y is also supplied.
<code>y</code>	optional second rating vector. If supplied, x and y are tabulated together.
<code>levels</code>	optional category levels. These <i>rename</i> the categories in place - the counts are not moved - so supplying an order different from the one a table already carries reassigns the counts to other labels. For mode = "agreement", use an atomic vector of common levels shared by both raters; for mode = "association", use a list of length 2, <code>list(x_levels, y_levels)</code> .
<code>useNA</code>	passed to table . Controls whether NA values appear as a level. One of "no" (default), "ifany", or "always".
<code>mode</code>	character string, either "agreement" (default) or "association". Agreement mode enforces a square table with identical row and column names. Association mode allows rectangular tables with independent row and column levels.

Details

The function handles the following input formats:

table A pre-computed 2D contingency table which is validated via [isConfusionTable](#). For mode = "agreement", the table must be square with identical row and column names.

matrix Either a pre-computed contingency matrix (square, passing [isConfusionTable](#) validation) or a two-rater matrix with exactly 2 columns. A 2-column non-square matrix is always treated as a rater matrix; each column is one rater's ratings.

two vectors If both x and y are supplied, they are tabulated via [table](#) after coercing to factors.

list or data.frame with 2 elements Each element is treated as one rater's ratings.

For mode = "agreement", levels must be shared between both raters and the resulting table is square. For mode = "association", row and column levels may differ and the table may be rectangular.

Value

a numeric contingency matrix with dimnames set according to levels, if supplied, or derived from the input

See Also

[isConfusionTable\(\)](#), [raterFrame\(\)](#), [pairApply\(\)](#)
 Other agreement: [blandAltmanData\(\)](#), [raterFrame\(\)](#)

Examples

```
A <- c("pos", "neg", "pos", "inc")
B <- c("pos", "pos", "neg", "inc")
normalizeToConfusion(A, B)

tab <- table(A, B)
normalizeToConfusion(tab)

set.seed(1)
C <- sample(c("pos", "neg", "inc"), length(A), TRUE)
df <- data.frame(R1=A, R2=B, R3=C)
normalizeToConfusion(df[, 1:2])      # R1 vs R2
normalizeToConfusion(df[, c(1,3)])   # R1 vs R3

# list of rating vectors:
normalizeToConfusion(list(A, B))

# use NAs
B[2] <- NA
normalizeToConfusion(A, B, useNA = "always")

anxiety <- data.frame(
  rater1 = c(3,3,3,4,5,5,2),
  rater2 = c(3,6,4,6,2,4,2),
  rater3 = c(2,1,4,4,3,2,1)
)

x <- anxiety[, 1]
y <- anxiety[, 2]

# two vectors:
normalizeToConfusion(x, y)

# matrix / data.frame with 2 columns (subjects x raters):
normalizeToConfusion(cbind(x, y))
normalizeToConfusion(data.frame(x, y))

# list with 2 elements:
normalizeToConfusion(list(x, y))

# pre-built table:
ratingscale <- sort(unique(c(x, y)))
normalizeToConfusion(table(factor(x, levels = ratingscale),
                           factor(y, levels = ratingscale)))

d.anxiety <- data.frame(
```

```

rater = c("rater1", "rater1", "rater1", "rater1", "rater1", "rater1", "rater1",
          "rater2", "rater2", "rater2", "rater2", "rater2", "rater2", "rater2",
          "rater3", "rater3", "rater3", "rater3", "rater3", "rater3", "rater3"),
rating = c(3, 3, 3, 4, 5, 5, 2,
           3, 6, 4, 6, 2, 4, 2,
           2, 1, 4, 4, 3, 2, 1),
subj    = c(1, 2, 3, 4, 5, 6, 7,
            1, 2, 3, 4, 5, 6, 7,
            1, 2, 3, 4, 5, 6, 7)
)

# via raterFrame (wide format, subjects x raters):
normalizeToConfusion(
  raterFrame(rating ~ subj | rater, data = d.anxiety,
             subset = rater %in% c("rater1", "rater2"), dropSubj = TRUE)
)

```

oddsRatio

Compute Odds Ratios

Description

Computes odds ratios, either from a 2x2 contingency table or from a binomial generalized linear model. The table method returns a single odds ratio, the model method one odds ratio per regression coefficient.

Usage

```

oddsRatio(x, ...)

## Default S3 method:
oddsRatio(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("wald", "exact", "midp"),
  interval = c(0, 1000),
  ...
)

## S3 method for class 'glm'
oddsRatio(
  x,
  conf.level = 0.95,
  sides = c("two.sided", "left", "right"),
  method = c("wald", "profile"),
  ...
)

## S3 method for class 'OddsRatio'
print(x, digits = 3, ...)

```

Arguments

<code>x</code>	a 2x2 contingency table, two vectors to be cross-tabulated, or a binomial <code>glm</code> object
<code>...</code>	further arguments passed to methods. For the default method with two vectors, these reach <code>table</code> , so <code>useNA</code> can be set here.
<code>y</code>	optional second variable. If supplied, <code>table(x, y, ...)</code> is computed.
<code>conf.level</code>	confidence level of the interval. For the table method NA (the default) returns the point estimate only; the model method computes an interval by default.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals . An odds ratio is bounded below by 0 and unbounded above, so the open side is reported at 0 or Inf accordingly. Ignored, with a warning, for <code>method = "profile"</code> .
<code>method</code>	character string specifying the interval method. For a contingency table one of "wald", "exact" or "midp"; for a binomial model one of "wald" or "profile". See Details.
<code>interval</code>	numeric vector of length two giving the search interval for the root finding in the mid-p method. Only used by <code>method = "midp"</code> ; widen it if the reported bound sits at one of its ends.
<code>digits</code>	number of digits used for printing

Details

Contingency tables: For a 2x2 table the odds ratio is

$$OR = \frac{n_{11}n_{22}}{n_{12}n_{21}}$$

Three interval methods are available. "wald" is the asymptotic interval on the log scale, fast and adequate for reasonably large counts. "exact" is the conditional interval based on the noncentral hypergeometric distribution (Fisher); it guarantees coverage but is conservative, sometimes markedly so. "midp" halves the probability of the observed table and lies between the two: it has coverage closer to the nominal level than the exact interval without the Wald interval's reliance on large counts. With a zero cell the point estimate is 0 or Inf and only "exact" and "midp" still deliver a finite bound on the informative side.

Binomial models: For a model fitted with `glm(family = binomial)`, each coefficient is exponentiated: $\exp(\beta_j)$ is the factor by which the odds of the response are multiplied when the corresponding predictor increases by one unit, all other predictors held fixed. For a dummy variable this is the odds ratio between the level and its reference level.

The intercept is exponentiated along with the rest, but $\exp(\beta_0)$ is *not* an odds ratio - it is the odds of the response when all predictors are zero. It is reported for completeness and is usually not the quantity of interest.

Two interval methods are available. "wald" is the symmetric interval on the log-odds scale, back-transformed. "profile" inverts the likelihood ratio test through `confint.glm`; it is asymmetric on the odds scale, generally more reliable in small samples or with sparse cells, and considerably slower because the model is refitted along each coefficient. Profile intervals are two-sided by construction, so `sides` is ignored for them and a warning is issued.

Unlike the table method, the model method computes an interval by default (`conf.level = 0.95`): a coefficient table without intervals would be less informative than `summary()` itself.

Value

For a contingency table with `conf.level = NA` a numeric scalar, otherwise a named numeric vector with the elements `est`, `lci` and `uci`.

For a binomial model an object of class "OddsRatio", a list with:

`coefficients` a data frame with one row per coefficient and the columns `term`, `est` (the exponentiated coefficient), `logEst` (the coefficient itself), `stdError` (on the log scale), `pValue`, `lci` and `uci`

`source` "glm"

`method`, `conf.level`, `sides` as supplied - with `sides` recording what was computed, which for `method = "profile"` is always "two.sided"

`nObs` number of observations used in the fit

`call` the model call

There is a print method; the interval bounds are on the odds scale, the standard error on the log scale.

Note

No short alias is exported by default to avoid conflicts with **rlang** and base R naming conventions. Call `attachAliases()` once per session (or script) to make `or()` available as a convenient shorthand.

References

Agresti, A. (2013). *Categorical Data Analysis* (3rd ed.). Wiley.

Fisher, R. A. (1935). The logic of inductive inference. *Journal of the Royal Statistical Society*, 98(1), 39–82.

Gart, J. J. (1966). Alternative analyses of contingency tables. *Journal of the Royal Statistical Society Series B*, 28(1), 164–179.

See Also

`attachAliases`, `relRisk`, `confint.glm`

Other effect.size: `cohenD()`, `cohenH()`, `etaSq()`, `glassDelta()`, `relRisk()`

Examples

```
# --- 2x2 table -----
tab <- matrix(c(10, 20,
                5, 30), nrow = 2)

oddsRatio(tab)
oddsRatio(tab, conf.level = 0.95)

# the exact interval is the widest, the Wald interval the narrowest
sapply(c("wald", "exact", "midp"),
       function(m) oddsRatio(tab, conf.level = 0.95, method = m))

# one-sided: "left" carries the finite lower bound
oddsRatio(tab, conf.level = 0.95, sides = "left")
```

```
# --- binomial model -----
fit <- glm(vs ~ am + wt, data = mtcars, family = binomial)

oddsRatio(fit)

# the exponentiated intercept is the baseline odds, not an odds ratio
res <- oddsRatio(fit)
res$coefficients

# profile likelihood intervals: asymmetric on the odds scale, slower
oddsRatio(fit, method = "profile")
```

ordAssocs	<i>Ordinal Association Measures</i>
-----------	-------------------------------------

Description

Computes concordance-based association measures for two ordinal variables or a contingency table, optionally with confidence intervals.

Usage

```
ordAssocs(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  which = c("all", "gamma", "tauA", "tauB", "tauC", "somers", "cstat"),
  direction = c("row", "column")
)

gkGamma(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  direction = c("row", "column")
)

kendallTauA(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right")
)

kendallTauB(
  x,
  y = NULL,
```

```

    conf.level = NA,
    sides = c("two.sided", "left", "right")
  )

  stuartTauC(
    x,
    y = NULL,
    conf.level = NA,
    sides = c("two.sided", "left", "right")
  )

  somersDelta(
    x,
    y = NULL,
    conf.level = NA,
    sides = c("two.sided", "left", "right"),
    direction = c("row", "column")
  )

```

Arguments

<code>x</code>	numeric or ordinal vector, or a two-dimensional contingency table
<code>y</code>	optional second vector of the same length as <code>x</code>
<code>conf.level</code>	confidence level of the interval; NA returns only the estimate
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See details in ConfidenceIntervals . The open side is closed at the boundary of the parameter's range - ± 1 for gamma, the tau family and Somers' D , and $[0, 1]$ for the c statistic - rather than at an infinity none of them can reach.
<code>which</code>	measure returned by <code>ordAssocs()</code> : "all", "gamma", "tauA", "tauB", "tauC", "somers", or "cstat"
<code>direction</code>	direction of Somers' D ; "row" or "column"

Details

Let P and Q denote the numbers of concordant and discordant pairs, T_X and T_Y the numbers tied on X and Y , $n_0 = n(n-1)/2$, and m the smaller table dimension. [conDisPairs](#) returns these pair counts as `C`, `D`, `Ties_X`, and `Ties_Y` and describes their calculation.

The measures are defined as follows:

Goodman–Kruskal γ (`gkGamma`):

$$\gamma = \frac{P - Q}{P + Q}$$

Kendall τ_a (`kendallTauA`):

$$\tau_a = \frac{P - Q}{n_0}$$

Kendall τ_b (`kendallTauB`):

$$\tau_b = \frac{P - Q}{\sqrt{(n_0 - T_X)(n_0 - T_Y)}}$$

Stuart τ_c (stuartTauC):

$$\tau_c = \frac{2m(P - Q)}{n^2(m - 1)}$$

Somers $D_{X|Y}$ (somersDelta):

$$D_{X|Y} = \frac{P - Q}{n_0 - T_Y}$$

Gamma and Kendall's coefficients are symmetric.

Somers' D is directional: for tables, `direction="row"` returns $D_{R|C}$ and `direction="column"` returns $D_{C|R}$. In vector mode, `ordAssocs()` returns $D_{X|Y}$; reverse x and y for the other direction. `somersDelta()` performs this reversal when `direction="column"`.

The c-statistic is used for a binary outcome and consistently ordered predictions and is related to Somers D as $C_{stat} = (D + 1)/2$. See [cStat](#) for direct estimation of the c-statistic from predicted values and a binary response.

Value

The extractor functions return an unnamed numeric scalar if `conf.level = NA`, and otherwise a named numeric vector with elements:

`est` point estimate

`lci` lower confidence interval bound

`uci` upper confidence interval bound

`ordAssocs()` returns the same structure in a named list containing the selected measures.

References

Agresti, A. (2002) *Categorical Data Analysis*. Wiley, pp. 57–59.

Brown, M. B. and Benedetti, J. K. (1977). Sampling behavior of tests for correlation in two-way contingency tables. *JASA*, 72, 309–315.

Goodman, L. A. and Kruskal, W. H. (1954, 1963). Measures of association for cross classifications. *JASA*, 49, 732–764; 58, 310–364.

Kendall, M. (1955) *Rank Correlation Methods*. Charles Griffin.

Somers, R. H. (1962). A new asymmetric measure of association for ordinal variables. *American Sociological Review*, 27, 799–811.

See Also

Other `assoc.ordinal`: [cStat\(\)](#), [conDisPairs\(\)](#), [kendallW\(\)](#)

Examples

```
# Table example:
tab <- as.table(rbind(
  c(26, 26, 23, 18, 9),
  c(6, 7, 9, 14, 23)
))

ordAssocs(tab, conf.level = 0.95)
kendallTauB(tab, conf.level = 0.95)
somersDelta(tab, direction = "column")
```

```
# Vector example
x <- c(1,2,2,3,3,3,4,5)
y <- c(1,3,2,1,5,3,4,5)

kendallTauA(x, y, conf.level=0.95)
somersDelta(x, y, direction = "column", conf.level=0.95)
```

outlier

Outlier

Description

Return outliers following Tukey's boxplot and Hampel's median/mad definition.

Usage

```
outlier(x, method = c("boxplot", "hampel"), value = TRUE, na.rm = FALSE)
```

Arguments

x	a non-empty numeric vector of data values
method	the method to be used. So far Tukey's boxplot and Hampel's rule are implemented.
value	logical. If FALSE, a vector containing the (integer) indices of the outliers is returned, and if TRUE (default), a vector containing the matching elements themselves is returned.
na.rm	logical. Should missing values be removed? Defaults to FALSE.

Details

Outlier detection is a tricky problem and should be handled with care. We implement Tukey's boxplot rule as a rough idea of spotting extreme values. The fences are built from the hinges, exactly as `boxplot.stats` does, so the result matches what a boxplot of the same data draws. Note that the hinges are not the type-7 quartiles of `quantile` and differ from them for many sample sizes.

Hampel considers values outside of median $\pm 3 \cdot$ (median absolute deviation) to be outliers.

Value

the outlying values if `value = TRUE`; otherwise their indices

Note

Performance improvement by Luis Gustavo Schuck.

References

Hampel F. R. (1974) The influence curve and its role in robust estimation, *Journal of the American Statistical Association*, 69, 382-393

See Also[boxplot](#)Other data.inspection: [abstract\(\)](#)**Examples**

```

outlier(Pizza$temperature, na.rm=TRUE)

# it's the same as the result from boxplot
sort(Pizza$temperature[outlier(Pizza$temperature, value=FALSE, na.rm=TRUE)])
b <- boxplot(Pizza$temperature, plot=FALSE)
sort(b$out)

# nice to find the corresponding rows
Pizza[outlier(Pizza$temperature, value=FALSE, na.rm=TRUE), ]

# compare to Hampel's rule
outlier(Pizza$temperature, method="hampel", na.rm=TRUE)

# outliers for the each driver
tapply(Pizza$temperature, Pizza$driver, outlier, na.rm=TRUE)

# the same as:
boxplot(temperature ~ driver, Pizza)$out

```

pabak

*Prevalence-Adjusted and Bias-Adjusted Kappa (PABAK)***Description**

Computes the prevalence-adjusted and bias-adjusted kappa (PABAK) as a measure of inter-rater agreement, together with an asymptotic confidence interval and the auxiliary prevalence and bias indices.

Usage

```

pabak(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  ...
)

```

Arguments

x a square confusion matrix (or data frame), or a categorical vector when y is provided.

<code>y</code>	NULL (default) or a categorical vector of the same length as <code>x</code> . When supplied, <code>table(x, y, ...)</code> is computed internally.
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>...</code>	further arguments passed to <code>table</code> for the vector interface, for example <code>useNA</code> .

Details

Cohen's kappa can be paradoxically low when the prevalence of one category differs markedly from 0.5 (prevalence effect) or when the two raters systematically disagree on the marginal frequency of a category (bias effect). PABAK corrects for both distortions by replacing the chance agreement term with the fixed value $p_e = 0.5$. The statistic simplifies to

$$\text{PABAK} = 2p_o - 1$$

where p_o is the observed proportion of agreement (diagonal sum of the relative-frequency table).

The asymptotic standard error is derived from the delta method applied to the above identity:

$$\text{SE} = 2\sqrt{p_o(1 - p_o)/n}$$

Two diagnostic indices quantify the prevalence and bias effects that can cause Cohen's kappa to differ from PABAK. They are defined only for 2×2 tables following Byrt et al. (1993); for larger tables they are NA:

Prevalence index $|p_{11} - p_{22}|$. A high value indicates that one outcome dominates, which deflates Cohen's kappa relative to PABAK.

Bias index $|p_{1.} - p_{.1}|$. A high value signals that the two raters assign the positive label at systematically different rates.

When both indices are near zero, PABAK and Cohen's kappa will be nearly identical. Both are returned as attributes rather than as elements of the result, since they are separate diagnostics rather than further views of the estimate, and they do not depend on `conf.level`.

The Wald confidence interval is truncated to the admissible range $[-1, 1]$.

`sides` names the side on which the finite bound lies: "left" yields $[lci, \infty)$ and "right" $(-\infty, uci]$.

Data can be passed either as a square confusion matrix (or data frame) in `x`, or as two vectors `x` and `y`, in which case `table(x, y, ...)` is computed internally.

Missing values are handled as `table` does - excluded by default. Pass `useNA = "ifany"` via `...` to include them.

Value

a named numeric vector.

If `conf.level = NA`, a single element est; otherwise the elements:

`est` point estimate of PABAK.

`lci` lower confidence interval bound.

`uci` upper confidence interval bound.

In both cases the diagnostics are attached as attributes:

nObs number of observations in the table.

prevalenceIndex prevalence index, or NA for $k > 2$.

biasIndex bias index, or NA for $k > 2$.

References

Byrt, T., Bishop, J., & Carlin, J. B. (1993). Bias, prevalence and kappa. *Journal of Clinical Epidemiology*, 46(5), 423-429. doi:10.1016/08954356(93)90018V

Hoehler, F. K. (2000). Bias and prevalence effects on kappa viewed in terms of sensitivity and specificity. *Journal of Clinical Epidemiology*, 53(5), 499-503. doi:10.1016/S08954356(99)001742

See Also

Other assoc.agreement: `ccc()`, `cohenKappa()`, `cronbachAlpha()`, `icc()`, `kappaM()`, `krippAlpha()`, `percAgreement()`, `randolphKappa()`

Examples

```
# 2x2 table: two raters classifying 100 patients as positive / negative
m2 <- matrix(c(45, 15, 5, 35), nrow = 2, byrow = TRUE,
             dimnames = list(rater1 = c("pos", "neg"),
                             rater2 = c("pos", "neg")))

pabak(m2)
pabak(m2, conf.level = 0.95)

# the diagnostics are available either way
attr(pabak(m2), "prevalenceIndex")
attr(pabak(m2), "biasIndex")

# Compare with cohenKappa: the indices show where the gap comes from
cohenKappa(m2)

# 4x4 table: the indices are NA for k > 2
m4 <- matrix(c(400, 40, 20, 10,
              50, 300, 60, 20,
              10, 40, 120, 5,
              5, 90, 50, 80),
            nrow = 4, byrow = TRUE)

pabak(m4, conf.level = 0.95)

# vector interface
x <- bedrock::untable(m2)
pabak(x$rater1, x$rater2, conf.level = 0.95)
```

Description

Find the confidence intervals for a specified correlation based on Fisher's z-transformation.

Usage

```
pearsonCor(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  scoresType = "table",
  na.rm = FALSE
)
```

Arguments

<code>x</code>	a numeric vector, matrix, or table
<code>y</code>	NULL (default) or a vector with compatible dimensions to <code>x</code> . If <code>y</code> is supplied, <code>table(x, y, ...)</code> is calculated.
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>scoresType</code>	score calculation method for table input
<code>na.rm</code>	logical, default FALSE determining if complete cases should be respected

Details

The sampling distribution of Pearson's r is not normal. Fisher developed a transformation now called "Fisher's z-transformation" used for the calculation of normal distributed confidence intervals.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of Pearson's correlation coefficient

`lci` lower confidence interval bound

`uci` upper confidence interval bound

Note

Based on code by William Revelle, adapted to conform to package standards.

See Also

[lumen::fisherZ](#), [lumen::fisherZInv](#)

Other assoc.continuous: [corPart\(\)](#), [corPolychor\(\)](#), [findCorrX\(\)](#), [hoeffdingD\(\)](#), [keepSig\(\)](#), [spearmanCor\(\)](#)

Examples

```
with(swiss, pearsonCor(Fertility, Agriculture))
with(swiss, pearsonCor(Fertility, Agriculture, conf.level=0.95))

bedrock::pairApply(swiss, pearsonCor)

bedrock::pairApply(swiss,
  function(x, y) fmCI(pearsonCor(x, y, conf.level=0.95),
    digits=3, leadDigits=0))
```

percAgreement	<i>Percent Agreement with Design-Based SE and CI</i>
---------------	--

Description

Computes the proportion of agreement (percent agreement) along with design-based standard errors and confidence intervals following Klein and Gwet.

Usage

```
percAgreement(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  input = c("auto", "confusion", "ratings"),
  fpc = 0,
  output = c("def", "ext"),
  ...
)
```

Arguments

x	a confusion matrix or a matrix or data frame of ratings with subjects in rows and raters in columns
y	optional second rating vector used to construct a confusion matrix
conf.level	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
sides	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals . A proportion of agreement lies in $[0, 1]$, so the open side is reported at that boundary rather than at $\pm\infty$.
input	character string specifying the input format: "auto", "confusion", or "ratings"
fpc	finite population correction, the sampling fraction n/N in $[0, 1]$ (default 0)
output	output format, either "def" (default) or "ext" for extended results
...	must be empty. Named arguments are rejected rather than silently ignored.

Details

For a confusion matrix the agreement indicator of a subject is 1 on the diagonal and 0 elsewhere. For a ratings matrix the subject-wise agreement is the proportion of agreeing rater *pairs*,

$$p_{o,i} = \frac{\sum_j n_{ij}(n_{ij} - 1)}{m_i(m_i - 1)},$$

where n_{ij} is the number of raters who assigned subject i to category j and m_i the number of non-missing ratings for that subject. Subjects with fewer than two ratings carry no information about agreement and are excluded (they still count towards n in the variance, following Gwet).

Value

if output = "def" and conf.level = NA, a numeric scalar; otherwise a named numeric vector with elements:

est proportion of agreement

lci lower confidence interval bound

uci upper confidence interval bound

if output = "ext", a list with the elements est, se, ci (the named triple above), n, nPairable and method.

See Also

Other assoc.agreement: [ccc\(\)](#), [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [randolphKappa\(\)](#)

percTable	<i>Percentage Table</i>
-----------	-------------------------

Description

Creates a 2-way contingency table along with percentages, marginal, and conditional distributions. All the frequencies are nested into one single table.

Usage

```
percTable(...)

## Default S3 method:
percTable(x, y = NULL, ...)

## S3 method for class 'formula'
percTable(formula, data, subset, na.action, ...)

## S3 method for class 'table'
percTable(
  x,
  freq = TRUE,
  prop = c("rows", "cols", "total"),
```

```

        expected = FALSE,
        ...
    )

## S3 method for class 'PercTable'
print(
  x,
  margins = NULL,
  col.vars = NULL,
  row.vars = NULL,
  justify = NULL,
  blockSep = NULL,
  ...
)

## S3 method for class 'matrix'
percTable(
  x,
  freq = TRUE,
  prop = c("rows", "cols", "total"),
  expected = FALSE,
  ...
)
```

Arguments

...	further arguments passed to <code>print.PercTable()</code>
x	a table, a matrix, or a vector to be tabulated
y	an optional second vector to be tabulated against x
formula	a formula of the form <code>lhs ~ rhs</code> where lhs will be tabled versus rhs (<code>table(lhs, rhs)</code>)
data	an optional matrix or data frame (or similar: see model.frame) containing the variables in the formula formula. By default the variables are taken from <code>environment(formula)</code> .
subset	an optional vector specifying a subset of observations to be used
na.action	a function which indicates what should happen when the data contain NAs. Defaults to <code>getOption("na.action")</code> .
freq	logical. Should absolute frequencies be included? Defaults to TRUE.
prop	character vector specifying the proportions to display, using "rows", "cols", "total", or "none"
expected	logical; whether to include expected counts under independence
margins	vector specifying the margins to include. Use 1 or "rows" for row margins, 2 or "cols" for column margins, or both; NULL includes none.
col.vars	a vector of column variables (see Details). If this is left to NULL the table structure will be preserved.
row.vars	a vector of row variables (see Details)
justify	either "left" or "right" for defining the alignment of the table cells
blockSep	logical, defining if an empty row should be introduced between the table rows. Default is FALSE, if only a table with one single description (either frequencies or percents) should be returned and TRUE in any other case.

Details

PercTable prints a 2-dimensional table. The absolute and relative frequencies are nested into one flat table by means of `fTable`. `row.vars`, resp. `col.vars` can be used to define the structure of the table. `row.vars` can either be the names of the dimensions (included percentages are named "idx") or numbers (1:3, where 1 is the first dimension of the table, 2 the second and 3 the percentages). Use `sortX()` if you want to have your table sorted by rows.

The style in which numbers are formatted is selected by `style()` from the DescToolsX options. Absolute frequencies will use `style("abs.sty")` and `style("per.sty")` will do it for the percentages. The options can be changed with `style(abs, digits=5)` which is basically a "style"-object containing any format information used in `fm`.

`margins` adds the marginal distributions. In the frequency table these are the usual row/column sums; in the percentage tables the margin holds the *marginal distribution*, i.e. the row resp. column sums of the frequency table divided by the grand total. A margin is only shown where it carries information: the sum column of the row percentages is 100% by construction and the sum row of the row percentages is not a distribution at all, so whichever of the two is uninformative is printed as ".".

Value

an object of class "PercTable" containing the requested frequency and percentage tables

References

Agresti, Alan (2007) *Introduction to categorical data analysis*. NY: John Wiley and Sons, Section 2.4.5

See Also

[table](#), [fTable](#), [proportions](#), [addmargins](#), [setDescToolsXOption](#), [pharos::style](#)

There are similar functions in [sfsmisc::printTable2](#) and package `vcd` [vcd::table2d_summary](#), both lacking some of the flexibility we needed here.

Other frequency: [expFreq\(\)](#), [freq\(\)](#), [freq2D\(\)](#), [tOne\(\)](#)

Examples

```
tab <- as.table(apply(HairEyeColor, c(1,2), sum))

percTable(tab, col.vars=2)

percTable(tab, col.vars=2, margins=c(1,2))
percTable(tab, col.vars=2, margins=2)
percTable(tab, col.vars=2, margins=1)
percTable(tab, col.vars=2, margins=NULL)

percTable(tab, col.vars=2, prop="none")

# just the percentages without absolute values
percTable(tab, col.vars=2, prop=c("total","rows"), freq=FALSE)

# just the row percentages
percTable(tab, freq= FALSE, prop="rows")
```

```
# just the expected frequencies
percTable(tab, prop="none", expected = TRUE)

# rearrange output such that freq are inserted as columns instead of rows
percTable(tab, col.vars=c(3,2))

# putting the areas in rows
percTable(tab, col.vars=c(3,1), prop="total", margins=c(1,2))

# formula interface with subset
percTable(driver ~ area, data=Pizza, subset=wine_delivered==0)

# sort the table by rows, order first column (Zurich), then third, then row.names (0)
percTable(sortX(tab, ord=c(1,3,0)))

# reverse the row variables, so that absolute frequencies and percents
# are not nested together
percTable(tab, row.vars=c(3, 1))

# the vector interface
percTable(x=Pizza$driver, y=Pizza$area)
percTable(x=Pizza$driver, y=Pizza$area, prop="rows",
margins=c("rows","cols"))
```

phi

Phi Coefficient

Description

Computes the Phi coefficient as a measure of association between two categorical variables.

Usage

```
phi(x, y = NULL, ...)
```

Arguments

x	a vector of categorical data or a contingency table (matrix or table)
y	optional second categorical vector. If provided, a contingency table is constructed from x and y.
...	additional arguments passed to table

Details

If y is provided, a contingency table is created using `table(x, y, ...)`. Otherwise, x is assumed to already be a contingency table.

Note: Yates' continuity correction is **not applied** when computing the chi-squared statistic.

The Phi coefficient is defined as:

$$\phi = \sqrt{\frac{\chi^2}{n}}$$

where χ^2 is the chi-squared test statistic and n is the total sample size.

This definition is **unsigned**. For a 2x2 table the signed coefficient $(n_{11}n_{22} - n_{12}n_{21}) / \sqrt{n_{1.}n_{2.}n_{.1}n_{.2}}$ equals the Pearson correlation of the two 0/1 indicators and lies in $[-1, 1]$; the value returned here is its absolute value, so the direction of the association is not reported. See [pearsonCor](#) if the sign is needed.

For contingency tables larger than 2x2, Phi is not bounded by 1 and may exceed 1. In such cases, [cramerV](#) is usually preferred.

Value

a numeric scalar containing the Phi coefficient

See Also

[chisq.test](#)

Other assoc.nominal: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
# Example with vectors
x <- c("A", "A", "B", "B")
y <- c("yes", "no", "yes", "no")
phi(x, y)

# Example with contingency table. Note that the signed coefficient is
# -0.0891 here: phi() reports the magnitude only.
tab <- matrix(c(10, 20, 30, 40), nrow = 2)
phi(tab)
```

print.BlandAltman	<i>Print a Bland-Altman Analysis</i>
-------------------	--------------------------------------

Description

Prints a compact summary of a Bland-Altman agreement analysis including the estimated bias, limits of agreement, corresponding confidence intervals, and the number of observations.

Usage

```
## S3 method for class 'BlandAltman'
print(x, digits = 3, ...)
```

Arguments

x	an object of class "BlandAltman" as returned by blandAltmanData
digits	number of decimal places to display
...	further arguments passed to or from other methods

Value

invisibly, x

print.Desc.Date	<i>Print method for "Desc.Date" objects</i>
-----------------	---

Description

Prints a structured summary of a "Desc.Date" object as created by [desc](#). The output includes core time-axis statistics such as range, span, coverage and fundamental quantiles.

Usage

```
## S3 method for class 'Desc.Date'
print(x, verbose = NULL, ...)
```

Arguments

x	an object of class "Desc.Date"
verbose	integer controlling the amount of printed detail. If NULL, the verbosity stored in the object metadata is used.
	0–1 core statistics only (range, span, coverage, quantiles, and sentinel detection)
	2 additionally prints weekday distribution with observed counts, expected counts, standardized residuals and chi-square p-value
	3 additionally prints month distribution
...	further arguments passed to underlying print methods

Details

Depending on the selected verbosity level, weekday and month distributions are shown including observed and expected counts, standardized residuals and chi-square p-values.

Standardized residuals are displayed together with a directional deviation marker using "+" and "-" symbols. These markers reflect the magnitude and direction of deviation from expectation but do not represent separate hypothesis tests.

Weekday and month distributions are compared to their expected probabilities as defined in [desc](#).

Standardized residuals are computed as

$$(Observed - Expected) / \sqrt{Expected}$$

. Larger absolute values indicate stronger deviations from the expected distribution.

Sentinel values (e.g., extreme future dates such as 3000-01-01 or implausibly early dates) are reported as potential data-quality issues.

`print.Desc.qq`*Describe a Contingency Table*

Description

Computes and displays a comprehensive set of descriptive statistics and association measures for a contingency table ($r \times c$ or 2×2). The function is also dispatched for `matrix` and cross-classified factor pairs via `Desc.qq` and `Desc.matrix`.

Usage

```
## S3 method for class 'Desc.qq'
print(x, digits = NULL, ...)

## S3 method for class 'Desc.qq'
plot(x, main = x$meta$main, which = 1, ...)

## S3 method for class 'table'
desc(
  x,
  conf.level = 0.95,
  prop = NULL,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  ...
)

## S3 method for class 'matrix'
desc(
  x,
  conf.level = 0.95,
  prop = NULL,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  ...
)

## S3 method for class 'array'
desc(
  x,
  conf.level = 0.95,
  prop = NULL,
  main = NULL,
  verbose = NULL,
  plotit = NULL,
  ...
)

## S3 method for class 'Desc.table'
```

```
print(x, print_header = TRUE, ...)
```

Arguments

<code>x</code>	a table or matrix object. For the formula interface, use <code>desc(y ~ x, data)</code> which dispatches to this function automatically.
<code>digits</code>	number of digits for numerical output
<code>...</code>	further arguments passed to or from other methods
<code>main</code>	main title for the plot
<code>which</code>	plots to produce
<code>conf.level</code>	numeric, confidence level for all confidence intervals. Default is 0.95.
<code>prop</code>	character string controlling which proportions are shown in the cross-tabulation. One of "rows" (default), "cols", "total", or "no" (frequencies only). At <code>verbose = 3</code> all three proportions are shown regardless of this argument.
<code>verbose</code>	integer controlling the amount of output (1, 2, or 3). NULL (default) falls back to <code>getOption("DescTools.verbose", 2)</code> . If set explicitly in the function call, that value takes priority over the global option. See Details for what each level produces.
<code>plotit</code>	whether a plot is produced automatically
<code>print_header</code>	whether the header is printed

Details

The `verbose` argument controls which statistics are computed and displayed. The following table gives an overview; items marked with *2x2* are only shown for 2 x 2 tables.

verbose = 1 — essential output:

- Summary: n, rows, columns, missings
- Cross-tabulation: frequencies
- Pearson chi-squared test
- Chi-squared with Yates continuity correction (*2x2*)
- Fisher's exact test (*2x2*)
- McNemar's test (*2x2*)
- Cramér's V with confidence interval and effect size label
- Odds ratio with confidence interval (*2x2*)

verbose = 2 — standard output (default):

All of the above, plus:

- Cross-tabulation: row proportions (or as set by `prop`)
- G-test (log likelihood ratio test of independence)
- Mantel-Haenszel chi-squared test
- Contingency coefficient
- Kendall's tau-b with confidence interval
- Relative risk col1/col2 and row1/row2 with confidence intervals (*2x2*)
- Proportions difference with confidence interval (*2x2*)

verbose = 3 — full output:

All of the above, plus:

- Cross-tabulation: row, column, and total proportions
- Lambda C_{IR}, R_{IC}, symmetric
- Uncertainty coefficient C_{IR}, R_{IC}, symmetric
- Mutual information
- Goodman-Kruskal gamma with confidence interval
- Stuart's tau-c with confidence interval
- Somers' D C_{IR} and R_{IC} with confidence intervals
- Pearson and Spearman correlation with confidence intervals

Table types:

For **r x c tables** (arbitrary number of rows and columns) all nominal and ordinal association measures listed above are available. For **2 x 2 tables** the output additionally includes tests and measures specific to the 2 x 2 case (Fisher's exact, McNemar, odds ratio, relative risk, proportions difference).

Dispatching:

`desc.matrix` and `desc.qq` both redirect to `desc.table`. When called via the formula interface `desc(y ~ x, data)`, the type of `y` and `x` is known and ordinal-specific measures (tau-b and above) are activated automatically when both variables are ordered factors.

Value

an object of class `c("Desc.table", "Desc")`. The object is a list containing all computed statistics and is intended to be used via its `print` and `plot` methods.

See Also

[desc](#) for the generic function and formula interface, [desc.numeric](#) for univariate numeric descriptions, [desc.factor](#) for univariate factor descriptions, [pharos::plot.Desc.table](#) for different plotting options, [stats::chisq.test](#), [stats::fisher.test](#), [cramerV](#), [oddsRatio](#)

Other desc: [desc\(\)](#), [desc.Date\(\)](#), [desc.factor\(\)](#), [desc.nn](#), [desc.nq](#), [desc.numeric\(\)](#), [desc.qn](#), [desc.qq](#), [desc.ts\(\)](#)

Examples

```
# from an existing table
tab <- table(Pizza$driver, Pizza$area)
desc(tab)
desc(tab, prop = "rows", verbose = 3)

# 2x2 table - additional measures are shown automatically
tab2 <- tab[1:2, 1:2]
desc(tab2)

# formula interface - dispatches to desc.table internally
desc(driver ~ area, data = Pizza)

# from a matrix
m <- matrix(c(153, 153, 167, 123, 108, 109, 89, 122, 167),
            nrow = 3, byrow = TRUE,
```

```

dimnames = list(c("Brent", "Camden", "Westminster"),
                c("Allanah", "Maria", "Rhonda"))
desc(m, verbose = 2)

```

quantileX	<i>(Weighted) Sample Quantiles</i>
-----------	------------------------------------

Description

Compute sample quantiles, with optional weights.

Usage

```

quantileX(
  x,
  weights = NULL,
  probs = seq(0, 1, 0.25),
  na.rm = FALSE,
  names = TRUE,
  type = 7,
  digits = 7
)

```

Arguments

<code>x</code>	a numeric vector
<code>weights</code>	an optional numeric vector giving the sample weights
<code>probs</code>	numeric vector of probabilities with values in $[0, 1]$
<code>na.rm</code>	a logical indicating whether missing values in <code>x</code> should be omitted
<code>names</code>	logical; if true, the result has a <code>names</code> attribute. Set to FALSE for speedup with many probs.
<code>type</code>	an integer between 1 and 9 selecting one of the nine quantile algorithms of quantile . All nine are available for unweighted data. With weights only 5 and 7 (default) exist; any other value is an error. See Details for how the two differ in their reading of the weights.
<code>digits</code>	used only when <code>names</code> is true: the precision to use when formatting the percentages. In R versions up to 4.0.x, this had been set to <code>max(2, getOption("digits"))</code> , internally.

Details

Without weights the call is handed to [quantile](#) unchanged, so all nine types are available and the results are identical to base R.

With weights only types 5 and 7 exist, and they interpret the weights **differently**:

`type = 5` treats them as relative weights: only the ratios matter, and multiplying every weight by a constant leaves the result unchanged. This follows the Eurostat definition (EU-SILC 131-rev/04).

`type = 7` treats them as *frequency* weights, i.e. as replication counts. The effective sample size is `sum(weights)`, so the result is **not** scale-invariant, and weights that have been normalized to sum to 1 are degenerate - see below.

This difference is inherited from the two source implementations and is not a free choice of the caller: it is worth knowing which of the two is meant before picking a type. Because `type = 7` needs `sum(weights)` to act as a sample size, it requires that sum to be at least 2 and raises an error otherwise.

Value

a numeric vector containing the weighted quantiles of `x` at probabilities `probs`, named when `names = TRUE`

Note

Based on code by Andreas Alfons, Matthias Templ, adapted to conform to package standards.

References

Working group on Statistics on Income and Living Conditions (2004) Common cross-sectional EU indicators based on EU-SILC; the gender pay gap. *EU-SILC 131-rev/04*, Eurostat.

See Also

[medianX](#), [quantile](#), [quantileCI](#)

Other quantile: [extremes](#)

Examples

```
# Pizza$temperature contains missing values, so na.rm is needed - without
# it the function returns NA for every prob, silently.
quantileX(Pizza$temperature, rep(c(1:3), length.out = nrow(Pizza)),
          na.rm = TRUE)

x <- c(3.7, 3.3, 3.5, 2.8)

# type 5 only looks at the ratios of the weights ...
quantileX(x, weights = c(5, 5, 4, 1), type = 5)
quantileX(x, weights = c(5, 5, 4, 1) / 15, type = 5) # identical

# ... while type 7 reads them as replication counts, so they have to be
# on that scale
quantileX(x, weights = c(5, 5, 4, 1), type = 7)
```

randolphKappa	<i>Randolph's Free-Marginal Multirater Kappa</i>
---------------	--

Description

Computes Randolph's free-marginal multirater kappa for m raters over N subjects. This agreement coefficient does not assume fixed marginal distributions (i.e., it is free-marginal).

Usage

```
randolphKappa(x, categories = NULL, conf.level = NA, ...)
```

Arguments

<code>x</code>	a matrix of size $N \times m$ with subjects in rows and raters in columns; cells contain the assigned categories
<code>categories</code>	the categories a rater could have chosen from, or a single number giving how many there were. Defaults to NULL, i.e. the categories actually observed in <code>x</code> . See Details.
<code>conf.level</code>	reserved for future confidence intervals; must be NA
<code>...</code>	reserved for future bootstrap options and currently ignored

Details

Let k be the number of categories, m the number of raters, and N the number of subjects. Randolph's kappa is

$$\kappa = \frac{P_o - 1/k}{1 - 1/k},$$

where the observed agreement P_o is the proportion of agreeing rater *pairs*,

$$P_o = \frac{1}{N} \sum_{i=1}^N \frac{\sum_j n_{ij}(n_{ij} - 1)}{m(m - 1)}.$$

Here, n_{ij} denotes the number of raters who assigned subject i to category j . This is the same observed agreement as in Fleiss' kappa (and as computed by [percAgreement](#)); Randolph's coefficient differs from Fleiss' only in the chance agreement P_e , which is fixed at $1/k$ instead of being estimated from the marginals.

$P_e = 1/k$ refers to the categories a rater could have *chosen*, not to those that happen to occur in the data. If a category was available but never used, the default `categories = NULL` understates k and thus overstates chance agreement, which biases κ downwards. Supply `categories` whenever the coding scheme is known.

Long-format ratings can first be reshaped with [raterFrame](#).

Value

a numeric scalar containing Randolph's kappa

References

Randolph, J. J. (2005). Free-Marginal Multirater Kappa (multirater κ_{free}): An Alternative to Fleiss' Fixed-Marginal Multirater Kappa. Online submission.

See Also

Other assoc.agreement: [ccc\(\)](#), [cohenKappa\(\)](#), [cronbachAlpha\(\)](#), [icc\(\)](#), [kappaM\(\)](#), [krippAlpha\(\)](#), [pabak\(\)](#), [percAgreement\(\)](#)

Examples

```
## Matrix (subjects x raters), 5 subjects, 3 raters
x <- matrix(c(
  1,1,1,
  2,2,2,
  1,2,1,
  3,3,3,
  2,2,1
), ncol = 3, byrow = TRUE)
randolphKappa(x)

# the raters could have chosen from five categories, not just the three
# they used
randolphKappa(x, categories = 5)

## Long format with a formula
df <- data.frame(
  subject = rep(1:5, each = 3),
  rater   = rep(paste0("r", 1:3), times = 5),
  rating  = c(1,1,1, 2,2,2, 1,2,1, 3,3,3, 2,2,1)
)
randolphKappa(raterFrame(rating ~ subject | rater,
                        data = df, dropSubj=TRUE))
```

rangeX	(Robust) Range
--------	----------------

Description

Determines the range of the data, which can possibly be trimmed before calculating the extreme values. The robust range version is calculated on the basis of the trimmed mean and variance (see Details).

Usage

```
rangeX(x, trim = NULL, robust = FALSE, na.rm = FALSE, ...)
```

Arguments

x	a numeric vector
trim	the fraction (0 to 0.5) of observations to be trimmed from each end of x before the range is computed. Values of trim outside that range are taken as the nearest endpoint. Default is 0 for robust = FALSE and 0.2 for robust = TRUE.
robust	logical; whether to return the robust or conventional range
na.rm	a logical value indicating whether NA values should be stripped before the computation proceeds

... further arguments passed to `.robRange`, including `fac`. Only used if `robust = TRUE`.

Details

The R base function `range` returns the minimum and maximum value of a numeric object. Here we return the span of a (possibly trimmed) numeric vector, say the difference of maximum and minimum value.

If `robust` is set to `TRUE` the function determines the trimmed mean `m` and then the "upper trimmed mean" `s` of absolute deviations from `m`, multiplied by `fac` (`fac` is 3 by default). The robust minimum is then defined as `m-fac*s` or `min(x)`, whichever is larger, and similarly for the maximum.

Value

a numeric scalar containing the range width. The corresponding lower and upper bounds are returned in the "bounds" attribute. Note that the attribute is dropped by subsetting and by [as.vector](#), so read it before computing on the result.

Note

Robust range contributed by Werner Stahel.

See Also

[range](#), [min](#), [max](#)

Other dispersion: [coefVar\(\)](#), [iqrX\(\)](#), [madX\(\)](#), [meanAD\(\)](#), [meanSE\(\)](#), [varX\(\)](#)

Examples

```
x <- c(0:10, 50)
xm <- rangeX(x)
c(xm, rangeX(x, trim = 0.10))

x <- c(rnorm(20), rnorm(3, 5, 20))
rangeX(x, robust=TRUE)

# compared to
rangeX(x)
```

raterFrame

Create a Data.frame for Interrater Agreement

Description

Creates a `data.frame` for a formula `rating ~ subjects | raters` with the subjects in rows and the raters in columns as base structure for interrater agreement (IRA) functions.

Usage

```
raterFrame(formula, data, subset, na.action, dropSubj = FALSE)
```

Arguments

formula	something like rating ~ subjects raters
data	the data
subset	potential subset, evaluated in the long-format data
na.action	what should happen with missings, applied per subject (i.e. to the rows of the wide result)
dropSubj	logical; whether to drop the subject column (default FALSE)

Details

Assessments made by raters are typically - and appropriately - stored and organized in databases. Data originating from databases are usually in long format. Converting this long format into a wide format suitable for analysis is a complex task, as it is not just a matter of simply displaying consecutive values in a new column (as in usual "from-long-to-wide" operations), but of assigning the values to the correct subjects, which requires a sequential join.

The present function supports this process by converting long-format data into a wide format that can be used by subsequent inter-rater agreement functions. Missing values are marked as NAs.

na.action is applied to the *wide* frame, i.e. per subject: na.omit removes subjects with at least one missing rating. The "na.action" attribute of the result carries a "values" attribute with the identifiers of the omitted subjects; it is absent if nothing was omitted.

Value

a data.frame of class "raterFrame" with subjects in rows and raters in columns. The name of the subject column is kept in the "subject" attribute (NA if it was dropped).

See Also

[bedrock::resolveFormula](#)

Other agreement: [blandAltmanData\(\)](#), [normalizeToConfusion\(\)](#)

Examples

```
d.long <- data.frame(
  expand.grid(subj=as.character(1:5), rater=LETTERS[1:3]),
  rating = c(1, 4, 5, 7, 2, 2, 5, 6, 7, 1, 1, 4, 6, 6, 2))

# default rater frame
raterFrame(rating ~ subj | rater, data=d.long)

# introduce some NAs
raterFrame(rating ~ subj | rater, data=d.long[-c(3, 6), ])

# omit cases containing NAs
raterFrame(rating ~ subj | rater, data=d.long[-c(3, 6), ],
  na.action=na.omit)

# omit the subject column
raterFrame(rating ~ subj | rater, data=d.long, dropSubj=TRUE)
```

relRisk

*Relative Risk***Description**

Computes the relative risk for a 2x2 contingency table together with optional confidence intervals.

Usage

```
relRisk(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("score", "wald", "use-or"),
  delta = 0.5,
  ...
)
```

Arguments

<code>x</code>	a numeric 2x2 matrix or table containing non-negative counts
<code>y</code>	an optional vector. If supplied, <code>table(x, y, ...)</code> is computed.
<code>conf.level</code>	confidence level for the interval estimate. If NA (default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See details in ConfidenceIntervals .
<code>method</code>	character string specifying the confidence interval method. One of "score", "wald", or "use-or".
<code>delta</code>	small continuity correction added to the event counts in the <i>standard error</i> of the Wald interval. Only used if <code>method = "wald"</code> ; see the note below.
<code>...</code>	further arguments passed to table

Details

The relative risk compares the event probability in the exposed group with the event probability in the unexposed group.

The function expects the exposure groups in the rows and the outcome in the columns:

	outcome = 1	outcome = 0
exposed = 1	x1	n1 - x1
exposed = 0	x2	n2 - x2

The relative risk is defined as:

$$RR = \frac{x_1/n_1}{x_2/n_2}$$

Confidence intervals can be calculated using the score method of Koopman (1984), a Wald approximation, or via transformation of the odds ratio.

The score interval is based on the method of Koopman (1984) and Miettinen and Nurminen (1985). It is obtained from the closed-form solution of the cubic equation in the constrained maximum likelihood estimate; if the unexposed group has no non-events ($x_2 == n_2$) that cubic has a root on the parameter boundary and its roots can no longer be assigned to the two interval bounds by their order alone. This case is therefore solved directly from the score statistic ([uniroot](#)); both routes agree to numerical precision wherever the closed form applies.

The Wald interval is asymptotic and may perform poorly for small counts or extreme probabilities. Note that `delta` enters the standard error only, not the point estimate the interval is centred on: with a zero cell the point estimate is 0 or Inf and the Wald interval degenerates accordingly. Use `method = "score"` for tables with zero cells.

If the table orientation differs from the required structure, rows or columns can be reversed using [revX](#) or transposed with [t](#).

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` relative risk estimate

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Koopman, P. A. R. (1984). Confidence intervals for the ratio of two binomial proportions. *Biometrics*, 40(2), 513–517.

Miettinen, O., & Nurminen, M. (1985). Comparative analysis of two rates. *Statistics in Medicine*, 4(2), 213–226.

Rothman, K. J., Greenland, S., & Lash, T. L. (2008). *Modern Epidemiology* (3rd ed.). Lippincott Williams & Wilkins.

See Also

Other effect.size: [cohenD\(\)](#), [cohenH\(\)](#), [etaSq\(\)](#), [glassDelta\(\)](#), [oddsRatio\(\)](#)

Examples

```
m <- matrix(
  c(78, 50,
    1422, 950),
  nrow = 2,
  dimnames = list(
    water = c("cont", "clean"),
    diarrhea = c("yes", "no")
  )
)

relRisk(m, conf.level = 0.95)

mm <- cbind(c(9, 20), c(41, 29))

relRisk(t(mm), conf.level = 0.95)
```

```

relRisk(
  t(mm),
  conf.level = 0.95,
  method = "wald"
)

relRisk(
  t(mm),
  conf.level = 0.95,
  method = "use-or"
)

# unexposed group without non-events: the score interval is still valid
relRisk(matrix(c(2, 5, 3, 0), nrow = 2), conf.level = 0.95)

```

rmse	<i>Root Mean Squared Error</i>
------	--------------------------------

Description

Computes the root mean squared error (RMSE) between predicted and reference values.

Usage

```

rmse(x, ...)

## S3 method for class 'lm'
rmse(x, ...)

## Default S3 method:
rmse(x, ref, na.rm = FALSE, ...)

```

Arguments

<code>x</code>	an object. Methods are available for numeric vectors and model objects (e.g. <code>lm</code>).
<code>...</code>	additional arguments passed to methods
<code>ref</code>	numeric vector of reference (true) values
<code>na.rm</code>	logical; whether to remove missing values

Details

The RMSE is defined as:

$$\sqrt{\frac{1}{n} \sum (ref - x)^2}$$

Value

a numeric scalar containing the RMSE

See Also[mse](#), [mae](#)Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [smape\(\)](#)**Examples**

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)

rmse(x, ref)

# with linear model
fit <- lm(mpg ~ hp, data = mtcars)
rmse(fit)
```

rosenbluth

*Rosenbluth Index***Description**

Computes the Rosenbluth index as a measure of concentration.

Usage

```
rosenbluth(x, n = rep(1, length(x)), na.rm = FALSE)
```

Arguments

x numeric vector of non-negative values, such as market shares or frequencies

n optional frequency weights. Each element of **x** is replicated **n** times.

na.rm logical. If TRUE, missing values are removed.

Details

The Rosenbluth index is based on the ranked shares and is directly related to market concentration. Larger values indicate stronger concentration.

With the shares $p_i = x_i / \sum x$ sorted in decreasing order and i their rank, the index is

$$HT = 1 / (2 \sum i p_i - 1).$$

It ranges from $1/k$ for k units of equal size to 1 for a single unit holding everything, so it is read on the same scale as the Herfindahl index rather than as an inequality measure.

If negative values or missing values (when `na.rm = FALSE`) are present, NA is returned. The index is undefined when all values are zero, and NA is returned in that case as well.

Value

a numeric scalar containing the Rosenbluth index

References

- Rosenbluth, G. (1955). Measures of concentration. In: *Business Concentration and Price Policy*. Princeton University Press, 57-99.
- Hall, M., Tideman, N. (1967). Measures of concentration. *Journal of the American Statistical Association*, 62, 162-168.

See Also

Other inequality: [atkinson\(\)](#), [divCoef\(\)](#), [gini\(\)](#), [lc\(\)](#), [theil\(\)](#)

Examples

```
# four units of equal size: the index takes its minimum 1/4
rosenbluth(c(1, 1, 1, 1))

# one unit holding everything: the maximum 1
rosenbluth(c(1, 0, 0, 0))

# a dominant unit next to three small ones
rosenbluth(c(10, 1, 1, 1))

# frequency weights replicate the values
rosenbluth(c(10, 1), n = c(1, 3))
```

scaleX	(Robust) Scaling and Centering
--------	--------------------------------

Description

Centers and scales the columns of a numeric matrix. The conventional version uses mean and standard deviation, the robust one median and MAD (see Details).

Usage

```
scaleX(x, center = TRUE, scale = TRUE, robust = FALSE, na.rm = TRUE)
```

Arguments

x	a numeric matrix-like object
center	logical scalar or numeric vector. If TRUE, the column means (or medians, for robust = TRUE) are subtracted; if FALSE, no centering is performed. Alternatively, a numeric vector of length ncol(x) supplies the values to subtract directly.
scale	logical scalar or numeric vector. If TRUE, the columns are divided by their standard deviation (or MAD, for robust = TRUE); if FALSE, no scaling is performed. Alternatively, a numeric vector of length ncol(x) supplies the divisors directly.
robust	logical; whether to standardize by median and MAD rather than by mean and standard deviation
na.rm	logical; if TRUE (default), missing values are omitted when the column centers and scales are computed. Ignored for whichever of center and scale is given as a numeric vector. Missing entries of x itself always remain missing in the result.

Details

The R base function `scale` centers each column by its mean and divides by the root mean square of the centered column, which for centered data is the standard deviation. Both are sensitive to outliers: a single extreme value moves the mean and inflates the standard deviation, so the remaining observations are compressed towards zero.

If `robust` is set to `TRUE` the column median takes the place of the mean and the median absolute deviation (`mad`) that of the standard deviation. Both have a breakdown point of 50 percent, so the standardization reflects the bulk of the data rather than its extremes, and genuine outliers keep large scores instead of being pulled in.

Whichever is chosen, `center` and `scale` accept either a logical flag or a numeric vector of values to use directly, in which case that vector must have one entry per column of `x`, as in `scale`.

The two versions differ in one further respect. The MAD is invariant to location shifts, so for `robust = TRUE` the returned `"scaled:scale"` is the same whether or not the columns were centered first, and does not depend on `center`. The root mean square is not invariant; for `robust = FALSE` it is computed after centering, matching `scale`, which is what makes it equal the standard deviation when `center` is `TRUE` and not otherwise.

A zero or non-finite scaling factor can produce undefined or non-finite results. `scaleX` emits a warning naming the affected columns rather than failing, since the result may still be usable when those columns are subsequently dropped.

Value

the centered and scaled matrix. The numeric centerings and scalings used (if any) are returned as attributes `"scaled:center"` and `"scaled:scale"`.

See Also

`scale`, `sweep`, `mad`, `rangeX`

Other transform: `boxCox()`, `boxCoxLambda()`, `logSt()`, `yeoJohnson()`

Examples

```
x <- bedrock::Pizza$temperature

# robust standardization is far less affected by the extreme values
plot(scaleX(x, robust = TRUE), col = "black", pch = 16, cex = 0.4,
     ylab = "standardized temperature")
points(scaleX(x), col = "red", pch = 16, cex = 0.4)
legend("topright", legend = c("robust (median/MAD)", "conventional"),
     col = c("black", "red"), pch = 16, bty = "n")

# the centerings and scalings used are recoverable
z <- scaleX(cbind(a = c(1, 2, 3, 4, 100), b = c(10, 20, 30, 40, 50)))
attr(z, "scaled:center")
attr(z, "scaled:scale")

# compared to the robust version, which the extreme value barely moves
attr(scaleX(cbind(a = c(1, 2, 3, 4, 100)), robust = TRUE), "scaled:scale")

# supplying the values directly, as base::scale allows
scaleX(matrix(1:6, ncol = 2), center = c(0, 0), scale = c(1, 2))
```

setDescToolsXOption	<i>Set DescToolsX Options</i>
---------------------	-------------------------------

Description

Set one or more package options. Each supplied name is stored with the "DescToolsX." prefix and can be read with [getOption\(\)](#).

Usage

```
setDescToolsXOption(...)
```

Arguments

... named option values

Details

Options used by descriptive methods include:

`plotit` whether descriptive functions create plots by default

`footnote` characters used as footnote markers

`lang` language used for month and weekday names

Value

a named list containing the previous values, invisibly

See Also

[options](#), [getOption](#)

Examples

```
old <- getOption("DescToolsX.plotit")
setDescToolsXOption(plotit = FALSE)
getOption("DescToolsX.plotit")
options(DescToolsX.plotit = old)
```

simpson

*Simpson Diversity Indices***Description**

Computes Simpson-type diversity indices, including the Gini-Simpson index, the Hunter-Gaston index (bias-corrected for sample size), and the Deltas-corrected Gini-Simpson index (bias-corrected for number of categories).

Usage

```
simpson(x, method = c("gini", "hunter", "deltas"), na.rm = FALSE)
```

Arguments

<code>x</code>	a factor or character vector of observations, or a vector of non-negative counts. Note that a <i>numeric</i> vector is always read as counts, never as observations; wrap it in <code>factor</code> to have it tabulated instead.
<code>method</code>	character string specifying the index to compute: "gini", "hunter", or "deltas"
<code>na.rm</code>	logical. If TRUE, missing values are removed before computation. If FALSE and <code>x</code> contains NA, the result will be NA_real_.

Details

The Gini-Simpson index is defined as $1 - \sum p_i^2$, where p_i are the relative frequencies of categories.

The Hunter-Gaston index is defined as $1 - \sum n_i(n_i - 1)/(N(N - 1))$, where n_i are counts and N is the total sample size.

The Deltas correction is defined as $(1 - \sum p_i^2) * k/(k - 1)$, where k is the number of observed (non-empty) categories. The factor is the reciprocal of $(k - 1)/k$, the largest value the Gini-Simpson index can attain with k categories, so the corrected index reaches 1 for a uniform distribution over the observed categories.

All indices represent the probability that two randomly selected observations belong to different categories.

The Hunter-Gaston index corrects for finite sample size, while the Deltas correction adjusts for a small number of observed categories. Note that the finite-sample correction $N/(N - 1)$ applied to the Gini-Simpson index reproduces the Hunter-Gaston index exactly; the two are the same adjustment with N and k in the correction factor.

A sample concentrated in a single category is perfectly homogeneous rather than undefined, so "gini" and "hunter" return 0 for it. Only the Deltas correction requires $k \geq 2$, since $k - 1$ appears in its denominator.

When `x` is numeric, it is treated as a vector of counts. Non-integer values produce a warning; the Hunter-Gaston index requires integer counts.

Value

a numeric scalar between 0 and 1. Returns NA_real_ if input is invalid or empty.

References

- Sachs, L. (1997). *Angewandte Statistik*. Springer.
- Hunter, P. R., & Gaston, M. A. (1988). Numerical index of the discriminatory ability of typing systems. *Journal of Clinical Microbiology*, 26(11), 2465-2466. <https://doi.org/10.1128/jcm.26.11.2465-2466.1988>
- Deltas, G. (2003). The small-sample bias of the Gini coefficient: Results and implications for empirical research. *Review of Economics and Statistics*, 85(1), 226-234. <https://doi.org/10.1162/rest.2003.85.1.226>

See Also

Other diversity.concentration: [entropy\(\)](#), [herfindahl\(\)](#)

Examples

```
x <- c("A", "A", "B", "C", "C", "C")

simpson(x, method = "gini")
simpson(x, method = "hunter")
simpson(x, method = "deltas")

# Using counts directly
counts <- c(A = 2, B = 1, C = 3)
simpson(counts, method = "hunter")

# a numeric vector of observations must be tabulated first, otherwise
# its values are read as counts
simpson(factor(c(1, 1, 2, 2, 3)), method = "gini")

# With missing values
x <- c("A", "A", NA, "B")
simpson(x, method = "gini", na.rm = TRUE)
```

skew	<i>Skewness</i>
------	-----------------

Description

skew computes the skewness, kurt the excess kurtosis of the values in x.

Usage

```
skew(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  method = c("boot", "classic"),
  estimator = 3,
  weights = NULL,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a numeric vector
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>method</code>	character string specifying the confidence interval method. "boot" (default) uses a nonparametric bootstrap, with BCa intervals unless another bootstrap type is supplied through ...; "classic" uses a Wald interval based on the asymptotic standard error. See Details and ConfidenceIntervals .
<code>estimator</code>	integer, either 1, 2 or 3 (default) defining the algorithm used for calculation. See Details.
<code>weights</code>	a numerical vector of weights the same length as <code>x</code> giving the weights to use for elements of <code>x</code> . The weights are read as frequencies, so that their sum takes the place of the sample size in the estimator's bias corrections and in the standard error.
<code>na.rm</code>	logical, indicating whether NA values should be stripped before the computation proceeds. Defaults to FALSE.
...	further arguments passed to boot when confidence intervals are calculated

Details

If `na.rm` is TRUE then missing values are removed before computation proceeds.

The estimator for calculating the skewness can either be:

```
1: g_1 = m_3 / m_2^(3/2)
2: G_1 = g_1 * sqrt(n(n-1)) / (n-2)
3: b_1 = m_3 / s^3 = g_1 ((n-1)/n)^(3/2)
```

1 is the typical definition used in Stata and in many older textbooks.

2 is used in SAS and SPSS.

3 is used in MINITAB and BMDP.

Cramer (1997) mentions the asymptotic standard error of G_1 , that is of estimator = 2:

$$ASE.skew = \sqrt{6 \cdot n \cdot (n-1) / ((n-2) \cdot (n+1) \cdot (n+3))}$$

to be used for calculating the confidence intervals. The standard errors of the other two estimators follow from it by the same factors that relate the estimators themselves, so that `est/se` does not depend on the choice of estimator. This is implemented here with `method="classic"`.

However, Joanes and Gill (1998) advise against this approach, pointing out that the normal assumptions would virtually always be violated. They suggest using the bootstrap method. That's why the default method for the confidence interval type is set to "boot". If not further specified the boot ci type will be chosen as "bca".

The standard error is only defined for $n \geq 3$; for shorter input the variance, and with it any `method="classic"` interval, is NA.

This implementation of the two functions is comparably fast, as the expensive sums are coded in C++.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` skewness estimate

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Cramer, D. (1997): *Basic Statistics for Social Research* Routledge.

Joanes, D. N., Gill, C. A. (1998): Comparing measures of sample skewness and kurtosis. *The Statistician*, 47, 183-189.

See Also

[meanX](#), [sdX](#), similar code in **e1071**

Other shape: [kurt\(\)](#)

Examples

```
skew(bedrock::Pizza$price, na.rm=TRUE)

# use sapply to calculate skewness for a data.frame
sapply(bedrock::Pizza[,c("temperature", "price", "delivery_min")],
       skew, na.rm=TRUE)

# the estimate lies inside its own confidence interval
set.seed(1)
skew(rlnorm(50), conf.level=0.95, method="classic")
```

smape

Symmetric Mean Absolute Percentage Error

Description

Computes the symmetric mean absolute percentage error (SMAPE) between predicted and reference values.

Usage

```
smape(x, ...)

## S3 method for class 'lm'
smape(x, ...)

## Default S3 method:
smape(x, ref, na.rm = FALSE, ...)
```

Arguments

<code>x</code>	an object. Methods are available for numeric vectors and model objects (e.g. <code>lm</code>).
<code>...</code>	additional arguments passed to methods
<code>ref</code>	numeric vector of reference (true) values
<code>na.rm</code>	logical; whether to remove missing and undefined terms

Details

The SMAPE is defined as:

$$\frac{1}{n} \sum \frac{2|ref - x|}{|x| + |ref|}$$

Values are bounded between 0 and 2. Note that this is a ratio, not a percentage: the factor 100 of the original definition is not applied, which is the form for which the bound of 2 holds.

A term is undefined when `x` and `ref` are both zero, and is set to NA. With the default `na.rm = FALSE` a single such pair therefore makes the whole result NA; with `na.rm = TRUE` those terms are dropped along with genuinely missing ones, so the mean is taken over fewer than `length(x)` terms.

Value

a numeric scalar containing the SMAPE

See Also

Other model.metrics: [auc\(\)](#), [brierScore\(\)](#), [mae\(\)](#), [mape\(\)](#), [mse\(\)](#), [nmae\(\)](#), [nmse\(\)](#), [rmse\(\)](#)

Examples

```
x <- c(2.5, 3.0, 2.8)
ref <- c(3.0, 2.5, 3.0)

smape(x, ref)

# with linear model
fit <- lm(mpg ~ hp, data = mtcars)
smape(fit)
```

spearmanCor

Spearman Rank Correlation

Description

Calculate Spearman correlation coefficient and its confidence interval. In addition to the base R function `cor(x, method="spearman")`, frequency tables are also accepted as arguments (i.e. actually weights are used).

Usage

```
spearmanCor(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  na.rm = FALSE
)
```

Arguments

<code>x</code>	either a contingency table, a two-column object (matrix, data.frame or list), or a vector of observations (together with <code>y</code>)
<code>y</code>	optional second vector. If <code>x</code> is not a vector, <code>y</code> must be <code>NULL</code> .
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>na.rm</code>	logical; whether to remove incomplete pairs. Applies to the vector interface; a frequency table must not contain missing counts.

Details

The function calculates Spearman's rho statistic by means of `cor(..., method="spearman")` when two variables `x` and `y` are supplied. If a frequency table is provided an implementation based on SAS documentation is used. Both routes use midranks for ties and agree exactly; see the examples. The confidence intervals are calculated via z-Transformation.

The number of observations entering the z-transformation is `length(x)` in the vector interface and `sum(x)`, the table total, in the table interface. At least four observations are needed for an interval. `sides` names the side of the interval that carries the finite bound, so "left" returns `[lci, 1]` and "right" returns `[-1, uci]`. Since rho is bounded, the open side is reported at the range boundary rather than as infinite.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of Spearman's rank correlation
`lci` lower confidence interval bound
`uci` upper confidence interval bound

References

Conover W. J. (1999) *Practical Nonparametric Statistics (3rd edition)*. Wiley

See Also

[Association](#)

Other assoc.continuous: [corPart\(\)](#), [corPolychor\(\)](#), [findCorrX\(\)](#), [hoeffdingD\(\)](#), [keepSig\(\)](#), [pearsonCor\(\)](#)

Examples

```
# Example from SAS documentation (PROC FREQ)
pain <- as.table(matrix(c(26, 6, 26, 7, 23,
                        9, 18, 14, 9, 23),
                      ncol=5,
                      dimnames=list(adverse=c("no", "yes"), dose=1:5)))

spearmanCor(pain)

spearmanCor(pain, conf.level=0.95)

# must be the same as
with(lapply(
  bedrock::untable(pain,
                   colnames = c("adverse", "dose")),
  ordered),
  spearmanCor(adverse, dose, conf.level=0.95))
```

theil	<i>Theil Index</i>
-------	--------------------

Description

Computes the Theil inequality index (Theil T).

Usage

```
theil(x, n = rep(1, length(x)), na.rm = FALSE)
```

Arguments

- x numeric vector of non-negative values, such as incomes
- n optional frequency weights. Each element of x is replicated n times. Must be a vector of non-negative integers of the same length as x.
- na.rm logical. If TRUE, missing values are removed.

Details

The Theil index is an entropy-based measure of inequality. It belongs to the class of Generalized Entropy measures with parameter $\alpha = 1$.
The Theil T index is defined as

$$T = \frac{1}{n} \sum \frac{x_i}{\bar{x}} \log \left(\frac{x_i}{\bar{x}} \right)$$

where $\bar{x}$ is the mean of x and n the number of (replicated) observations.
Zero values are admissible: following the usual convention $x \log x \rightarrow 0$ as $x \rightarrow 0$, they contribute 0 to the sum.
The index is decomposable into within- and between-group components, which makes it particularly useful in applied inequality analysis.
If negative values or missing values (when `na.rm = FALSE`) are present, NA is returned. The same holds if no observation remains after removing missing values.

Value

a numeric scalar containing the Theil index. The value is 0 under perfect equality and increases with inequality, up to a maximum of $\log(n)$, attained when a single unit holds the entire total.

References

Theil, H. (1967). Economics and Information Theory.

See Also

Other inequality: [atkinson\(\)](#), [divCoef\(\)](#), [gini\(\)](#), [lc\(\)](#), [rosenbluth\(\)](#)

Examples

```
theil(c(10, 10, 10, 10))      # perfect equality: 0
theil(c(0, 0, 0, 40))        # everything in one hand: log(4)
theil(c(1, 2, 3, 4, 5))

# frequency weights replicate the observations
theil(1:3, n = c(1, 2, 3))
theil(rep(1:3, times = c(1, 2, 3)))
```

time-conversions	<i>Convert h:m:s To/From seconds</i>
------------------	--------------------------------------

Description

`hmsToSec` - Converts a vector of h:m:s to seconds.

Usage

```
hmsToMinute(x)

hmsToSec(x)

secToHms(x, digits = NULL)
```

Arguments

<code>x</code>	date-time object for <code>hmsToMinute()</code> , vector of times in h:m:s format for <code>hmsToSec()</code> , or numeric vector of seconds for <code>secToHms()</code>
<code>digits</code>	the number of digits to use for potential fractions of seconds

Details

`secToHms` - Converts a vector of seconds to h:m:s.

Value

depending on the function:

hmsToMinute() numeric vector of times in minutes

hmsToSec() numeric vector of times in seconds

secToHms() character vector of times in h:m:s format

#'

Note

Based on code by Tyler Rinker, adapted to conform to package standards.

See Also

[times](#)

Other date.time: [addMonths\(\)](#), [countWorkDays\(\)](#), [date-time-predicates](#), [date_functions](#), [generation\(\)](#), [zodiac\(\)](#)

Examples

```
hmsToSec(c("02:00:03", "04:03:01"))
hmsToSec(secToHms(c(222, 1234, 55)))
secToHms(c(256, 3456, 56565))
```

tOne	Create Table One Describing Baseline Characteristics
------	--

Description

Create a table summarizing continuous, categorical and dichotomous variables, optionally stratified by one or more variables, while performing adequate statistical tests.

Usage

```
tOne(
  x, groups = NA, add.length = TRUE,
  colnames = NULL, vnames = NULL, total = TRUE,
  align = "\\l", FUN = NULL, TEST = NULL,
  intref = "high",
  fmt = list(abs = "abs.sty", num = "num.sty", per = "per.sty",
             pval = style(fmt = "*", naForm = " ")))
)

## S3 method for class 'tOne'
print(x, ...)

## S3 method for class 'tOne'
x[i, j, ..., drop = FALSE]
```

Arguments

x	a tOne object to subset
groups	the grouping variable
add.length	logical. If set to TRUE (default), a row with the group sizes will be inserted as first row of the table.
colnames	a vector of column names for the result table
vnames	a vector of variable names to be placed in the first column instead of the real names
total	logical (default TRUE), defines whether the results should also be displayed for the whole, ungrouped variable
align	the character on whose position the strings will be aligned. Left alignment can be requested by setting sep = "\l", right alignment by "\r" and center alignment by "\c". Mind the backslashes, as if they are omitted, strings would be aligned to the character l, r or c respectively. Default value is "\l", thus left alignment.
FUN	the function to be used as location and dispersion measure for numeric (including integer) variables (mean/sd is default, alternatives as median/IQR are possible by defining a function). See examples.
TEST	a list of functions to be used to test the variables. Must be named as "num", "cat" and "dich" and be defined as function with arguments (x, g), generating something similar to a p-value. Use TEST=NA to suppress test. (See examples.)
intref	one out of "high" (default), "low" or "both", defining which value of a dichotomous variable should be reported. Usually this will be 1 or TRUE. Setting it to "low" will report the lower value 0 or FALSE, "both" reports the variable as a categorical one with all its levels. Dichotomous factors are treated the same way, "high" reporting the last and "low" the first level.
fmt	fm codes for absolute, numeric and percentage values, and for the p-values of the tests
...	further parameters (not used here)
i	rowindex
j	columnindex
drop	drop the structure in case of total reduction

Details

In research the characteristics of study populations are often characterised through some kind of a "Table 1", containing descriptives of the used variables, as mean/standard deviation for continuous variables, and proportions for categorical variables. In many cases, a comparison is made between groups within the framework of the scientific question.

```

var      total      Brent      Camden      Westminster
1 n      1'199      474 (39.5%)  344 (28.7%)  381 (31.8%)
2 temperature 47.937 (9.938) 51.139 (8.734) 47.420 (10.111) 44.258 (9.836) *** 1
3 delivery_min 25.653 (10.843) 19.717 (7.975) 26.763 (9.994) 31.993 (10.803) *** 1
4 driver      95 (8.0%)   72 (15.2%)   1 (0.3%)    22 (5.8%)    *** 2
5 Butcher     269 (22.6%)   29 (6.1%)   19 (5.6%)   221 (58.2%)
6 Carpenter   229 (19.4%)   177 (37.4%)  47 (13.8%)   5 (1.3%)
7 Farmer      117 (9.7%)    19 (4.0%)   87 (25.5%)   11 (2.9%)
8 Hunter      156 (13.0%)   128 (27.1%)  4 (1.2%)    24 (6.3%)
9 Miller      124 (10.4%)   6 (1.3%)    41 (12.0%)   77 (20.3%)
10 Taylor     204 (16.9%)   42 (8.9%)   142 (41.6%)  20 (5.3%)
11 wine_ordered (= 1) 187 (15.8%)  44 (9.4%)   62 (18.1%)   81 (21.4%) *** 2
---
') Kruskal-Wallis test, *) Fisher exact test, >) Chi-Square test
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Creating such a table can be very time consuming and there's a need for a flexible function that helps us to solve the task. `tOne()` is designed to be easily used with sensible defaults, and yet flexible enough to allow free definition of the essential design elements.

This is done by breaking down the descriptive task to three types of variables: quantitative (numeric, integer), qualitative (factor, characters) and dichotomous variables (the latter having exactly two values or levels). Depending on the variable type, the descriptives and the according sensible tests are chosen. By default mean/sd are chosen to describe numeric variables.

```
FUN = function(x)
  gettextf("%s (%s)",
           fm(mean(x, na.rm = TRUE), fmt = fmt$num),
           fm(sd(x, na.rm = TRUE), fmt = fmt$num))
```

Their difference is tested with the Kruskal-Wallis test. For categorical variables the absolute and relative frequencies are calculated and tested with a chi-square test.

The tests can be changed with the argument `TEST`. These must be organised as list containing elements named "num", "cat" and "dich". Each of them must be a function with arguments (x, g), returning something similar to a p-value.

```
TEST = list( num = list(fun = function(x, g){
  summary(aov(x ~ g))\verb{[[1]][1, "Pr(>F)"]}}, lbl = "ANOVA"),
  cat = list(fun = function(x, g){
    chisq.test(table(x, g))$p.val}, lbl = "Chi-Square test"),
  dich = list(fun = function(x, g){
    fisher.test(table(x, g))$p.val}, lbl = "Fisher exact test")
)
```

The legend text of the test, which is appended to the table together with the significance codes, can be set with the variable `lbl`.

Great importance was attached to the free definition of the number fms. By default, the optionally definable fm templates of **DescToolsX** are used. Deviations from this can be freely passed as arguments to the function. fms can be defined for integers, floating point numbers, percentages and for the p-values of statistical tests. All options of the function `fm()` are available and can be provided as a list. See examples which show several different implementations.

```
fmt = list(abs = "abs.sty",
           num = "num.sty",
           per = "per.sty",
           pval = style(fmt = "*", naForm = " "))
)
```

Several tables can be appended using `appendX()`. This can be useful, if e.g. the mean/sd AND median/IQR should be displayed together. Another use case is to introduce a delimiter row.

The function returns a character matrix as result, which can easily be subset or combined with other matrices. An interface for `toWrd()` is available such that the matrix can be transferred to MS-Word. Both font and alignment are freely selectable in the Word table.

Value

a character matrix of class `tOne`

See Also[appendX\(\)](#)Other frequency: [expFreq\(\)](#), [freq\(\)](#), [freq2D\(\)](#), [percTable\(\)](#)**Examples**

```

opt <- options(scipen = 8)

# define some special fms for count data, percentages and numeric results
# (those will be supported by tOne)
abs.sty <- style(digits = 0, bigMark = "'") # counts
per.sty <- style(digits = 1, fmt = "%")     # percentages
num.sty <- style(digits = 1, bigMark = "'") # numeric

tOne(x = Pizza[, c("temperature", "delivery_min", "driver", "wine_ordered")],
     groups = Pizza$quality)

# the same but no groups now...
tOne(x = Pizza[, c("temperature", "delivery_min", "driver", "wine_ordered")])

# define median/IQR as describing functions for the numeric variables
tOne(iris[, -5], iris[, 5],
     FUN = function(x) {
       gettextf("%s / %s",
               fm(median(x, na.rm = TRUE), digits = 1),
               fm(IQR(x, na.rm = TRUE), digits = 3))
     })

# replace kruskal.test by ANOVA and report the p.value
# Change tests for all the types
tOne(x = iris[, -5], groups = iris[, 5],
     FUN = function(x) gettextf("%s / %s",
                               fm(mean(x, na.rm = TRUE), digits = 1),
                               fm(sd(x, na.rm = TRUE), digits = 3)),

     TEST = list(
       num = list(fun = function(x, g){summary(aov(x ~ g))[[1]][1, "Pr(>F)"]},
                 lbl = "ANOVA"),
       cat = list(fun = function(x, g){chisq.test(table(x, g))$p.val},
                 lbl = "Chi-Square test"),
       dich = list(fun = function(x, g){fisher.test(table(x, g))$p.val},
                  lbl = "Fisher exact test")),
     fmt = list(abs = "abs.sty", num = "num.sty", per = "per.sty",
               pval = style(fmt = "*", naForm = " ")),

)

t1 <- tOne(x = Pizza[,c("temperature", "driver", "rebate")],
          groups = Pizza$area,
          align = " ",
          total = FALSE,

          FUN = function(x) gettextf("%s / %s (%s)",
                                     fm(mean(x, na.rm = TRUE), digits = 1),
                                     fm(sd(x, na.rm = TRUE), digits = 3),
                                     fm(median(x, na.rm = TRUE), digits = 1)),

```

```

TEST = NA,

fmt = list(abs = style(bigMark = " ", digits=0),
           num = style(bigMark = " ", digits=1),
           per = style(fmt=function(x)
             strPad(fm(x, fmt="%", digits=1), width=5, adj = "r")),
           pval = style(fmt = "*", naForm = " "))
)
# add a userdefined legend
attr(t1, "legend") <- "numeric: mean / sd (median), factor: n (n%)"

t1

# dichotomous integer or logical values can be reported by the high or low value
set.seed(1)
x <- sample(x = c(0, 1), size = 100, prob = c(0.3, 0.7), replace = TRUE)
y <- sample(x = c(0, 1), size = 100, prob = c(0.3, 0.7), replace = TRUE) == 1
z <- factor(sample(x = c(0, 1), size = 100, prob = c(0.3, 0.7), replace = TRUE))
g <- sample(x = letters[1:4], size = 100, replace = TRUE)
d.set <- data.frame(x = x, y = y, z = z, g = g)

tOne(d.set[1:3], d.set$g, intref = "low")

tOne(d.set[1:3], d.set$g, intref = "high")

# report both levels of the factor
tOne(data.frame(z = z), g, intref = "both")

options(opt)

## Not run:

# Send the whole stuff to Word
wrđ <- getNewWrđ()
toWrđ(
  tOne(x = Pizza[, c("temperature", "delivery_min", "driver", "wine_ordered")],
       groups = Pizza$quality,
       fmt = list(num=style(digits=1))
  ),
  font = list(name="Arial narrow", size=8),
  align = c("l", "r") # this will be recycled: left-right-left-right ...
)

## End(Not run)

```

Description

Computes Tschuprow's T, a measure of association between two categorical variables based on the chi-squared statistic.

Usage

```
tschuprowT(x, y = NULL, correct = FALSE, ...)
```

Arguments

x	a vector of categorical data (then y must be given) or a two-dimensional contingency table (matrix or table)
y	optional second categorical vector. If provided, a contingency table is constructed from x and y.
correct	logical; if TRUE, applies a bias correction according to Bergsma (2013).
...	additional arguments passed to table . This refers only to the vector interface.

Details

If y is provided, a contingency table is created using `table(x, y, ...)`. Otherwise, x is assumed to already be a two-dimensional contingency table.

Tschuprow's T is defined as:

$$T = \sqrt{\frac{\chi^2}{n\sqrt{(r-1)(c-1)}}$$

where χ^2 is the chi-squared statistic, n is the total sample size, and r and c are the number of rows and columns of the contingency table.

If `correct = TRUE`, a bias-corrected version is computed based on Bergsma (2013), which adjusts the estimate especially for small samples. It replaces $\phi^2 = \chi^2/n$ by $\tilde{\phi}^2 = \max(0, \phi^2 - (r-1)(c-1)/(n-1))$ and the dimensions by $\tilde{r} = r - (r-1)^2/(n-1)$ and $\tilde{c} = c - (c-1)^2/(n-1)$.

For a 2x2 table T coincides with Cramer's V and with the absolute value of the phi coefficient; the sign of the association is not reported.

Value

a numeric scalar containing Tschuprow's T

References

Tschuprow, A. A. (1939). *Principles of the Mathematical Theory of Correlation*. W. Hodge & Co.

Bergsma, W. (2013). A bias-correction for Cramer's V and Tschuprow's T. *Journal of the Korean Statistical Society*, 42(3), 323–328. <https://doi.org/10.1016/j.jkss.2012.10.002>

See Also

[chisq.test](#), [cramerV](#)

Other assoc.nominal: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [uncertCoef\(\)](#), [yule](#)

Examples

```
# Example with vectors
x <- c("A", "A", "B", "B")
y <- c("yes", "no", "yes", "no")
tschuprowT(x, y)

# Example with contingency table
tab <- matrix(c(10, 20, 30, 40), nrow = 2)
tschuprowT(tab) # 0.08908708

# Bias-corrected version: the correction exceeds the estimate here,
# so the corrected value is 0
tschuprowT(tab, correct = TRUE)
```

tukeyBiweight	<i>Tukey's Biweight Mean</i>
---------------	------------------------------

Description

Computes Tukey's biweight robust mean (also known as the bisquare mean) of a numeric vector, optionally with a bootstrap confidence interval.

Usage

```
tukeyBiweight(
  x,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  const = 9,
  na.rm = FALSE,
  ...
)
```

Arguments

<code>x</code>	a non-empty numeric vector of data values
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>const</code>	tuning constant passed to <code>tbrm_cpp()</code> . Defaults to 9.
<code>na.rm</code>	logical. Should missing values be removed before computation? Defaults to FALSE.
<code>...</code>	further arguments passed to the bootstrap engine when a confidence interval is requested, namely R and type. Any other name is an error rather than a silent no-op.

Details

The biweight mean is a robust location estimator that downweights observations far from the median. It is defined via the tuning constant `const` (default 9), which controls the breakdown point: larger values are less resistant but more efficient under normality.

When `conf.level` is not NA a bootstrap confidence interval is returned. The resampling is done in C++; the R random number generator is used only to draw the seed, so `set.seed()` makes the result reproducible. Bootstrap arguments are passed through `...`:

`R` Number of bootstrap replicates (default 999).

`type` CI type: "perc" or "bca" (default).

The biweight mean is a location estimator and therefore unbounded, so the open side of a one-sided interval is reported at $\pm\infty$ - unlike the bounded measures in this package, where it is reported at the range limit. See [ConfidenceIntervals](#).

Value

if `conf.level` = NA, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of Tukey's biweight mean

`lci` lower confidence interval bound

`uci` upper confidence interval bound

See Also

Other location: [gmean\(\)](#), [hmean\(\)](#), [hodgesLehmann\(\)](#), [huberM\(\)](#), [meanX\(\)](#), [medianX\(\)](#), [modeX\(\)](#)

Examples

```
set.seed(1)
x <- c(rnorm(50), 10) # one outlier

tukeyBiweight(x)

set.seed(2) # will yield reproducible intervals
tukeyBiweight(x, conf.level = 0.95)
tukeyBiweight(x, conf.level = 0.95, type = "perc", R = 499)
tukeyBiweight(x, conf.level = 0.95, type = "bca", R = 499)

# one-sided: "left" carries the finite lower bound
set.seed(2)
tukeyBiweight(x, conf.level = 0.95, sides = "left")
```

uncertCoef	<i>Uncertainty Coefficient</i>
------------	--------------------------------

Description

Computes directional or symmetric uncertainty coefficients. The directional coefficient $U(C|R)$ measures the proportion of uncertainty (entropy) in the column variable Y that is explained by the row variable X . The function has interfaces for a table, a matrix, a data frame, and single vectors.

Usage

```
uncertCoef(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  direction = c("symmetric", "row", "column"),
  pZeroCorrection = 1/sum(x)^2,
  ...
)
```

Arguments

<code>x</code>	a numeric vector, factor, matrix, or data frame
<code>y</code>	NULL (default) or a vector, an ordered factor, matrix or data frame with compatible dimensions to <code>x</code>
<code>conf.level</code>	confidence level of the interval. If set to NA (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See ConfidenceIntervals .
<code>direction</code>	direction of calculation, one of "symmetric" (default), "row", or "column". The row direction calculates $U(RIC)$, and the column direction calculates $U(CIR)$.
<code>pZeroCorrection</code>	small positive value used to replace zero cells before taking logarithms
<code>...</code>	further arguments are passed to the function table , allowing, for example, <code>useNA</code> to be set. This refers only to the vector interface.

Details

The uncertainty coefficient is computed as

$$U(C|R) = \frac{H(X) + H(Y) - H(XY)}{H(Y)}$$

and ranges from $[0, 1]$.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

```
est  uncertainty coefficient estimate
lci  lower confidence interval bound
uci  upper confidence interval bound
```

Confidence interval

The interval is based on the asymptotic standard error (Goodman & Kruskal) and is truncated to the parameter range $[0, 1]$. For a one-sided interval (`sides = "left"` or `"right"`) the open side is reported at the corresponding range limit, not at $\pm\infty$.

Note

Based on code from Antti Arppe

References

Theil, H. (1972), *Statistical Decomposition Analysis*, Amsterdam: North-Holland Publishing Company.

See Also

[Association](#)

Other `assoc.nominal`: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [yule](#)

Examples

```
# example from Goodman Kruskal (1954)

m <- as.table(cbind(c(1768,946,115), c(807,1387,438), c(189,746,288), c(47,53,16)))
dimnames(m) <- list(paste("A", 1:3), paste("B", 1:4))
m

# direction default is "symmetric"
uncertCoef(m)
uncertCoef(m, conf.level=0.95)

uncertCoef(m, direction="row")
uncertCoef(m, direction="column")
```

varX

(Weighted) Variance and Standard Deviation

Description

`varX()` computes the variance of `x`, allowing the definition of weights (unlike base R's [var\(\)](#) function). Using the estimator `m1` returns the uncorrected sample variance (which is a biased estimator for the sample variance).

`sdX` yields the standard deviation following the same logic.

Usage

```
sdX(x, estimator = c("unbiased", "ml"), weights = NULL, na.rm = FALSE, ...)

varX(x, ...)

## Default S3 method:
varX(x, estimator = c("unbiased", "ml"), weights = NULL, na.rm = FALSE, ...)

## S3 method for class 'Freq'
varX(x, breaks, estimator = c("unbiased", "ml"), ...)
```

Arguments

x	a numeric vector, matrix, or data frame
estimator	determines the estimator type; if "unbiased" (the default) then the usual unbiased estimate (using $n - 1$ as denominator) is returned, if "ml" then it is the maximum likelihood estimate for a Gaussian distribution (denominator n).
weights	non-negative numeric vector of weights the same length as x, interpreted as frequency (replication) weights. Observations with larger weights contribute more strongly to the empirical distribution. Weights are supported for vector input only.
na.rm	logical. Should missing values be removed?
...	further arguments passed to or from other methods
breaks	breaks for calculating the variance for classified data as composed by freq

Details

Using estimator "unbiased" the denominator $n - 1$ is used (known as "Bessel's correction") which gives an unbiased estimator of the (co)variance for i.i.d. observations. "ml" yields the biased version using the denominator n . With frequency weights n is the sum of the weights.

These functions return [NA](#) when there is only one observation and NA when x has length zero.

Note: Analytic (precision) weights are not supported. For likelihood-based weighted variance estimation, see [cov.wt](#).

Value

varX() a numeric scalar for vector input or a covariance matrix for a matrix or data frame
 sdX() a numeric scalar containing the standard deviation

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

[varCI](#) for confidence intervals, [varTest](#) for tests and base R's implementations [var](#), [sd](#), [cov](#)
 Other dispersion: [coefVar\(\)](#), [iqrX\(\)](#), [madX\(\)](#), [meanAD\(\)](#), [meanSE\(\)](#), [rangeX\(\)](#)

Examples

```

varX(1:10)          # 9.166667
sdX(1:10)

# frequency weights replicate the observations, so the result is the
# variance of the expanded vector c(1, 2,2, 3,3,3, 4,4,4,4, 5,5,5,5,5)
varX(1:5, weights=1:5) # 1.666667
varX(rep(1:5, times=1:5)) # 1.666667

# weighted Variance
set.seed(45)
(z <- as.numeric(names(w <- table(x <- sample(-10:20, size=50, replace=TRUE))))
varX(z, weights=w)
sdX(z, weights=w)

# check!
all.equal(varX(x), varX(z, weights=w))

# Variance for frequency tables
varX(freq(as.table(c(6,16,24,25,17))),
        breaks=c(0, 10, 20, 30, 40, 50))

```

yeoJohnson

Yeo-Johnson Transformation

Description

yeoJohnson() applies the Yeo-Johnson transformation to a numeric vector.
yeoJohnsonInv() reverses the transformation.

Usage

```

yeoJohnson(x, lambda, tol = 0.000001)

yeoJohnsonInv(x, lambda, tol = 0.000001)

```

Arguments

x	a numeric vector
lambda	a single numeric transformation parameter
tol	numeric tolerance for detecting the special cases $\lambda \approx 0$ and $\lambda \approx 2$

Details

The Yeo-Johnson transformation extends the Box-Cox transformation to allow for zero and negative values. It is defined piecewise:

$$f_{\lambda}(x) = \frac{(x+1)^{\lambda} - 1}{\lambda}$$

for $x \geq 0$ and $\lambda \neq 0$,

$$f_{\lambda}(x) = \log(x + 1)$$

for $x \geq 0$ and $\lambda = 0$,

$$f_{\lambda}(x) = -\frac{(-x + 1)^{2-\lambda} - 1}{2 - \lambda}$$

for $x < 0$ and $\lambda \neq 2$, and

$$f_{\lambda}(x) = -\log(-x + 1)$$

for $x < 0$ and $\lambda = 2$.

The transformation is defined for all real-valued inputs and is continuous and differentiable for all x . It is commonly used as an alternative to the Box-Cox transformation when the data include zero or negative values.

The transformation is strictly increasing and maps 0 to 0, so the sign of the transformed value identifies the branch to be inverted.

The inverse transformation recovers the original data (up to numerical precision) when the same `lambda` and `tol` are used. For $\lambda < 0$ the image of the transformation is bounded above by $-1/\lambda$ (and correspondingly for $\lambda > 2$ below); values outside that range have no preimage and are reported as an error.

Value

a numeric vector of the same length as `x`. Names and dimensions of `x` are preserved.

References

Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, **87**(4), 954–959.

See Also

Other transform: [boxCox\(\)](#), [boxCoxLambda\(\)](#), [logSt\(\)](#), [scaleX\(\)](#)

Examples

```
set.seed(1)
x <- rnorm(500)

y <- yeoJohnson(x, lambda = 0.5)
x_back <- yeoJohnsonInv(y, lambda = 0.5)

# Check inversion
max(abs(x - x_back))

# Compare with log-like transformation
y0 <- yeoJohnson(x, lambda = 0)
```

yule

*Yule's Coefficients of Association (Q and Y)***Description**

Computes Yule's Q or Y for a 2x2 contingency table, optionally with asymptotic confidence interval based on the log odds ratio.

Usage

```
yuleQ(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  correct = FALSE,
  ...
)

yuleY(
  x,
  y = NULL,
  conf.level = NA,
  sides = c("two.sided", "left", "right"),
  correct = FALSE,
  ...
)
```

Arguments

<code>x</code>	a 2x2 contingency table (matrix or table). If <code>y</code> is supplied, <code>x</code> and <code>y</code> are cross-tabulated via <code>table()</code> .
<code>y</code>	optional second variable for cross-tabulation
<code>conf.level</code>	confidence level of the interval. If set to <code>NA</code> (the default), only the point estimate is returned.
<code>sides</code>	character string specifying the sidedness of the confidence interval (one of "two.sided" (default), "left" or "right"). See details in ConfidenceIntervals .
<code>correct</code>	logical; if <code>TRUE</code> , applies the Haldane–Anscombe correction by adding 0.5 to all cells
<code>...</code>	further arguments passed to <code>table()</code>

Details

For a 2x2 table with cell counts a, b, c, d :

Odds ratio:

$$OR = \frac{ad}{bc}$$

Yule's Q:

$$Q = \frac{OR - 1}{OR + 1} = \tanh\left(\frac{1}{2} \log(OR)\right)$$

Yule's Y:

$$Y = \frac{\sqrt{OR} - 1}{\sqrt{OR} + 1} = \tanh\left(\frac{1}{4} \log(OR)\right)$$

Both coefficients are computed from the tanh form, which stays well defined when a zero cell drives the odds ratio to 0 or Inf (the coefficient is then -1 or 1).

Confidence intervals are obtained from the asymptotic normal approximation:

$$\log(OR) \pm z \cdot \sqrt{1/a + 1/b + 1/c + 1/d}$$

and then transformed to the selected coefficient. With a zero cell the standard error is infinite and the interval degenerates to $[-1, 1]$; use `correct = TRUE` to obtain a finite interval.

For a one-sided interval the open side is reported at the range limit (-1 resp. 1), not at $\pm\infty$.

Value

if `conf.level = NA`, a numeric scalar. Otherwise a named numeric vector with elements:

`est` point estimate of Yule's Q or Y

`lci` lower confidence interval bound

`uci` upper confidence interval bound

References

Yule, G. U. (1912). On the methods of measuring association between two attributes.

See Also

Other `assoc.nominal`: [contCoef\(\)](#), [cramerV\(\)](#), [gkTau\(\)](#), [lambda\(\)](#), [mutInf\(\)](#), [phi\(\)](#), [tschuprowT\(\)](#), [uncertCoef\(\)](#)

Examples

```
m <- matrix(c(12, 5, 3, 20), nrow = 2)
yuleQ(m) # 0.8823529
yuleY(m, conf.level = 0.95) # 0.6

# a zero cell yields the limiting value 1 (and not NaN)
yuleQ(matrix(c(12, 5, 0, 20), nrow = 2), conf.level = NA)

# ... a finite interval requires the Haldane-Anscombe correction
yuleQ(matrix(c(12, 5, 0, 20), nrow = 2), correct = TRUE)
```

zodiac*Calculate the Zodiac of a Date*

Description

Calculate the sign of zodiac of a date.

Usage

```
zodiac(x, lang = c("en", "de"), stringsAsFactors = TRUE)
```

Arguments

x	the date to transform, a Date or anything that month() and day() accept
lang	language of the zodiac names, either English ("en") or German ("de")
stringsAsFactors	logical. If set to TRUE (default) the result will consist of a factor with zodiac signs as levels.

Details

The really relevant things can sometimes hardly be found. You just discovered such a function... ;-)

The following rule to determine zodiac symbols is implemented:

```
Dec. 22 - Jan. 19 : Capricorn | Jan. 20 - Feb. 17 : Aquarius |  
Feb. 18 - Mar. 19 : Pisces | March 20 - April 19 : Aries | April 20 - May 19 :  
Taurus | May 20 - June 20 : Gemini | June 21 - July 21 : Cancer | July 22 - Aug.  
22 : Leo | Aug 23 - Sept. 21 : Virgo | Sept. 22 - Oct. 22 : Libra | Oct. 23 -  
Nov. 21 : Scorpio | Nov. 22 - Dec. 21 : Sagittarius
```

The boundaries are fixed calendar dates; the astronomical dates of the sun's entry into a sign shift by up to a day from year to year.

Value

a character vector or factor containing the zodiac signs

Note

Based on code from Markus Naepflin, adapted to conform to package standards.

See Also

Other date.time: [addMonths\(\)](#), [countWorkDays\(\)](#), [date-time-predicates](#), [date_functions](#), [generation\(\)](#), [time-conversions](#)

Examples

```
zodiac(as.Date(c("1937-07-28", "1936-06-01", "1966-02-25",  
                "1964-11-17", "1972-04-25")), lang="de")  
  
# the boundary days  
zodiac(as.Date(c("2015-01-19", "2015-01-20", "2015-12-21", "2015-12-22")))  
  
set.seed(1)  
d <- sample(seq(as.Date("2015-01-01"), as.Date("2015-12-31"), 1), 120)  
z <- zodiac(d)  
desc(z)
```

Index

- * **agreement**
 - blandAltmanData, 16
 - icc, 114
 - isConfusionTable, 120
 - krippAlpha, 126
 - normalizeToConfusion, 154
 - percAgreement, 167
 - randolphKappa, 179
 - raterFrame, 181
- * **anova-effect-size**
 - etaSq, 84
- * **assoc.agreement**
 - ccc, 22
 - cohenKappa, 30
 - cronbachAlpha, 49
 - icc, 114
 - kappaM, 121
 - krippAlpha, 126
 - pabak, 163
 - percAgreement, 167
 - randolphKappa, 179
- * **assoc.continuous**
 - corPart, 42
 - corPolychor, 44
 - findCorrX, 89
 - hoeffdingD, 109
 - keepSig, 123
 - pearsonCor, 165
 - spearmanCor, 194
- * **assoc.nominal**
 - contCoef, 40
 - cramerV, 47
 - gkTau, 97
 - lambda, 131
 - mutInf, 150
 - phi, 171
 - tschuprowT, 202
 - uncertCoef, 206
 - yule, 211
- * **assoc.ordinal**
 - conDisPairs, 34
 - cStat, 52
 - kendallW, 124
 - ordAssocs, 159
- * **association-measures**
 - desc.qn, 75
 - desc.qq, 76
- * **association-measure**
 - Association, 9
 - conDisPairs, 34
 - contCoef, 40
 - corPart, 42
 - cramerV, 47
 - cStat, 52
 - gkTau, 97
 - hoeffdingD, 109
 - kendallW, 124
 - lambda, 131
 - mutInf, 150
 - ordAssocs, 159
 - pearsonCor, 165
 - phi, 171
 - tschuprowT, 202
 - uncertCoef, 206
 - yule, 211
- * **binary-association**
 - yule, 211
- * **binary-outcome**
 - cohenH, 29
 - oddsRatio, 156
 - relRisk, 183
- * **binning**
 - cut.integer, 53
 - cutAge, 54
 - cutQ, 55
- * **bivariate numeric regression correlation**
 - scatterplot
 - desc.nn, 70
- * **calibration**
 - brierScore, 20
- * **categorical-agreement**
 - cohenKappa, 30
 - kappaM, 121
 - krippAlpha, 126
 - pabak, 163
 - percAgreement, 167

- randolphKappa, 179
- * **categorization**
 - generation, 95
 - zodiac, 213
- * **chi-square-based**
 - contCoef, 40
 - cramerV, 47
 - expFreq, 87
 - phi, 171
 - tschuprowT, 202
- * **class-predicate**
 - date-time-predicates, 57
- * **classification**
 - conf, 36
- * **concentration-index**
 - atkinson, 11
 - divCoef, 82
 - gini, 96
 - herfindahl, 104
 - rosenbluth, 186
 - simpson, 190
 - theil, 196
- * **concordance**
 - conDisPairs, 34
- * **confusion-matrix**
 - conf, 36
 - isConfusionTable, 120
 - normalizeToConfusion, 154
- * **convenience**
 - attach-detach-aliases, 13
- * **correlation**
 - ccc, 22
 - corPart, 42
 - corPolychor, 44
 - findCorrX, 89
 - keepSig, 123
 - pearsonCor, 165
 - spearmanCor, 194
- * **cut**
 - cut.integer, 53
 - cutAge, 54
 - cutQ, 55
- * **data-description**
 - desc, 63
 - desc.Date, 66
 - desc.factor, 68
 - desc.nq, 72
 - desc.numeric, 73
 - desc.qn, 75
 - desc.qq, 76
 - desc.ts, 77
 - print.Desc.qq, 174
- * **data.inspection**
 - abstract, 4
 - outlier, 162
- * **date-handling**
 - desc.Date, 66
- * **date-time**
 - addMonths, 6
 - as_ym, 10
 - countWorkDays, 46
 - date-time-predicates, 57
 - date_functions, 58
 - generation, 95
 - time-conversions, 197
 - zodiac, 213
- * **date.time**
 - addMonths, 6
 - countWorkDays, 46
 - date-time-predicates, 57
 - date_functions, 58
 - generation, 95
 - time-conversions, 197
 - zodiac, 213
- * **demographics**
 - cutAge, 54
- * **density-based**
 - lof, 135
- * **descriptive-statistics**
 - desc, 63
 - desc.Date, 66
 - desc.factor, 68
 - desc.nq, 72
 - desc.numeric, 73
 - desc.qn, 75
 - desc.qq, 76
 - desc.ts, 77
 - kurt, 129
 - lc, 132
 - print.Desc.qq, 174
- * **descriptive**
 - desc, 63
- * **desc**
 - desc, 63
 - desc.Date, 66
 - desc.factor, 68
 - desc.nn, 70
 - desc.nq, 72
 - desc.numeric, 73
 - desc.qn, 75
 - desc.qq, 76
 - desc.ts, 77
 - print.Desc.qq, 174
- * **dispersion**

- coefVar, 24
- iqrX, 119
- madX, 138
- meanAD, 142
- meanSE, 143
- rangeX, 180
- varX, 207
- * **distribution-summary**
 - extremes, 88
 - quantileX, 177
- * **diversity.concentration**
 - entropy, 83
 - herfindahl, 104
 - simpson, 190
- * **diversity**
 - divCoef, 82
- * **effect-size**
 - cohenD, 27
 - cohenH, 29
 - etaSq, 84
 - glassDelta, 100
 - oddsRatio, 156
 - relRisk, 183
- * **effect.size**
 - cohenD, 27
 - cohenH, 29
 - etaSq, 84
 - glassDelta, 100
 - oddsRatio, 156
 - relRisk, 183
- * **factor-handling**
 - desc.factor, 68
- * **feature-selection**
 - findCorrX, 89
- * **frequency-table**
 - expFreq, 87
 - freq, 91
 - freq2D, 93
 - percTable, 168
 - tOne, 198
- * **frequency**
 - expFreq, 87
 - freq, 91
 - freq2D, 93
 - percTable, 168
 - tOne, 198
- * **hypothesis-testing**
 - desc.nq, 72
- * **imputation**
 - impute, 116
 - imputeKnn, 117
- * **impute**
 - impute, 116
 - imputeKnn, 117
- * **inequality**
 - atkinson, 11
 - divCoef, 82
 - gini, 96
 - herfindahl, 104
 - lc, 132
 - rosenbluth, 186
 - theil, 196
- * **information-theory**
 - entropy, 83
 - mutInf, 150
 - uncertCoef, 206
- * **internal-consistency**
 - cronbachAlpha, 49
- * **interrater-agreement**
 - kappaM, 121
 - kendallW, 124
 - pabak, 163
- * **introspection**
 - concepts, 33
- * **latent-variable**
 - corPolychor, 44
- * **location**
 - gmean, 102
 - hmean, 105
 - hodesLehmann, 107
 - huberM, 112
 - meanX, 144
 - medianX, 145
 - modeX, 147
 - tukeyBiweight, 204
- * **method-comparison**
 - blandAltmanData, 16
 - ccc, 22
- * **missing-value**
 - impute, 116
 - imputeKnn, 117
- * **model-evaluation**
 - auc, 14
 - brierScore, 20
 - conf, 36
 - mae, 140
 - mape, 141
 - mse, 149
 - nmae, 152
 - nmse, 153
- * **model.classification**
 - conf, 36
- * **model.metrics**
 - auc, 14

- brierScore, 20
- mae, 140
- mape, 141
- mse, 149
- nmae, 152
- nmse, 153
- rmse, 185
- smape, 193
- * **moments**
 - skew, 191
- * **multiple-testing**
 - keepSig, 123
- * **nominal**
 - contCoef, 40
 - cramerV, 47
 - gkTau, 97
 - lambda, 131
 - mutInf, 150
 - phi, 171
 - tschuprowT, 202
 - uncertCoef, 206
 - yule, 211
- * **nonlinear-association**
 - hoeffdingD, 109
- * **nonlinear-mean**
 - gmean, 102
 - hmean, 105
- * **order-statistic**
 - extremes, 88
- * **ordinal**
 - conDisPairs, 34
 - corPolychor, 44
 - cStat, 52
 - kendallW, 124
 - ordAssocs, 159
- * **outlier-detection**
 - lof, 135
 - outlier, 162
- * **outlier-resistance**
 - scaleX, 187
- * **outlier**
 - lof, 135
- * **pearson spearman r-squared residuals**
 - heteroscedasticity
 - desc.nn, 70
- * **pkg.introspection**
 - concepts, 33
- * **prediction-error**
 - mae, 140
 - mape, 141
 - mse, 149
 - nmae, 152
 - nmse, 153
 - rmse, 185
 - smape, 193
- * **programming**
 - setDescToolsXOption, 189
- * **quantile**
 - extremes, 88
 - quantileX, 177
- * **rank-correlation**
 - spearmanCor, 194
- * **rater-data**
 - raterFrame, 181
- * **reliability**
 - ccc, 22
 - cronbachAlpha, 49
 - icc, 114
 - krippAlpha, 126
- * **robust-statistics**
 - hodgesLehmann, 107
 - huberM, 112
 - kurt, 129
 - madX, 138
 - tukeyBiweight, 204
- * **robust-statistic**
 - scaleX, 187
- * **roc**
 - auc, 14
 - cStat, 52
- * **shape**
 - kurt, 129
 - skew, 191
- * **standardization**
 - scaleX, 187
- * **summary**
 - abstract, 4
 - desc, 63
- * **table-manipulation**
 - print.Desc.qq, 174
- * **table-summary**
 - tOne, 198
- * **time-series**
 - desc.ts, 77
- * **transformation**
 - boxCox, 17
 - boxCoxLambda, 19
 - logSt, 136
 - yeoJohnson, 209
- * **transform**
 - boxCox, 17
 - boxCoxLambda, 19
 - logSt, 136
 - scaleX, 187

- yeoJohnson, 209
- * **utils**
 - setDescToolsXOption, 189
- * **variance-analysis**
 - etaSq, 84
- * **variance-component**
 - icc, 114
- * **variance-stabilization**
 - boxCox, 17
 - boxCoxLambda, 19
 - logSt, 136
 - yeoJohnson, 209
- .descNN(desc.nn), 70
- .descNQ(desc.nq), 72
- .descQN(desc.qn), 75
- .descQQ(desc.qq), 76
- [.tOne(tOne), 198
- abs, 140, 152
- abstract, 4, 74
- abstract(), 163
- addmargins, 170
- addMonths, 6
- addMonths(), 10, 47, 58, 62, 95, 198, 213
- Agreement, 7, 79
- Anova, 86
- anova, 86
- aov, 86
- aovlDetails(etaSq), 84
- aovlErrorTerms(etaSq), 84
- appendX, 200, 201
- approx, 15
- as.Date, 6, 46
- as.Date.ym(as_ym), 10
- as.POSIXlt, 62
- as.vector, 181
- as.ym(as_ym), 10
- as.ym(), 7
- as_ym, 10
- assign, 14
- Association, 9, 40, 47, 49, 79, 99, 132, 195, 207
- atkinson, 11, 105
- atkinson(), 83, 97, 134, 187, 197
- attach-detach-aliases, 13
- attachAliases, 13, 158
- attachAliases(attach-detach-aliases), 13
- auc, 14
- auc(), 22, 140, 141, 150, 152, 153, 186, 194
- base::plot(), 74
- base::summary(), 5, 74
- base::table, 49
- bedrock::naReplace(), 117
- bedrock::pairApply, 42, 49
- bedrock::recycle(), 6
- bedrock::resolveFormula, 182
- binomCI, 65
- blandAltmanData, 16, 172
- blandAltmanData(), 155, 182
- boot, 40, 103, 106, 127, 129, 192
- boot.ci, 40, 102, 106
- bootCI, 128
- boxCox, 17
- boxCox(), 20, 138, 188, 210
- boxCoxInv(boxCox), 17
- boxCoxLambda, 18, 19, 78
- boxCoxLambda(), 18, 138, 188, 210
- boxplot, 163
- boxplot.stats, 162
- bpTest, 71
- brierScore, 20
- brierScore(), 15, 140, 141, 150, 152, 153, 186, 194
- ccc, 8, 22, 31
- ccc(), 32, 51, 115, 122, 128, 165, 168, 180
- chisq.test, 9, 87, 172, 203
- coefVar, 24
- coefVar(), 119, 139, 142, 144, 181, 208
- coefVarCI(coefVar), 24
- cohenD, 27, 101
- cohenD(), 30, 86, 101, 158, 184
- cohenH, 29
- cohenH(), 28, 86, 101, 158, 184
- cohenKappa, 8, 30
- cohenKappa(), 24, 51, 115, 122, 128, 165, 168, 180
- colMeans, 145
- columnWrap(), 5
- complete.cases, 8, 9
- conceptAudit(concepts), 33
- conceptMap(concepts), 33
- concepts, 33
- conDisPairs, 34, 160
- conDisPairs(), 53, 126, 161
- conf, 36
- ConfidenceIntervals, 7–9, 21, 23, 25, 27, 29, 31, 39, 41, 48, 50, 79, 96, 98, 100, 102, 106, 107, 112, 114, 121, 122, 127, 129, 131, 157, 160, 164, 166, 167, 183, 192, 195, 204–206, 211
- confint, 40
- confint.glm, 157, 158

- constants, 40
- contCoef, 10, 40, 48
- contCoef(), 49, 99, 132, 151, 172, 203, 207, 212
- cor, 9, 43, 123, 194
- cor.test, 71, 124
- corCI, 71
- corPart, 42
- corPart(), 45, 90, 111, 124, 166, 195
- corPolychor, 10, 44
- corPolychor(), 43, 90, 111, 124, 166, 195
- countWorkDays, 46, 62
- countWorkDays(), 7, 58, 62, 95, 198, 213
- cov, 43, 208
- cov.wt, 25, 208
- cramerV, 9, 41, 47, 99, 172, 176, 203
- cramerV(), 42, 99, 132, 151, 172, 203, 207, 212
- cronbachAlpha, 8, 49
- cronbachAlpha(), 24, 32, 115, 122, 128, 165, 168, 180
- cStat, 52, 161
- cStat(), 35, 126, 161
- cumsum, 93
- cut, 53–56, 92–94
- cut.default, 53, 54
- cut.integer, 53
- cut.integer(), 55, 56
- cutAge, 54, 54, 95
- cutAge(), 54, 56
- cutQ, 54, 55
- cutQ(), 54, 55
- Date, 146
- date, 144
- date-time, 144
- date-time-predicates, 57
- date_functions, 7, 47, 58, 58, 95, 198, 213
- DateTimeClasses, 58, 62
- day(date_functions), 58
- day.abb(constants), 40
- day.name(constants), 40
- day<- (date_functions), 58
- desc, 63, 67, 71, 73, 76, 77, 173, 176
- desc(), 5, 67, 69, 71, 73, 74, 76–78, 176
- desc.array(print.Desc.qq), 174
- desc.character(desc.factor), 68
- desc.Date, 65, 66
- desc.Date(), 66, 69, 71, 73, 74, 76–78, 176
- desc.factor, 65, 68, 176
- desc.factor(), 66, 67, 71, 73, 74, 76–78, 176
- desc.logical, 65
- desc.matrix(print.Desc.qq), 174
- desc.nn, 65–67, 69, 70, 73, 74, 76–78, 176
- desc.nq, 65–67, 69, 71, 72, 74, 76–78, 176
- desc.numeric, 65, 73, 176
- desc.numeric(), 66, 67, 69, 71, 73, 76–78, 176
- desc.qn, 65–67, 69, 71, 73, 74, 75, 77, 78, 176
- desc.qq, 65–67, 69, 71, 73, 74, 76, 76, 78, 176
- desc.table, 65, 77
- desc.table(print.Desc.qq), 174
- desc.ts, 65, 77
- desc.ts(), 66, 67, 69, 71, 73, 74, 76, 77, 176
- DescToolsX, 79
- DescToolsX-package (DescToolsX), 79
- detachAliases, 14
- detachAliases(attach-detach-aliases), 13
- diffDays360(date_functions), 58
- divCoef, 82
- divCoef(), 12, 97, 134, 187, 197
- entropy, 83, 151
- entropy(), 105, 191
- etaSq, 84
- etaSq(), 28, 30, 101, 158, 184
- expFreq, 87
- expFreq(), 93, 94, 170, 201
- extremes, 88, 178
- factor, 190
- findCorrX, 89
- findCorrX(), 43, 45, 111, 124, 166, 195
- fm, 170, 200
- format, 58
- formula, 91, 94
- Formulas, 79, 91
- freq, 91, 144, 146, 208
- freq(), 87, 94, 170, 201
- freq2D, 93
- freq2D(), 87, 93, 170, 201
- friedman.test, 125
- fTable, 170
- generation, 95
- generation(), 7, 47, 58, 62, 198, 213
- getConcepts(concepts), 33
- getOption, 189
- gini, 96, 105
- gini(), 12, 83, 134, 187, 197
- gkGamma, 10, 35
- gkGamma(ordAssocs), 159
- gkTau, 10, 97
- gkTau(), 42, 49, 132, 151, 172, 203, 207, 212
- glassDelta, 28, 100

- glassDelta(), 28, 30, 86, 158, 184
- glm, 157
- gmean, 102
- gmean(), 107, 108, 113, 145, 147, 148, 205
- gsd (gmean), 102
- hasVaryingTime (date-time-predicates), 57
- herfindahl, 104
- herfindahl(), 84, 191
- highLow (extremes), 88
- hist, 91, 93
- hmean, 105
- hmean(), 104, 108, 113, 145, 147, 148, 205
- hmsToMinute (time-conversions), 197
- hmsToSec (time-conversions), 197
- hodesLehmann, 107
- hodesLehmann(), 104, 107, 113, 145, 147, 148, 205
- hoeffdingD, 10, 109
- hoeffdingD(), 43, 45, 90, 124, 166, 195
- hour (date_functions), 58
- huber, 112, 113
- huberM, 112
- huberM(), 104, 107, 108, 145, 147, 148, 205
- icc, 8, 114
- icc(), 24, 32, 51, 122, 128, 165, 168, 180
- impute, 116
- impute(), 118
- imputeKnn, 117
- imputeKnn(), 117
- inherits, 58
- integrate, 15
- IQR, 119, 139
- iqrX, 119, 139
- iqrX(), 26, 139, 142, 144, 181, 208
- isConfusionTable, 120, 154, 155
- isDate (date-time-predicates), 57
- isDateTime (date-time-predicates), 57
- isLeapYear (date_functions), 58
- isTime (date-time-predicates), 57
- isWeekend (date_functions), 58
- kappaM, 8, 121
- kappaM(), 24, 32, 51, 115, 128, 165, 168, 180
- keepSig, 123
- keepSig(), 43, 45, 90, 111, 166, 195
- kendallTauA, 35
- kendallTauA (ordAssocs), 159
- kendallTauB, 10, 35, 111
- kendallTauB (ordAssocs), 159
- kendallW, 8, 124
- kendallW(), 35, 53, 161
- krippAlpha, 8, 126
- krippAlpha(), 24, 32, 51, 115, 122, 165, 168, 180
- kruskal.test, 73, 76
- kurt, 129
- kurt(), 193
- label, 5
- lambda, 10, 99, 131
- lambda(), 42, 49, 99, 151, 172, 203, 207, 212
- large (extremes), 88
- lastDayOfMonth (date_functions), 58
- lc, 132
- lc(), 12, 83, 97, 187, 197
- leveneTest, 76
- lm, 71
- lof, 135, 135, 136
- log, 138
- log10, 138
- logSt, 136
- logSt(), 18, 20, 188, 210
- logStInv (logSt), 136
- lumen::adfTest, 78
- lumen::corTest, 124
- lumen::fisherZ, 166
- lumen::fisherZInv, 166
- lumen::kpssTest, 78
- lumen::leveneTest, 73
- lumen::madCI(), 139
- lumen::meanCI, 144
- mad, 113, 138, 139, 142, 188
- madCI, 139
- madX, 138
- madX(), 26, 119, 142, 144, 181, 208
- mae, 140, 186
- mae(), 15, 22, 141, 150, 152, 153, 186, 194
- mape, 141
- mape(), 15, 22, 140, 150, 152, 153, 186, 194
- max, 89, 181
- mean, 140, 150, 152, 153
- mean.POSIXct, 145
- meanAD, 142
- meanAD(), 26, 119, 139, 144, 181, 208
- meanCI, 106
- meanSE, 143
- meanSE(), 26, 119, 139, 142, 181, 208
- meanX, 26, 28, 130, 142, 144, 193
- meanX(), 104, 107, 108, 113, 147, 148, 205
- median, 139
- medianX, 119, 145, 178
- medianX(), 104, 107, 108, 113, 145, 148, 205

- `min`, [89](#), [181](#)
- `minute`(`date_functions`), [58](#)
- `model.frame`, [64](#), [91](#), [169](#)
- `modelX`, [147](#)
- `modelX()`, [104](#), [107](#), [108](#), [113](#), [145](#), [147](#), [205](#)
- `month`(`date_functions`), [58](#)
- `month()`, [7](#)
- `monthDays`(`date_functions`), [58](#)
- `mse`, [149](#), [186](#)
- `mse()`, [15](#), [22](#), [140](#), [141](#), [152](#), [153](#), [186](#), [194](#)
- `mutInf`, [10](#), [150](#)
- `mutInf()`, [42](#), [49](#), [99](#), [132](#), [172](#), [203](#), [207](#), [212](#)
-
- `NA`, [208](#)
- `na.pass`, [133](#)
- `names`, [177](#)
- `nmae`, [152](#)
- `nmae()`, [15](#), [22](#), [140](#), [141](#), [150](#), [153](#), [186](#), [194](#)
- `nmse`, [153](#)
- `nmse()`, [15](#), [22](#), [140](#), [141](#), [150](#), [152](#), [186](#), [194](#)
- `normalizeToConfusion`, [41](#), [48](#), [131](#), [154](#)
- `normalizeToConfusion()`, [17](#), [182](#)
- `now`(`date_functions`), [58](#)
-
- `oddsRatio`, [14](#), [156](#), [176](#)
- `oddsRatio()`, [28](#), [30](#), [86](#), [101](#), [184](#)
- `optim`, [44](#), [45](#)
- `optimize`, [19](#)
- `options`, [189](#)
- `options(width)`, [4](#)
- `ordAssocs`, [34](#), [35](#), [159](#)
- `ordAssocs()`, [35](#), [53](#), [126](#)
- `order`, [110](#)
- `ordered`, [34](#)
- `outlier`, [162](#)
- `outlier()`, [5](#)
-
- `pabak`, [163](#)
- `pabak()`, [24](#), [32](#), [51](#), [115](#), [122](#), [128](#), [168](#), [180](#)
- `pairApply`, [9](#), [32](#)
- `pairApply()`, [155](#)
- `pearsonCor`, [165](#), [172](#)
- `pearsonCor()`, [43](#), [45](#), [90](#), [111](#), [124](#), [195](#)
- `percAgreement`, [167](#), [179](#)
- `percAgreement()`, [24](#), [32](#), [51](#), [115](#), [122](#), [128](#), [165](#), [180](#)
- `percTable`, [168](#)
- `percTable()`, [87](#), [93](#), [94](#), [201](#)
- `pharos::plot.Desc.qn`, [73](#)
- `pharos::plot.Desc.table`, [77](#), [176](#)
- `pharos::plot.Lc`, [134](#)
- `pharos::plotCor`, [49](#), [124](#)
- `pharos::plotTimeSeries`, [78](#)
-
- `pharos::plotWeb`, [124](#)
- `pharos::style`, [170](#)
- `phi`, [98](#), [171](#)
- `phi()`, [42](#), [49](#), [99](#), [132](#), [151](#), [203](#), [207](#), [212](#)
- `plot`, [66](#), [74](#)
- `plot.Conf`(`conf`), [36](#)
- `plot.Desc`(`desc`), [63](#)
- `plot.Desc.nn`(`desc.nn`), [70](#)
- `plot.Desc.nq`(`desc.nq`), [72](#)
- `plot.Desc.qq`(`print.Desc.qq`), [174](#)
- `plot.Desc.ts`(`desc.ts`), [77](#)
- `plotCatDist`, [69](#)
- `plotCor`, [123](#)
- `plotPropCI`, [66](#)
- `plotWeb`, [123](#)
- `pmvnorm`, [45](#)
- `predict`, [22](#)
- `predict.Lc`(`lc`), [132](#)
- `pretty`, [94](#)
- `print.Abstract`(`abstract`), [4](#)
- `print.BlandAltman`, [172](#)
- `print.Conf`(`conf`), [36](#)
- `print.Desc`(`desc`), [63](#)
- `print.Desc.Date`, [67](#), [173](#)
- `print.Desc.factor`(`desc.factor`), [68](#)
- `print.Desc.nn`(`desc.nn`), [70](#)
- `print.Desc.nq`(`desc.nq`), [72](#)
- `print.Desc.qn`(`desc.qn`), [75](#)
- `print.Desc.qq`, [174](#)
- `print.Desc.qq()`, [66](#), [67](#), [69](#), [71](#), [73](#), [74](#), [76–78](#)
- `print.Desc.table`(`print.Desc.qq`), [174](#)
- `print.Desc.ts`(`desc.ts`), [77](#)
- `print.Freq`(`freq`), [91](#)
- `print.OddsRatio`(`oddsRatio`), [156](#)
- `print.PercTable`(`percTable`), [168](#)
- `print.table`, [94](#)
- `print.tOne`(`tOne`), [198](#)
- `print.ym`(`as_ym`), [10](#)
- `prop.table`, [93](#)
- `proportions`, [170](#)
-
- `quantile`, [56](#), [119](#), [147](#), [162](#), [177](#), [178](#)
- `quantileCI`, [178](#)
- `quantileX`, [119](#), [145](#), [177](#)
- `quantileX()`, [89](#)
- `quarter`(`date_functions`), [58](#)
-
- `randolphKappa`, [179](#)
- `randolphKappa()`, [24](#), [32](#), [51](#), [115](#), [122](#), [128](#), [165](#), [168](#)
- `range`, [181](#)
- `rangeX`, [180](#), [188](#)

- rangeX(), 26, 119, 139, 142, 144, 208
- rank, 89, 110
- raterFrame, 155, 179, 181
- raterFrame(), 17, 155
- Rd_db, 33
- relRisk, 14, 158, 183
- relRisk(), 28, 30, 86, 101, 158
- revX, 184
- rmse, 185
- rmse(), 15, 22, 140, 141, 150, 152, 153, 194
- rosenbluth, 186
- rosenbluth(), 12, 83, 97, 134, 197
- sapply, 106
- scale, 188
- scaleX, 187
- scaleX(), 18, 20, 138, 210
- sd, 208
- sdX, 26, 130, 193
- sdX (varX), 207
- second (date_functions), 58
- secToHms (time-conversions), 197
- sensX (conf), 36
- seq, 55
- set.seed, 40, 53, 97, 108, 113, 115, 130
- setDescToolsXOption, 170, 189
- sfsmisc::printTable2, 170
- simpson, 190
- simpson(), 84, 105
- skew, 191
- skew(), 130
- small (extremes), 88
- smape, 193
- smape(), 15, 22, 140, 141, 150, 152, 153, 186
- solve, 43
- somersDelta, 10, 35
- somersDelta (ordAssocs), 159
- sort, 89
- spearmanCor, 111, 194
- spearmanCor(), 43, 45, 90, 111, 124, 166
- specX (conf), 36
- splinefun, 15
- stats::acf, 78
- stats::Box.test, 78
- stats::chisq.test, 176
- stats::cor, 126
- stats::cor.test, 124
- stats::fisher.test, 176
- stats::friedman.test, 126
- strptime, 62
- stuartTauC, 10, 35
- stuartTauC (ordAssocs), 159
- style, 170
- sum, 153
- summary, 66
- sweep, 188
- t, 184
- t.test, 39
- table, 9, 31, 41, 44, 48, 83, 91–94, 98, 131, 154, 157, 164, 170, 171, 183, 203, 206
- terms, 91
- theil, 196
- theil(), 12, 83, 97, 134, 187
- time interval, 144
- time-conversions, 197
- times, 198
- timezone (date_functions), 58
- today (date_functions), 58
- tOne, 198
- tOne(), 87, 93, 94, 170
- ts, 19
- tschuprowT, 202
- tschuprowT(), 42, 49, 99, 132, 151, 172, 207, 212
- tukeyBiweight, 113, 204
- tukeyBiweight(), 104, 107, 108, 113, 145, 147, 148
- uncertCoef, 10, 206
- uncertCoef(), 42, 49, 99, 132, 151, 172, 203, 212
- uniroot, 184
- utils::str(), 5
- var, 139, 207, 208
- varCI, 208
- varTest, 208
- varX, 28, 207
- varX(), 26, 119, 139, 142, 144, 181
- vcd::table2d_summary, 170
- week (date_functions), 58
- weekday (date_functions), 58
- weighted.mean, 145
- wilcox.test, 108
- year (date_functions), 58
- year(), 7
- yearDay (date_functions), 58
- yearDays (date_functions), 58
- yearMonth (date_functions), 58
- yearWeek (date_functions), 58
- yeoJohnson, 209
- yeoJohnson(), 18, 20, 138, 188

yeoJohnsonInv (yeoJohnson), [209](#)
yule, [42](#), [49](#), [99](#), [132](#), [151](#), [172](#), [203](#), [207](#), [211](#)
yuleQ (yule), [211](#)
yuleY (yule), [211](#)

zodiac, [213](#)
zodiac(), [7](#), [47](#), [58](#), [62](#), [95](#), [198](#)